



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ECA09-DSP LIST OF EXPERIMENTS

LIST OF EXPERIMENTS	DURATION (EXPT CONDUCTION + EVALUATION)	CO	BL	WK	TYPE OF EXPT
1.Butterworth IIR Filters 1.1. Design and Analysis of a Butterworth IIR Low-Pass Filter Using MATLAB. 1.2. Design and Analysis of a Butterworth IIR High-Pass Filter Using MATLAB.	1.5 HOURS	CO1	BL3 BL4 BL6	WK4 WK5	SIMULATION
2.Chebyshev IIR Filters 2.1 Design and performance Evaluation of a Chebyshev Type I IIR Low Pass Filter using MATLAB 2.2 Design and performance Evaluation of a Chebyshev Type I IIR High Pass Filter using MATLAB	1.5 HOURS	CO1	BL3 BL4 BL6	WK4 WK5	SIMULATION
3. Performance Analysis of IIR Filters. 3.1 Design and Comparative Analysis of Butterworth and Chebyshev Type-I IIR Band-Pass Filters Using MATLAB	1 HOURS	CO1	BL3 BL4 BL6	WK4 WK5	SIMULATION
4. FIR Low Pass FilterS using Windowing Techniques 4.1. Design and Analysis of an FIR Low-Pass Filter Using the Rectangular Window Technique in	1.5 HOURS	CO2	BL3 BL4 BL6	WK4 WK5	SIMULATION

MATLAB. 4.2. Design and Analysis of an FIR Low-Pass Filter Using the Hamming Window Technique in MATLAB. . . 4.3. Design and Analysis of an FIR Low-Pass Filter Using the Hanning Window Technique in MATLAB.					
5. FIR High Pass FilterS using Windowing Techniques 5.1. Design and Analysis of an FIR High-Pass Filter Using the Rectangular Window Technique in MATLAB. 5.2. Design and Analysis of an FIR High-Pass Filter Using the Hamming Window Technique in MATLAB. . 5.3. Design and Analysis of an FIR High-Pass Filter Using the Hanning Window Technique in MATLAB.	1.5 HOURS	CO2	BL3 BL4 BL6	WK4 WK5	SIMULATION
6. Finite Word length Effects 6.1 Uniform Quantization and Quantization Error Analysis of a Discrete-Time Sinusoidal Signal Using MATLAB 6.2 Analysis of Finite Word Length Effects in Digital Signal Representation Using Truncation and Rounding Techniques in MATLAB	1 HOURS	C03	BL3 BL4 BL5	WK4	SIMULATION
7. Spectral Estimation 7.1 Power Spectral Density Estimation and Spectral Analysis of Noisy Sinusoidal Signals Using Periodogram Method in MATLAB	1 HOURS	CO3	BL3 BL4 BL5	WK4	SIMULATION

7.2 Analysis of Windowing Effects on Spectral Estimation Using Rectangular, Hamming, and Hanning Windows in MATLAB					
8. Multirate Signal Processing 8.1 Implementation and Analysis of Decimation and Interpolation in Multirate Signal Processing Using MATLAB 8.2 Polyphase Implementation of Decimation and Interpolation Filters Using MATLAB	1 HOURS	CO4	BL3 BL4 BL5	WK4 WK5	SIMULATION
9.Adaptive Filtering for Echo Cancellation 9.1. Implementation and Performance Analysis of LMS and NLMS Algorithms for Adaptive Echo Cancellation	1 HOURS	CO5	BL4 BL5 BL6	WK5 WK6	SIMULATION
10.Digital Speech Processing and Noise Reduction 10.1. Implementation of Spectral Subtraction for Speech Noise Reduction	1 HOURS	C05	BL4 BL5 BL6	WK5 WK6	SIMULATION
11. IIR Filters using DSP Processor 11.1 Design and Implementation of an IIR Low-Pass Filter using DSP Processor	1.5 HOURS	C05	BL4 BL5 BL6	WK5	HARDWARE
12. FIR Filters using Window Techniques on DSP Processor 12.1 Design and Implementation of FIR High Pass filter using Rectangular Window	1.5 HOURS	C05	BL4 BL5 BL6	WK6	HARDWARE
Total Duration	15 Hours				



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ECA09-DIGITAL SIGNAL PROCESSING

LAB MANUAL

ECA09-LIST OF EXPERIMENTS

S.No	List of Experiments	Tools
Expt 1	Butterworth IIR Filters	MATLAB
	1.1. Design and Analysis of a Butterworth IIR Lowh-Pass Filter Using MATLAB. 1.2. Design and Analysis of a Butterworth IIR High-Pass Filter Using MATLAB.	
Expt 2	Chebyshev IIR Filters	MATLAB
	2.1 Design and performance Evaluation of a Chebyshev Type I IIR Low Pass Filter using MATLAB 2.2 Design and performance Evaluation of a Chebyshev Type I IIR High Pass Filter using MATLAB	
Expt 3	Performance Analysis of IIR Filters Coefficients.	MATLAB
	3.1 Design and Comparative Analysis of Butterworth and Chebyshev Type-I IIR Band-Pass Filters Using MATLAB	
Expt 4	FIR Low Pass Filters using Windowing Techniques	MATLAB
	4.1.Design and Analysis of an FIR Low-Pass Filter Using the Rectangular Window Technique in MATLAB. 4.2. Design and Analysis of an FIR Low-Pass Filter Using the Hamming Window Technique in MATLAB. . . 4.3. Design and Analysis of an FIR Low-Pass Filter Using the Hanning Window Technique in MATLAB.	
Expt 5	FIR High Pass Filters using Windowing Techniques	
	5.1. Design and Ana lysis of an FIR High-Pass Filter Using the Rectangular Window Technique in MATLAB. 5.2. Design and Analysis of an FIR High-Pass Filter Using the Hamming Window Technique in MATLAB. . 5.3. Design and Analysis of an FIR High-Pass Filter Using the Hanning Window Technique in MATLAB.	MATLAB
Expt 6	Finite Word length Effects	MATLAB
	6.1 Uniform Quantization and Quantization Error Analysis of a Discrete-Time Sinusoidal Signal Using MATLAB 6.2 Analysis of Finite Word Length Effects in Digital Signal Representation Using Truncation and Rounding Techniques in MATLAB	
Expt 7	Spectral Estimation	MATLAB
	7.1 Power Spectral Density Estimation and Spectral Analysis of Noisy Sinusoidal Signals Using Periodogram Method in MATLAB 7.2 Analysis of Windowing Effects on Spectral Estimation Using Rectangular, Hamming, and Hanning Windows in MATLAB	
Expt 8	8.Multirate Signal Processing	MATLAB
	8.1 Implementation and Analysis of Decimation and Interpolation in Multirate Signal Processing Using MATLAB	

	8.2 Polyphase Implementation of Decimation and Interpolation Filters Using MATLAB	
Expt 9	Adaptive Filtering for Echo Cancellation	MATLAB
	9.1. Implementation and Performance Analysis of LMS and NLMS Algorithms for Adaptive Echo Cancellation	
Expt 10	Digital Speech Processing and Noise Reduction	MATLAB
	10.1. Implementation of Spectral Subtraction for Speech Noise Reduction	
Expt 11	Digital Filter Implementation using DSP Processor	HARDWARE
	11.1 Design and Implementation of an IIR Band-Pass Filter using DSP Processor	
Expt 12	Audio Signal Processing using DSP Processor	HARDWARE
	12.1 Implementation of FIR Filtering for Audio Signal Processing using TMS320C6713 DSP Processor	

EX NO:1	Butterworth IIR Filters
DATE:	1.1. Design and Analysis of a Butterworth IIR Low-Pass Filter Using MATLAB.

Objectives

1. To determine the minimum filter order and cutoff frequency, required to satisfy the design constraints.
2. To design and analyze the magnitude response, phase response, characteristics of the Butterworth IIR low-pass filter for the given passband and stopband specifications.
3. To analyze the impulse response, step response and group delay characteristics of the Butterworth IIR low-pass filter for the given passband and stopband specifications.
4. To examine the pole-zero plot and verify the stability and performance of the designed low-pass filter.

Procedure

1. Open MATLAB and create a new script file in the MATLAB Editor.
2. Define the filter specifications such as passband ripple (R_p), stopband attenuation (R_s), passband frequency (F_p), stopband frequency (F_{sb}), and sampling frequency (F_s).
3. Calculate the normalized passband and stopband frequencies with respect to the Nyquist frequency ($F_s/2$).
4. Use the buttord() function to determine the minimum order (N) and normalized cutoff frequency (W_n) required for the Butterworth low-pass filter.
5. Design the Digital IIR Butterworth Low-Pass Filter using the butter() function and obtain the numerator (b) and denominator (a) filter coefficients.
6. Display the filter order and cutoff frequency obtained from the design process.
7. Compute the frequency response of the designed filter using the freqz() function.
8. Plot the magnitude response of the filter and observe the passband characteristics and stopband attenuation.
9. Plot the phase response of the filter and analyze the phase variation over the frequency range.
10. Generate the pole-zero plot using the zplane() function to verify the stability of the designed IIR filter.
11. Create a test signal consisting of low-frequency and high-frequency components.
12. Apply the designed IIR low-pass filter to the test signal using the filter() function.

Program (Matlab Code)

```
clc; clear
```



```

Fs=10000; Fp=1000; Fst=1500;
[N,Wn]=buttord(Fp/(Fs/2),Fst/(Fs/2),1,60);
[b,a]=butter(N,Wn);

freqz(b,a)          % Magnitude & Phase

figure
[h,n]=impz(b,a,50);
...title('Impulse Response')

figure
stepz(b,a)

figure
zplane(b,a)

fprintf('Minimum Filter Order (N) = %d\n',N);
fprintf('Cutoff Frequency (Wn) = %.4f\n',Wn); Input:

```

- Passband Ripple (R_p) = 1 dB
- Stopband Attenuation (R_s) = 20 dB
- Passband Frequency (F_p) = 1000 Hz
- Stopband Frequency (F_s) = 3000 Hz
- Sampling Frequency ($F_{sampling}$) = 8000 Hz
- Filter Choice = Low-Pass Filter (LPF)

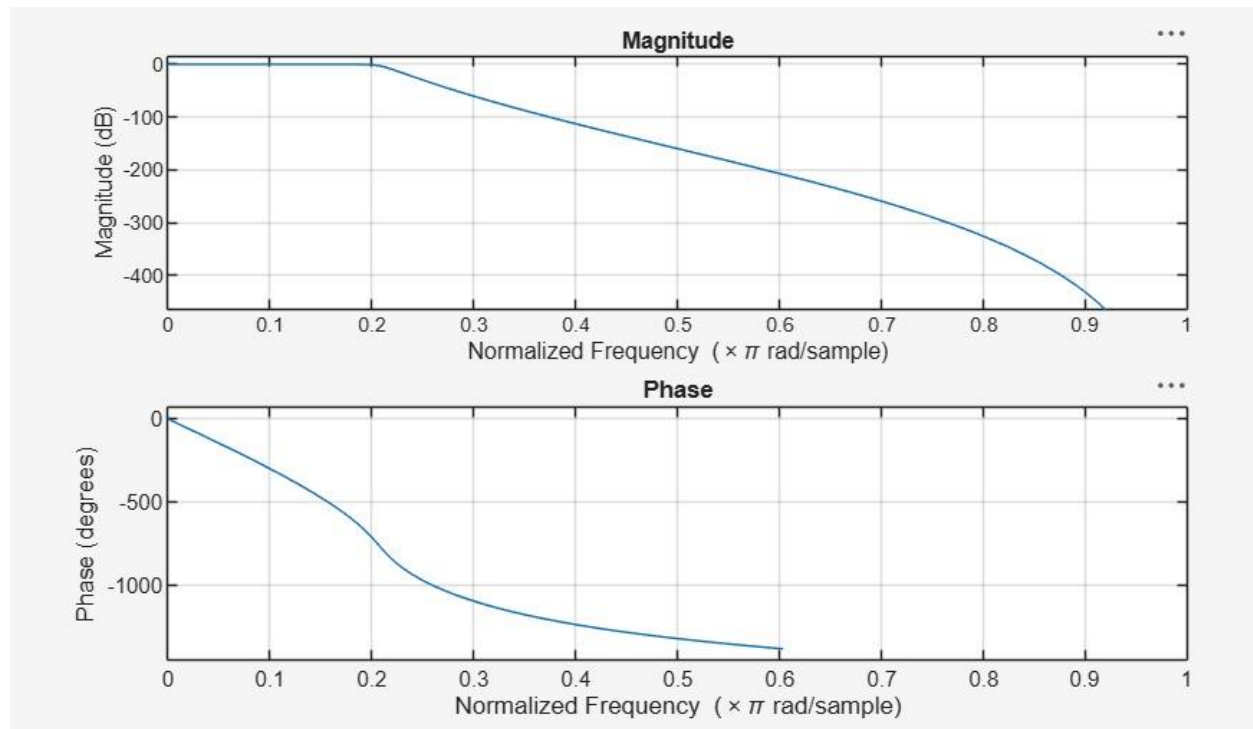
Output:

Objective 1:

Minimum Filter Order (N) = 17

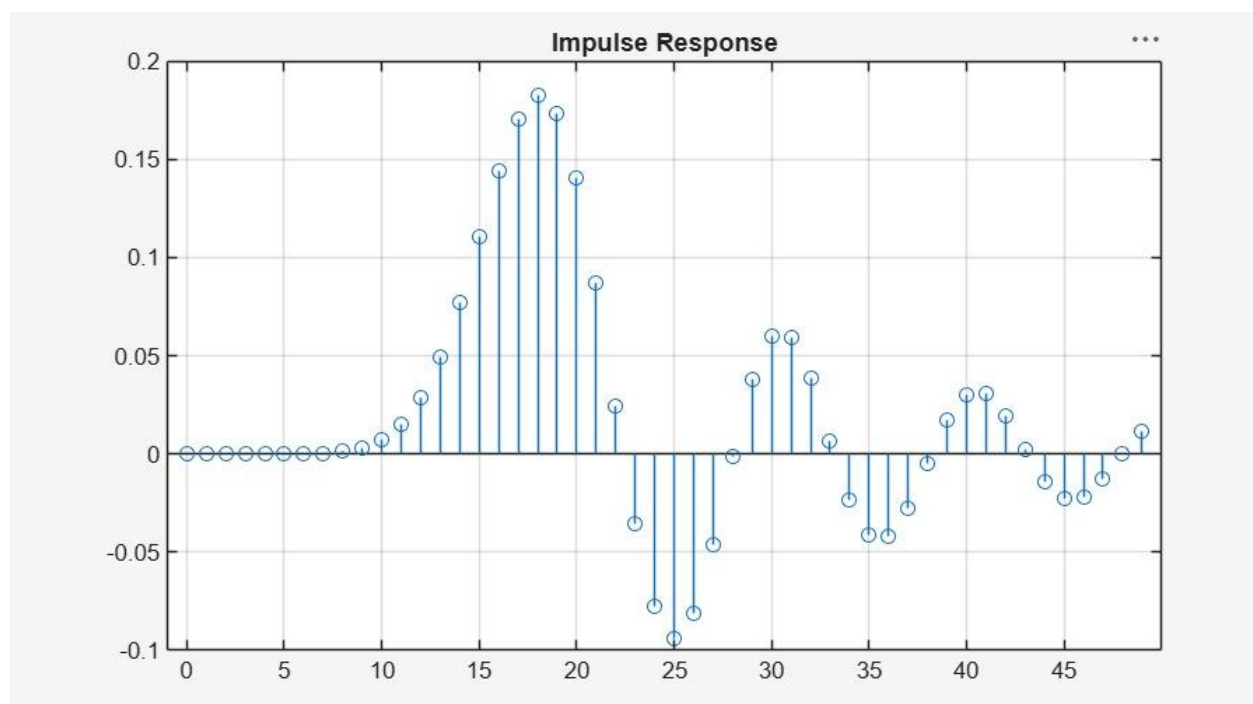
Cutoff Frequency (Wn) = 0.2083

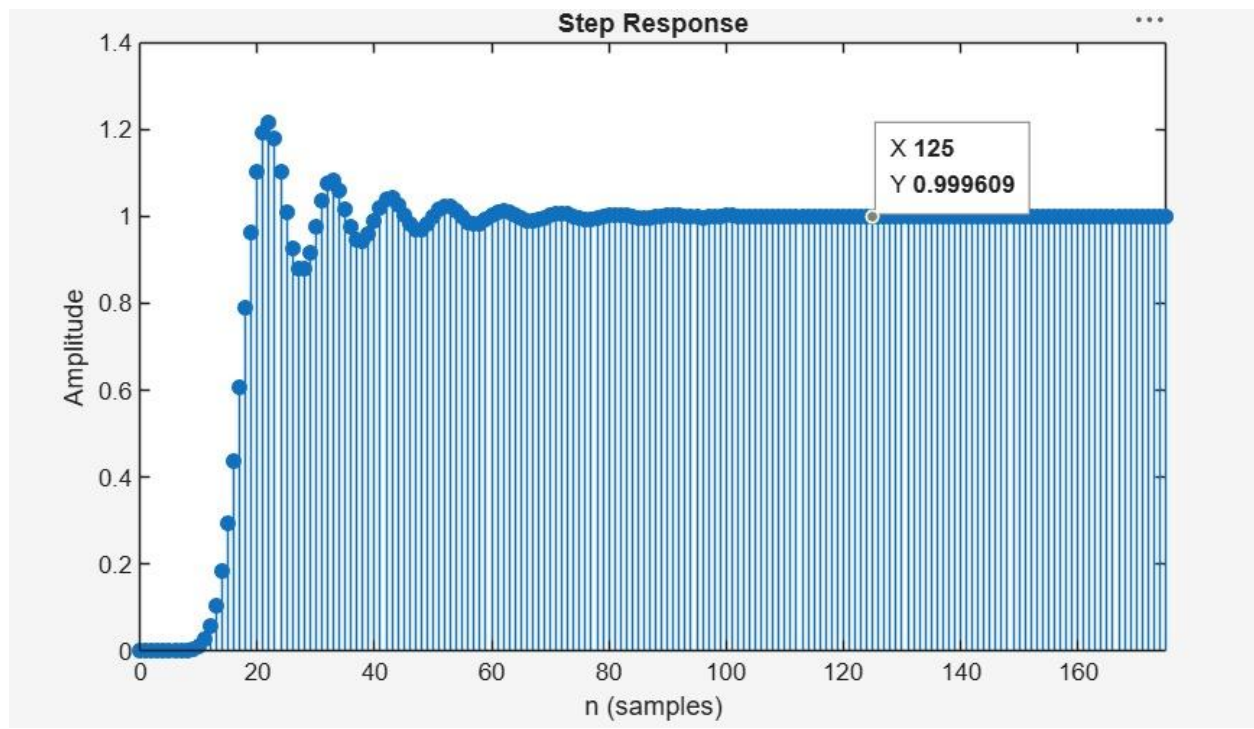
Objective 2:



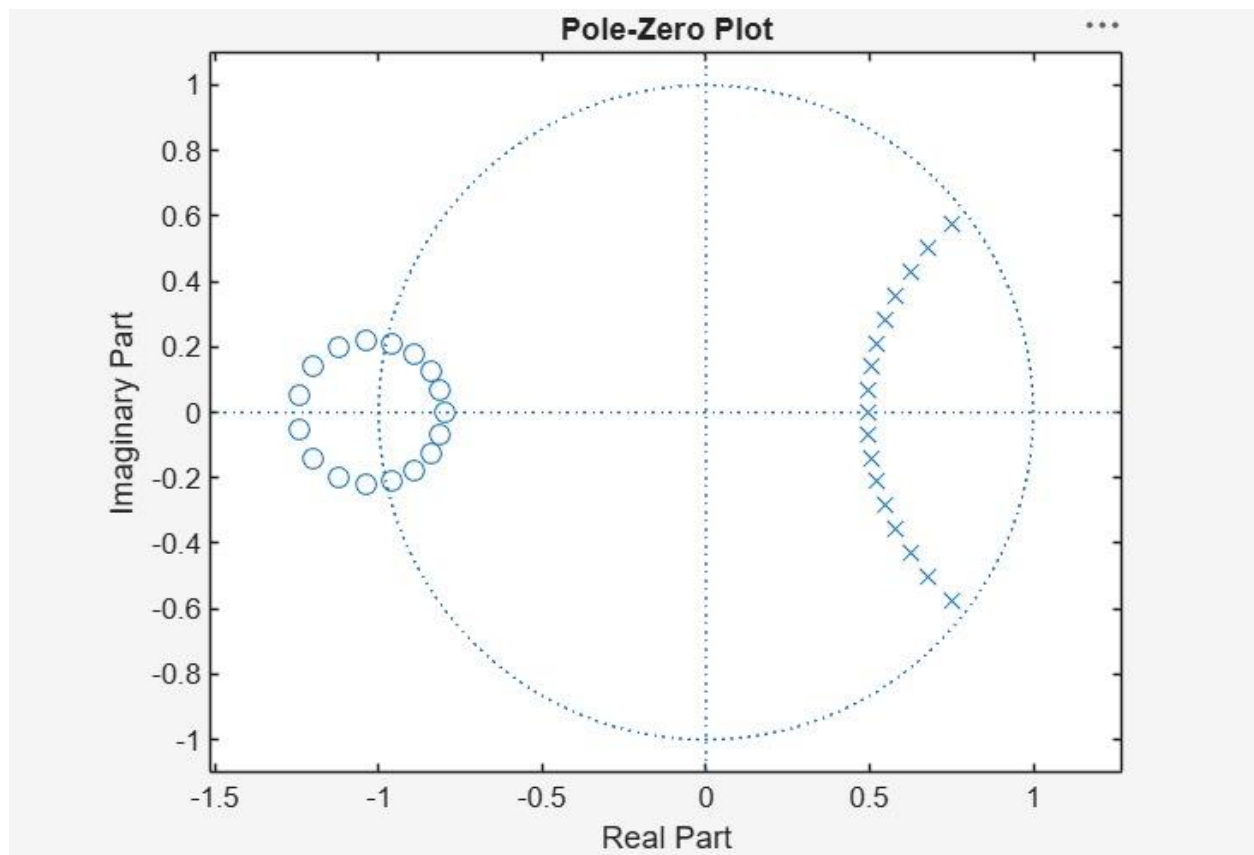
.

Objective 3:





Objective 4:



Result:

- 1.The minimum filter order and cutoff frequency satisfying the given specifications were determined successfully.
- 2.The magnitude and phase responses of the Butterworth IIR low-pass filter were obtained and analyzed.
- 3.The impulse response, step response, and group delay characteristics were evaluated successfully.
4. The pole-zero plot confirmed the stability and proper performance of the designed filter.

Practical Question – 1

Design a **digital Butterworth IIR Low-Pass Filter** using MATLAB to meet the following specifications: Passband ripple of **1 dB**, minimum stopband attenuation of **40 dB**, passband edge frequency of **1000 Hz**, stopband edge frequency of **1500 Hz**, and sampling frequency of **8000 Hz**. Determine the minimum filter order and cutoff frequency required to satisfy the given specifications. Obtain and analyze the magnitude and phase responses of the designed filter. Also, generate the pole-zero plot and verify the stability and performance characteristics of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question – 2

Design a **digital Butterworth IIR Low-Pass Filter** using MATLAB for a signal processing application with a passband ripple of **0.5 dB**, stopband attenuation of **50 dB**, passband edge frequency of **1200 Hz**, stopband edge frequency of **1800 Hz**, and sampling frequency of **10000 Hz**. Calculate the minimum filter order and corresponding cutoff frequency required to satisfy the design requirements. Plot the magnitude and phase responses of the filter and evaluate its frequency-selective characteristics. Generate the pole-zero plot and comment on the stability and effectiveness of the designed low-pass filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

EX NO:1	Butterworth IIR Filters
DATE:	1.2. Design and Analysis of a Butterworth IIR High-Pass Filter Using MATLAB.

Objective

1. To determine the minimum filter order and cutoff frequency, required to satisfy the design constraints.
2. To design and analyze the magnitude response, phase response, characteristics of the Butterworth IIR High-pass filter for the given passband and stopband specifications.
3. To analyze the impulse response, step response and group delay characteristics of the Butterworth IIR High-pass filter for the given passband and stopband specifications.
4. To examine the pole-zero plot and verify the stability and performance of the designed High-pass filter.

Procedure

1. Open MATLAB and create a new script file.
2. Enter the filter specifications: passband ripple (R_p), stopband attenuation (R_s), passband frequency (F_p), stopband frequency (F_s), and sampling frequency ($F_{sampling}$).
3. Normalize the passband and stopband frequencies with respect to the Nyquist frequency ($F_{sampling}/2$).
4. Use the buttord() function to determine the minimum filter order and cutoff frequency for the Butterworth high-pass filter.
5. Design the IIR Butterworth high-pass filter using the butter() function and obtain the numerator and denominator coefficients.
6. Display the filter order and cutoff frequency.
7. Compute the frequency response of the filter using the freqz() function.
8. Plot the magnitude response and verify that low-frequency components are attenuated while high-frequency components are passed.
9. Plot the phase response and analyze the phase characteristics.
10. Generate the pole-zero plot using the zplane() function and verify that all poles lie inside the unit circle.
11. Create a test signal consisting of low-frequency and high-frequency sinusoidal components.
12. Apply the designed high-pass filter to the test signal using the filter() function.
13. Plot and compare the input and filtered output signals.
14. Analyze the filter performance in terms of frequency selectivity and stability.
15. Verify that the designed filter satisfies the given specifications.

Program (Matlab Code)

```
clc; clear
```

```

Fs=10000; Fp=3000; Fst=2000;
[N,Wn]=buttord(Fp/(Fs/2),Fst/(Fs/2),1,60);
[b,a]=butter(N,Wn,'high');

% Magnitude & Phase
freqz(b,a)

% Impulse Response
figure
[h,n]=impz(b,a,50);
stem(n,h), grid on
title('Impulse Response')

% Step Response
figure
stepz(b,a)
title('Step Response')

% Group Delay
figure
grpdelay(b,a)
title('Group Delay')

% Pole-Zero Plot
figure
zplane(b,a)
title('Pole-Zero Plot')

fprintf('Minimum Filter Order (N) = %d\n',N);
fprintf('Cutoff Frequency (Wn) = %.4f\n',Wn);

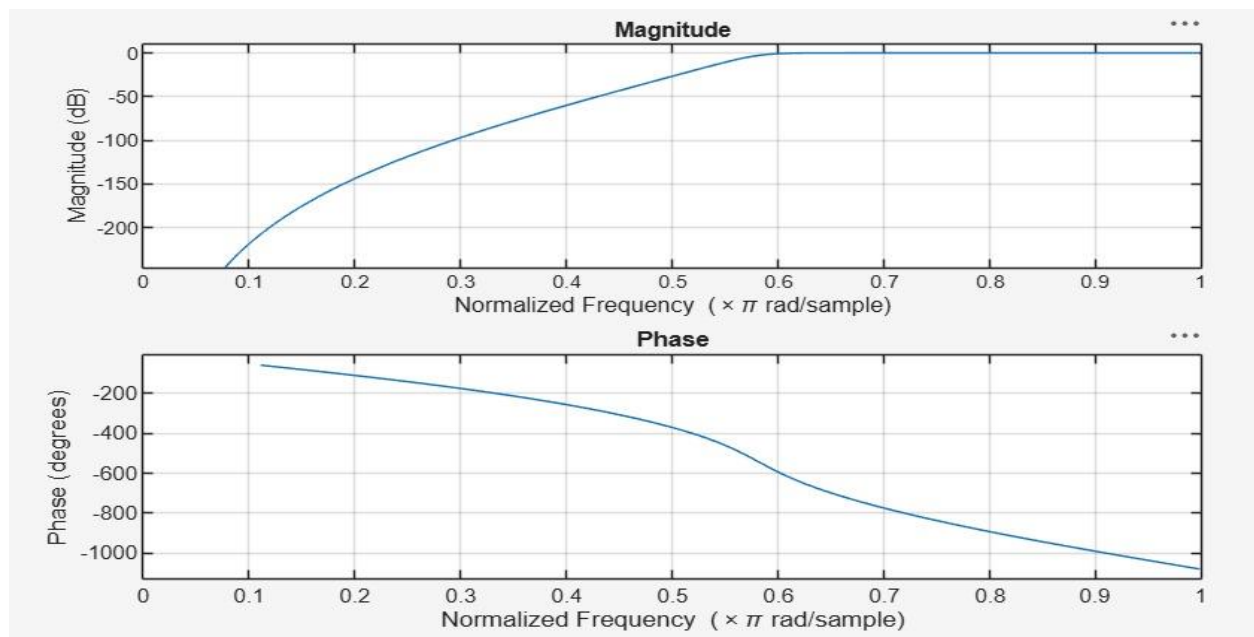
```

Output:

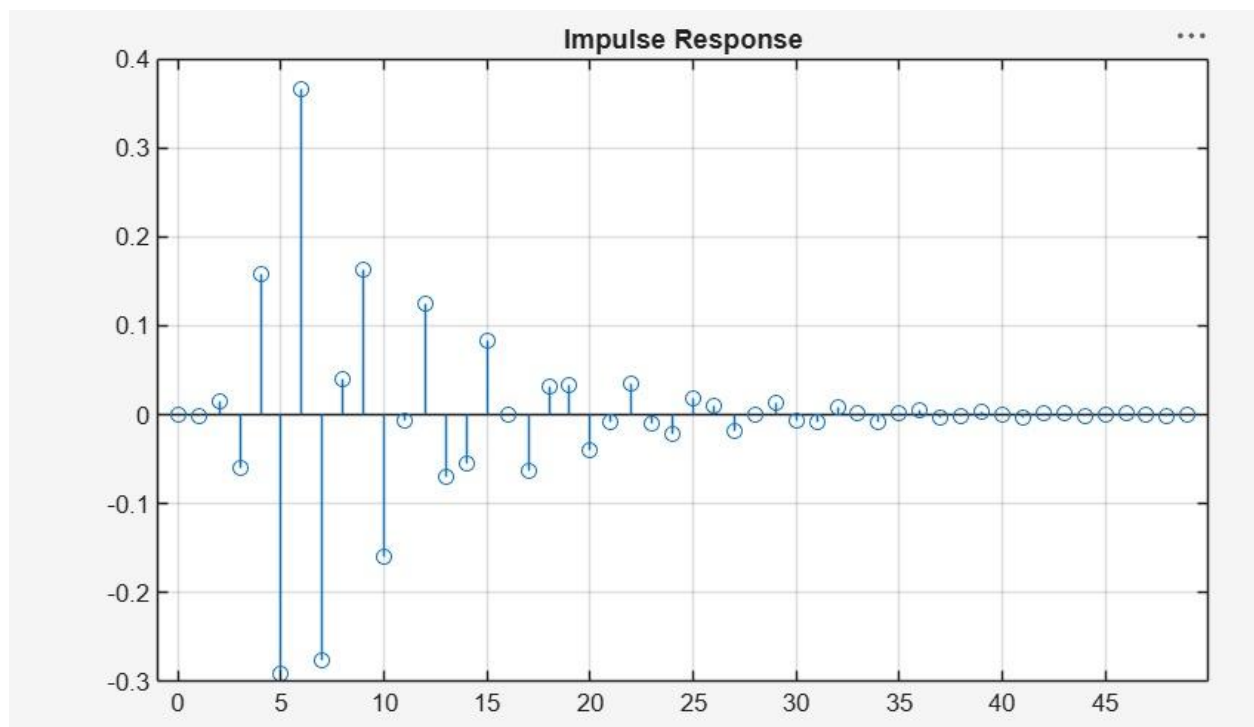
Objective 1:

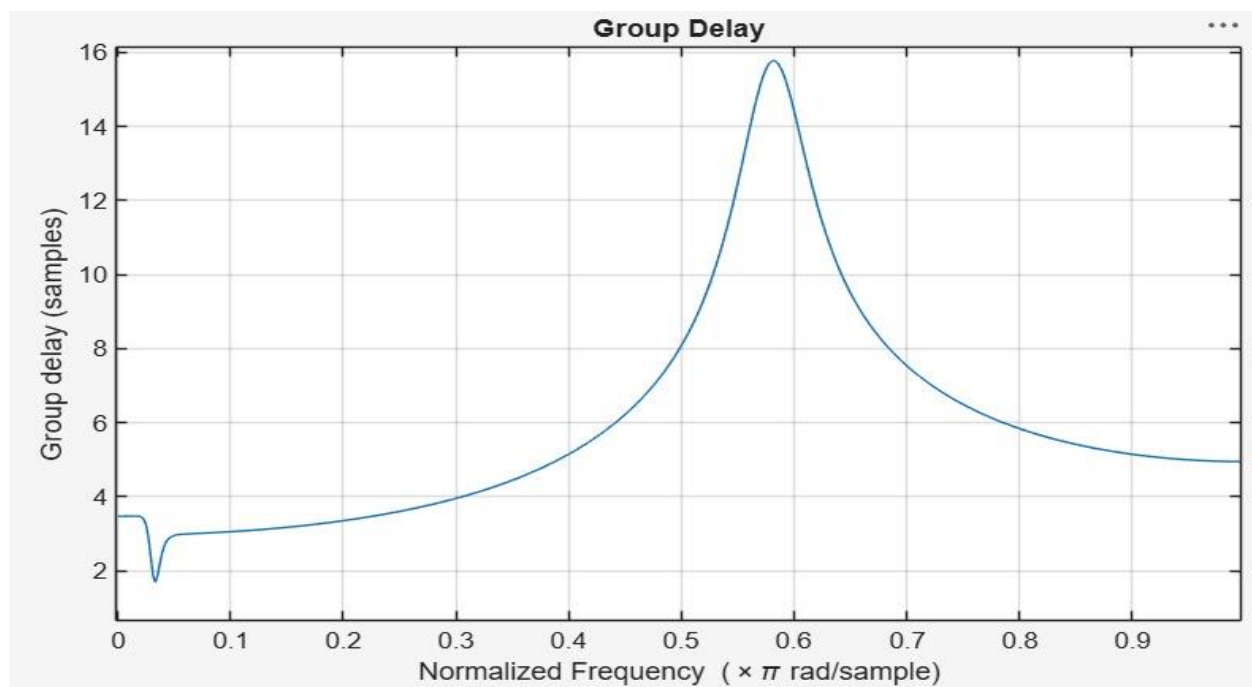
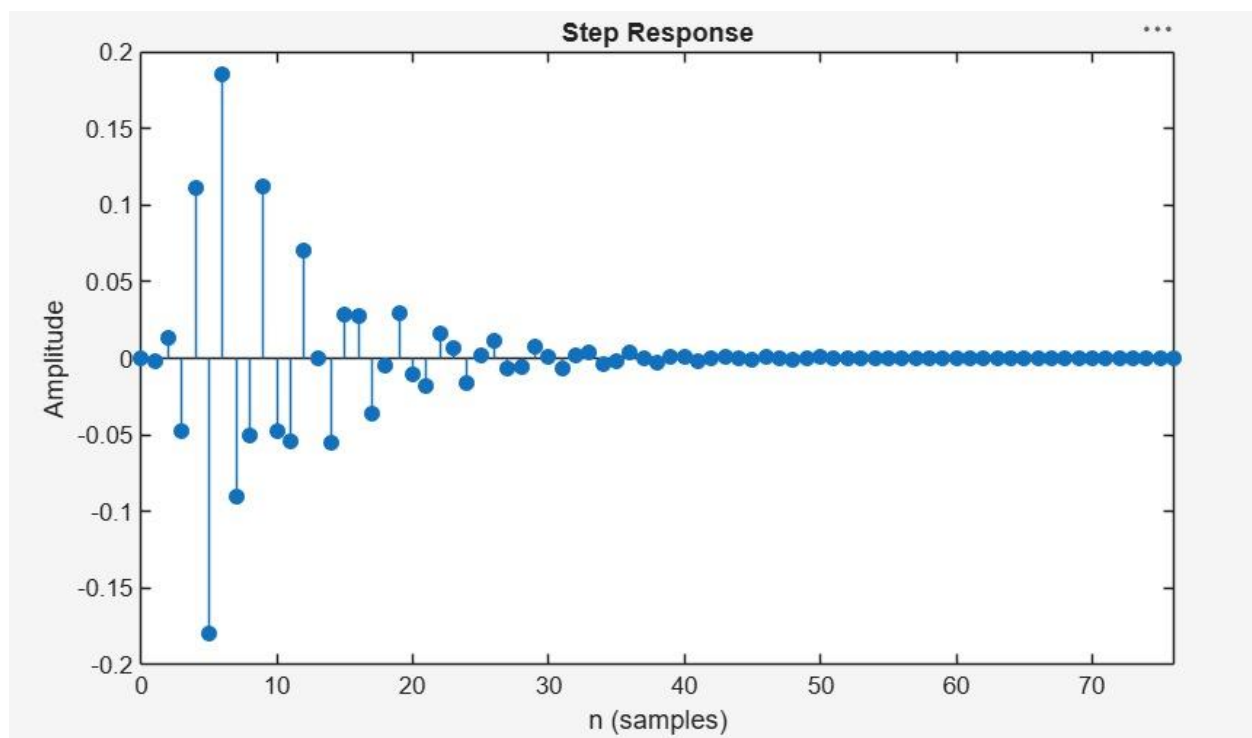
Minimum Filter Order (N) = 12
Cutoff Frequency (Wn) = 0.5807

Objective 2:

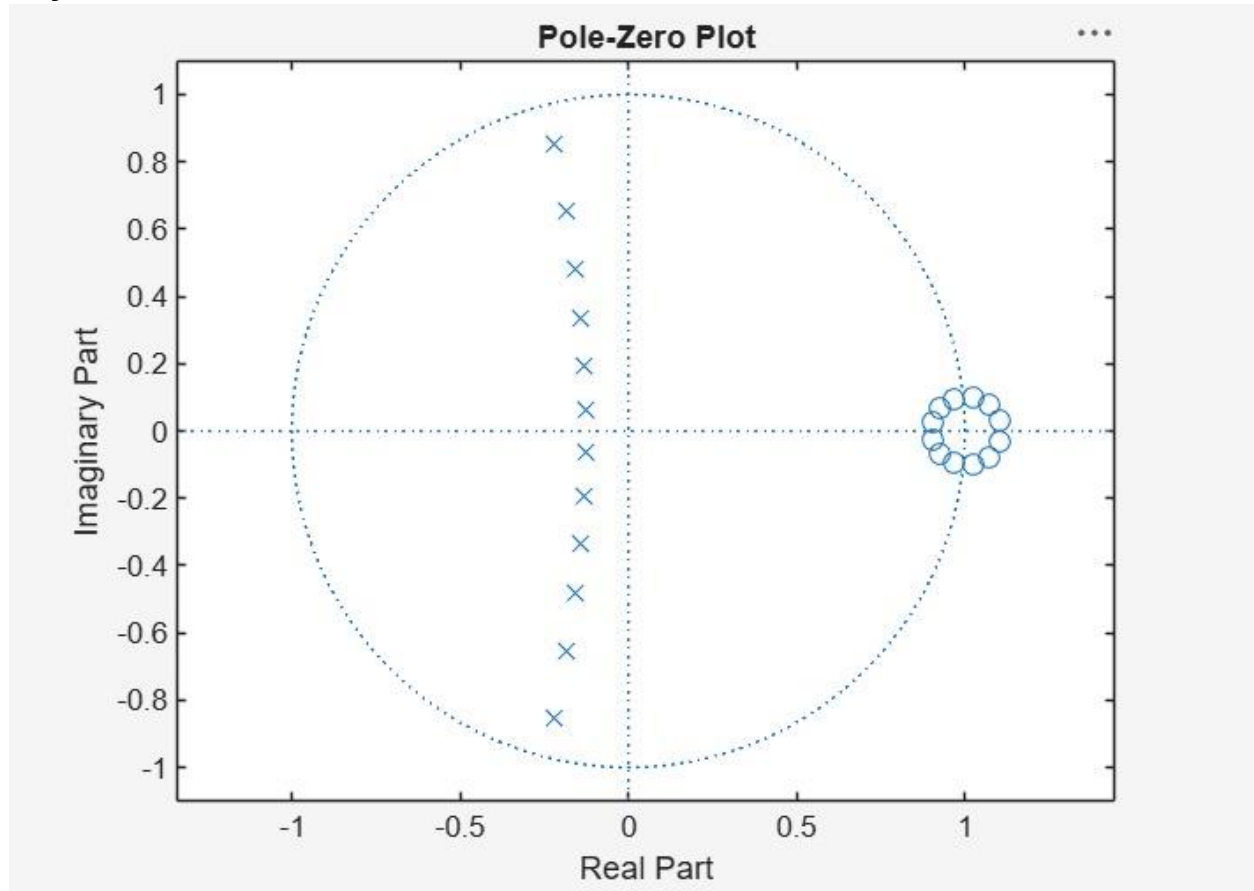


Objective 3:





Objective 4:



Result:

1. The minimum filter order and cutoff frequency satisfying the given specifications were determined successfully.
2. The magnitude and phase responses of the Butterworth IIR High-pass filter were obtained and analyzed.
3. The impulse response, step response, and group delay characteristics were evaluated successfully.
4. The pole-zero plot confirmed the stability and proper performance of the designed filter.

Practical Question 1

Question:

Design a digital Butterworth IIR High-Pass Filter in MATLAB for the following specifications:

- Sampling Frequency, $F_s = 8000\text{Hz}$
- Passband Edge Frequency, $F_p = 2000\text{Hz}$
- Stopband Edge Frequency, $F_s = 1200\text{Hz}$
- Passband Ripple, $A_p = 1\text{dB}$
- Stopband Attenuation, $A_s = 40\text{dB}$

Tasks:

1. Determine the minimum filter order and cutoff frequency.
2. Design the Butterworth IIR High-Pass Filter and obtain the filter coefficients.

3. Plot and analyze the magnitude and phase responses.
4. Obtain the pole-zero plot and verify the stability of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

Question:

Design a digital Butterworth IIR High-Pass Filter in MATLAB for the following specifications:

- Sampling Frequency, $F_s = 10000\text{Hz}$
- Passband Edge Frequency, $F_p = 3000\text{Hz}$
- Stopband Edge Frequency, $F_s = 1800\text{Hz}$
- Passband Ripple, $A_p = 0.5\text{dB}$
- Stopband Attenuation, $A_s = 50\text{dB}$

Tasks:

1. Calculate the minimum filter order and cutoff frequency using the Butterworth design method.
2. Generate the numerator and denominator coefficients of the filter.
3. Plot the magnitude response and phase response of the designed filter.
4. Analyze the pole-zero plot and comment on the stability and performance of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

EX NO:2	Chebyshev IIR Filter
DATE:	2.1. Design and Performance Evaluation of a Chebyshev Type-I IIR Low-Pass Filter using MATLAB

Objectives:

1. To design and analyze the magnitude response, phase response, impulse response, and group delay characteristics of the Chebyshev Type-I IIR low-pass filter for the given passband and stopband specifications.
2. To analyze the impulse response, and group delay characteristics of the Chebyshev Type-I IIR low-pass filter for the given passband and stopband specifications.
3. To examine the pole-zero plot and verify the stability and performance of the designed low-pass filter.
4. To determine the minimum filter order and cutoff frequency required to satisfy the design constraints.

Procedure

1. Open MATLAB and create a new script file.
2. Define the filter specifications such as passband ripple (R_p), stopband attenuation (R_s), passband frequency (F_p), stopband frequency (F_{sb}), and sampling frequency (F_s) for the Chebyshev Type-I filter design.
3. Normalize the passband and stopband frequencies with respect to the Nyquist frequency ($F_s/2$).
4. Use the `cheb1ord()` function to determine the minimum order and cutoff frequency of the Chebyshev Type-I low-pass and high-pass filters.
5. Design the Chebyshev Type-I low-pass and high-pass filters using the `cheby1()` function and obtain the filter coefficients.
6. Display the filter order and cutoff frequency, and compute the frequency response of the designed filters using the `freqz()` function.
7. Plot the magnitude response, phase response, impulse response, group delay response, and pole-zero plot using the appropriate MATLAB functions to analyze the filter characteristics and stability.
8. Execute the program, analyze the obtained responses and filter characteristics, record the results, and verify that the designed Chebyshev Type-I filters satisfy the given specifications

Program (MATLAB Code)

```
clc; clear;
```

```
Wp=0.4; Ws=0.55; Rp=1; As=40;
```

```

[N,Wn]=cheb1ord(Wp,Ws,Rp,As);
[b,a]=cheby1(N,Rp,Wn);

fprintf('Filter Order = %d\n',N);
fprintf('Cutoff Frequency = %.2f * pi rad/sample\n',Wn);

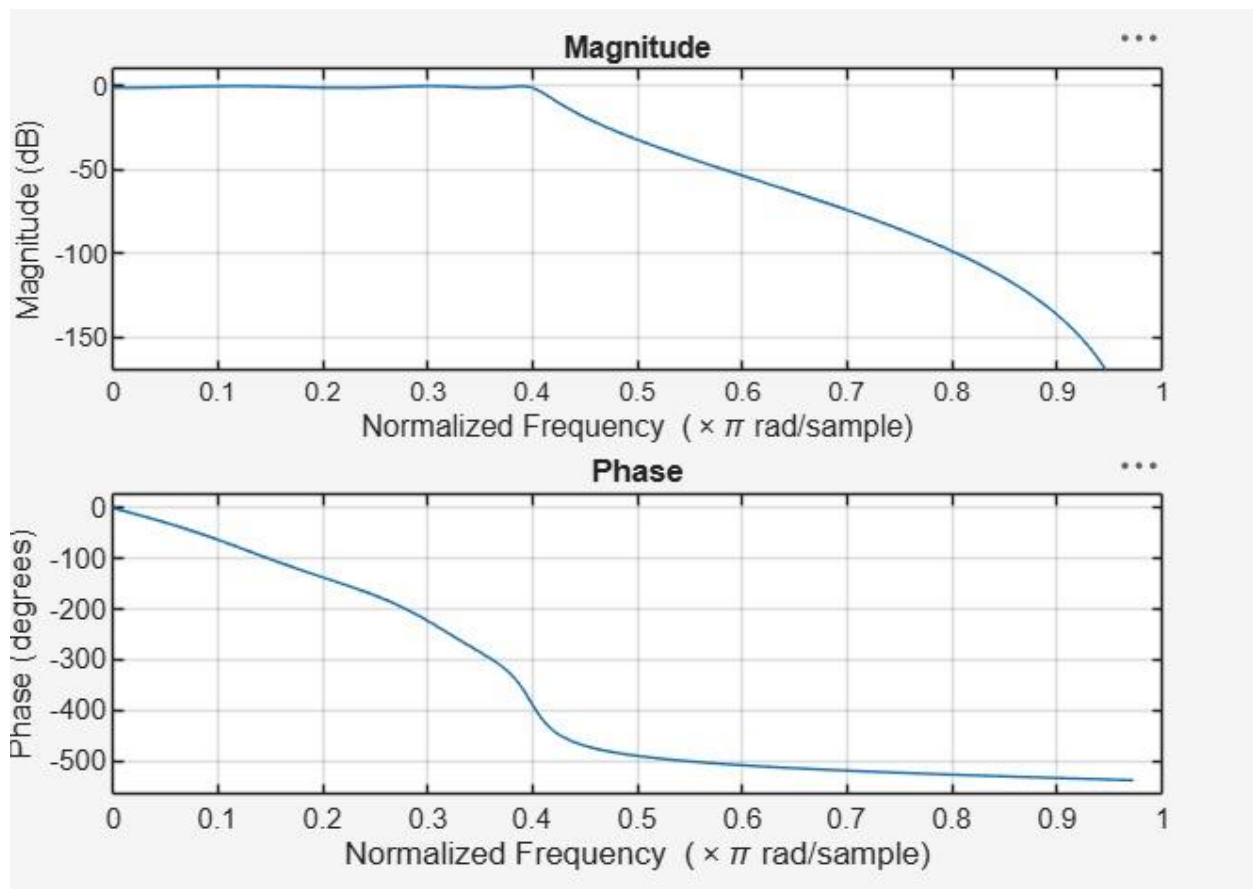
if all(abs(roots(a))<1)
    disp('System is STABLE');
else
    disp('System is UNSTABLE');
end

% ----- GRAPHS -----
figure; freqz(b,a);      % Magnitude + Phase
figure; impz(b,a);       % Impulse Response
figure; grpdelay(b,a);   % Group Delay
figure; zplane(b,a);     % Pole-Zero

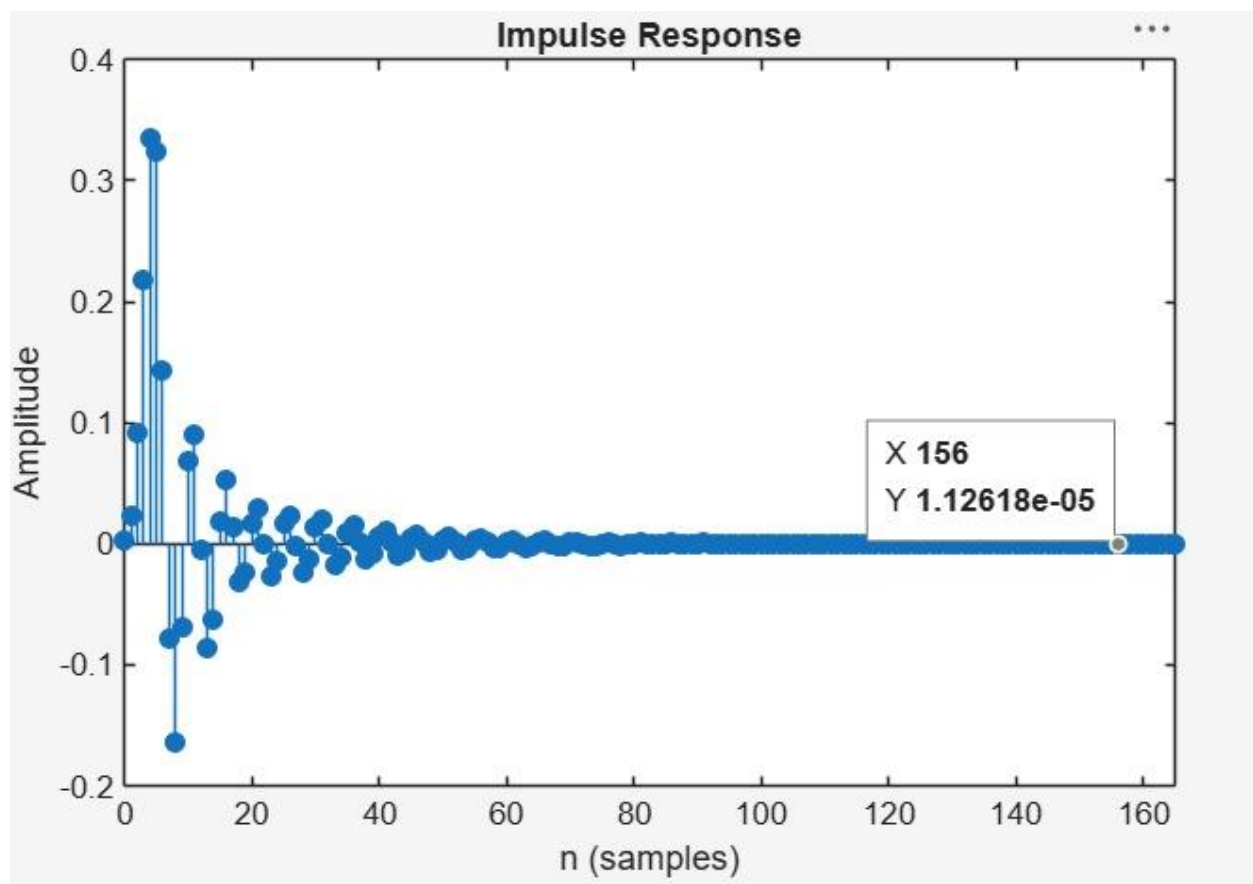
```

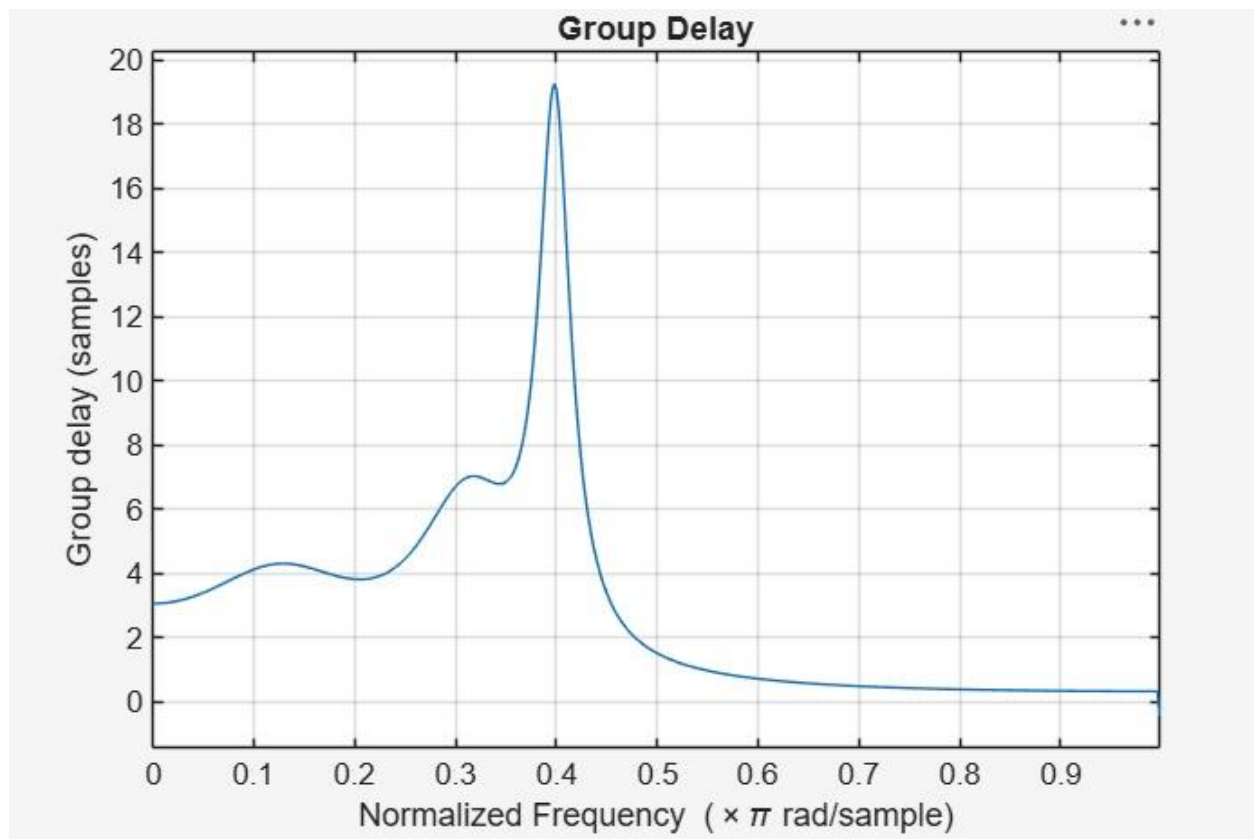
Output:

Objective 1:

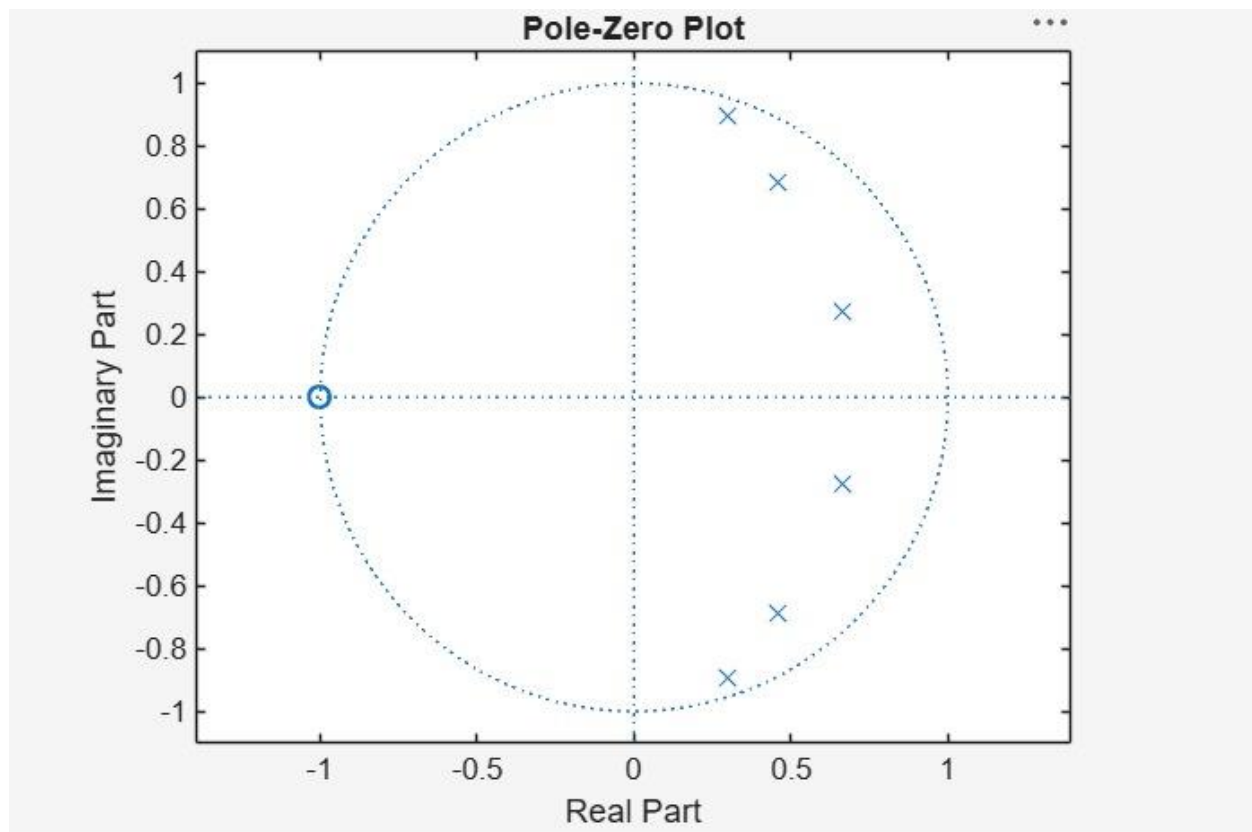


Objective 2:





Objective 3:



Objective 4:

Filter Order = 6

Cutoff Frequency = $0.40 * \pi$ rad/sample

System is STABLE

Result:

1. The Chebyshev Type-I IIR low-pass filter was successfully designed, and its magnitude response, phase response, impulse response, and group delay were analyzed.
2. The impulse response and group delay characteristics were obtained and found to satisfy the given specifications.
3. The pole-zero plot confirmed that all poles lie inside the unit circle, verifying the stability and proper performance of the filter.
4. The minimum filter order and cutoff frequency were successfully determined and found to satisfy the given design specifications.

Practical Question – 1

Design a digital **Chebyshev Type-I IIR Low-Pass Filter** using MATLAB to meet the following specifications: passband ripple of **1 dB**, minimum stopband attenuation of **40 dB**, passband edge frequency of **1000 Hz**, stopband edge frequency of **2000 Hz**, and sampling frequency of **8000 Hz**.

Task:

1. Determine the minimum filter order and cutoff frequency required to satisfy the given specifications.
2. Obtain and analyze the magnitude response, phase response.
3. Obtain and analyze impulse response, and group delay characteristics of the designed filter.
4. (iv) generate the pole-zero plot and verify the stability and performance characteristics of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question – 2

Design a digital Chebyshev Type-I IIR Low-Pass Filter using MATLAB to meet the following specifications: passband ripple of 1 dB, minimum stopband attenuation of 50 dB, passband edge frequency of 2500 Hz, stopband edge frequency of 1500 Hz, and sampling frequency of 8000 Hz.

- (i) Determine the minimum filter order and cutoff frequency required to satisfy the given specifications.
- (ii) Obtain and analyze the magnitude response and phase response of the designed filter.
- (iii) Obtain and analyze the impulse response and group delay characteristics of the designed filter.
- (iv) Generate the pole-zero plot and verify the stability and performance characteristics of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

EX NO:2	Chebyshev IIR Filter
DATE:	2.2. Design and Performance Evaluation of a Chebyshev Type-I IIR High-Pass Filter

Objectives:

1. To design and analyze the magnitude response, phase response, impulse response, and group delay characteristics of the Chebyshev Type-I IIR High-pass filter for the given passband and stopband specifications.
2. To analyze the impulse response, and group delay characteristics of the Chebyshev Type-I IIR high-pass filter for the given passband and stopband specifications.
3. To examine the pole-zero plot and verify the stability and performance of the designed high-pass filter.
4. To determine the minimum filter order and cutoff frequency required to satisfy the design constraints.

Procedure

1. Open MATLAB and specify the passband frequency, stopband frequency, passband ripple, stopband attenuation, and sampling frequency.
2. Normalize the frequencies with respect to the Nyquist frequency.
3. Determine the minimum filter order and cutoff frequency using `cheb1ord`.
4. Design the Chebyshev Type-I IIR High-Pass Filter using `cheby1`.
5. Obtain the filter coefficients and frequency response using `freqz`.
6. Plot and analyze the magnitude and phase responses.
7. Obtain and analyze the impulse response using `impz`.
8. Determine the group delay response using `grpdelay`.
9. Plot the pole-zero diagram using `zplane`.
10. Verify stability by ensuring that all poles lie inside the unit circle and record the observations.

Program (MATLAB Code)

```
clc;
```

```
clear;
```

```
close all;
```

```
% Specifications
```

```
Rp = 1;    % Passband Ripple (dB)
```

```
Rs = 40;   % Stopband Attenuation (dB)
```

```
Wp = 0.5;  % Passband Edge
```

```
Ws = 0.35; % Stopband Edge
```

```
% Find Filter Order
```

```
[n,Wn] = cheb1ord(Wp,Ws,Rp,Rs);
```

```
% Design High-Pass Filter
```

```
[b,a] = cheby1(n,Rp,Wn,'high');
```

```
% Frequency Response
```

```
figure;
```

```
freqz(b,a);
```

```
title('Frequency Response');
```

```
% Impulse Response
```

```
figure;
```

```
impz(b,a);
```

```
title('Impulse Response');
```

```
% Group Delay
```

```
figure;
```

```
grpdelay(b,a);
```

```
title('Group Delay');
```

```
% Pole-Zero Plot
```

```
figure;
```

```
zplane(b,a);
```

```
title('Pole-Zero Plot');
```

```
fprintf('Filter Order = %d\n',n);
```

```
fprintf('Cutoff Frequency = %.4f\n',Wn);
```

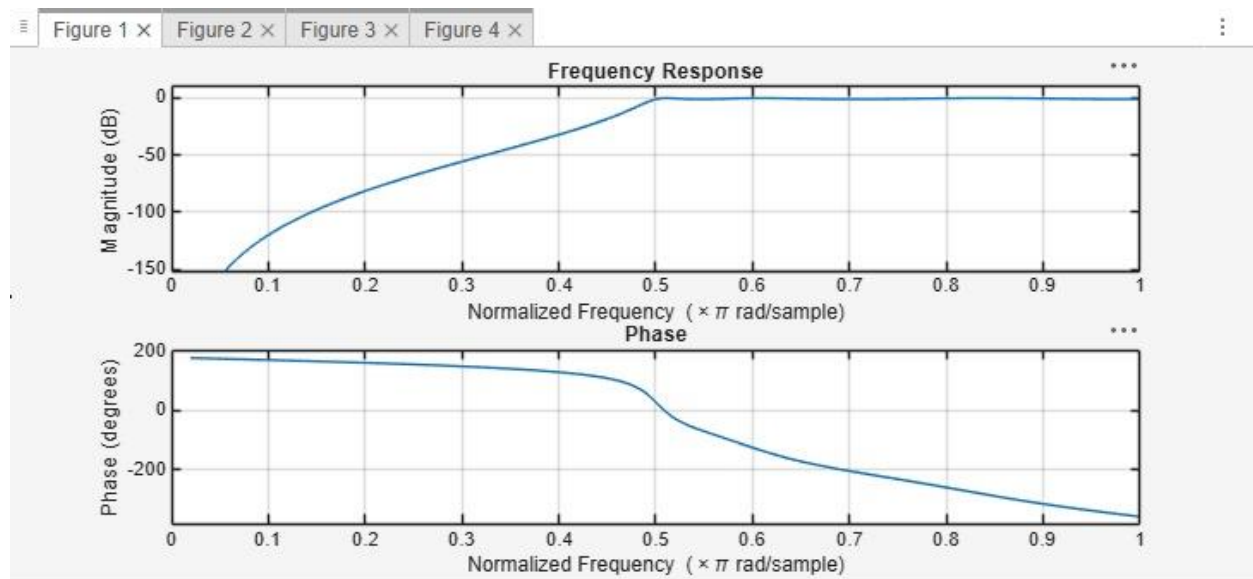
Output::

Objective-1:

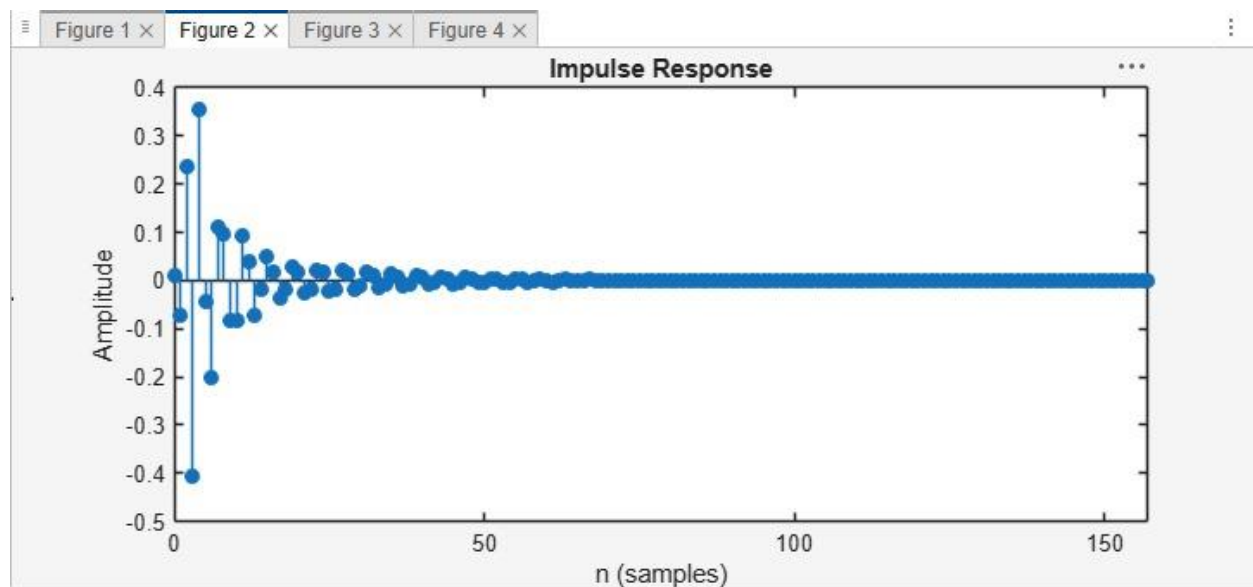
Filter Order = 6

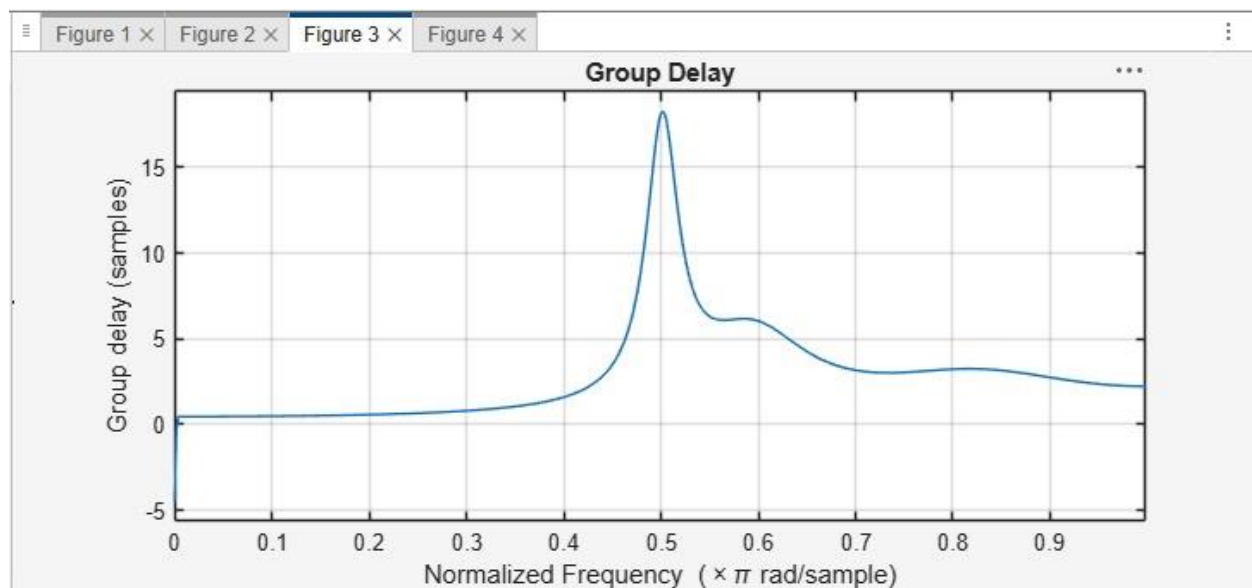
Cutoff Frequency = 0.5000

Objective 2:

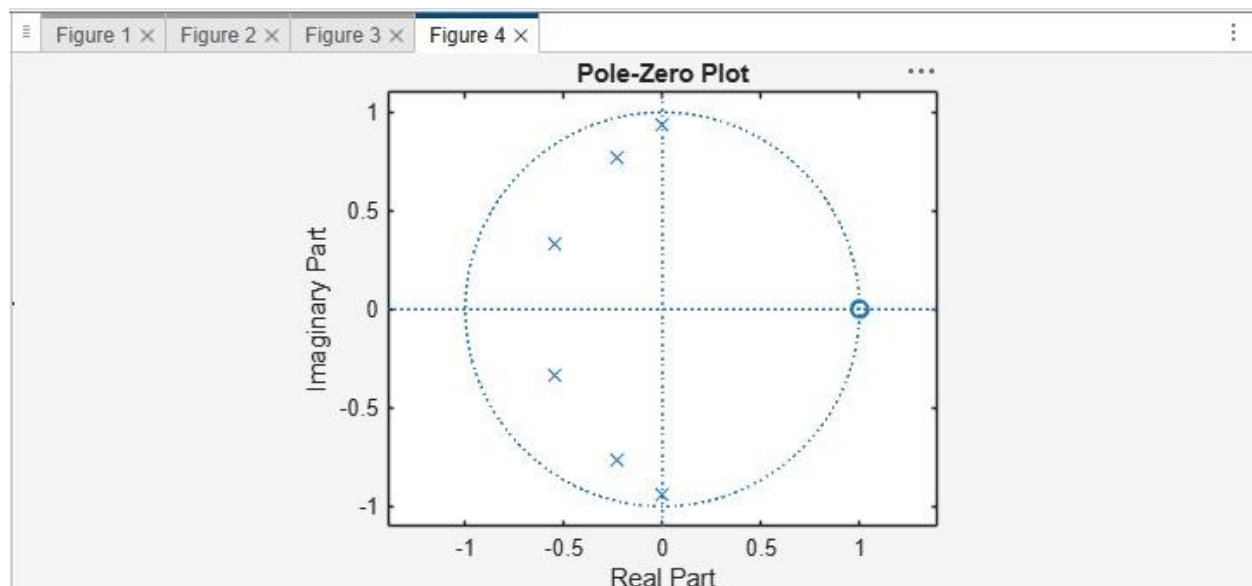


Objective 3:





Objective 4:



Result:

1. The Chebyshev Type-I IIR High-pass filter was successfully designed, and its magnitude response, phase response, impulse response, and group delay were analyzed.
2. The impulse response and group delay characteristics were obtained and found to satisfy the given specifications.
3. The pole-zero plot confirmed that all poles lie inside the unit circle, verifying the stability and proper performance of the filter.
4. The minimum filter order and cutoff frequency were successfully determined and found to satisfy the given design specifications.

Practical Question 1:

Design a digital **Chebyshev Type-I IIR High-Pass Filter** using MATLAB to meet the following specifications: passband ripple of **1 dB**, minimum stopband attenuation of **40 dB**, passband edge frequency of **2000 Hz**, stopband edge frequency of **1000 Hz**, and sampling frequency of **8000 Hz**.

1. Determine the minimum filter order and cutoff frequency required to satisfy the given specifications.
2. Obtain and analyze the magnitude response and phase response of the designed filter.
3. Obtain and analyze the impulse response and group delay characteristics of the designed filter.
4. Generate the pole-zero plot and verify the stability and performance characteristics of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

Design a digital **Chebyshev Type-I IIR High-Pass Filter** using MATLAB to meet the following specifications:

- Passband Ripple = **0.5 dB**
- Minimum Stopband Attenuation = **45 dB**
- Passband Edge Frequency = **3000 Hz**
- Stopband Edge Frequency = **2000 Hz**
- Sampling Frequency = **10000 Hz**

Tasks

1. Determine the minimum filter order and cutoff frequency required to satisfy the given specifications.
2. Obtain and analyze the magnitude response and phase response of the designed filter.
3. Obtain and analyze the impulse response and group delay characteristics of the designed filter.
4. Generate the pole-zero plot and verify the stability and performance characteristics of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

--	--	--	--	--	--	--

Expt 3	Performance Analysis of IIR Filters
Date	3.1 Design and Comparative Analysis of Butterworth and Chebyshev Type-I IIR Band-Pass Filters Using MATLAB

Objectives

1. To design Butterworth and Chebyshev Type-I IIR Band-Pass Filters in MATLAB for the given passband, stopband, ripple, attenuation, and sampling frequency specifications.
2. To determine the minimum filter order and corresponding cutoff frequencies and obtain the filter coefficients for both IIR filter designs.
3. To analyze and compare the frequency-domain characteristics of the Butterworth and Chebyshev Type-I filters using magnitude response, phase response, and group delay plots.
4. To evaluate the stability and performance of the designed filters using pole-zero plots, impulse responses, and pole magnitude analysis.

Procedure

1. Open MATLAB and create a new script file.
2. Define the filter specifications, including sampling frequency, passband frequencies, stopband frequencies, passband ripple, and stopband attenuation.
3. Normalize the passband and stopband frequencies with respect to the Nyquist frequency ($F_s/2$).
4. Use the buttord() function to determine the minimum order and cutoff frequencies of the Butterworth Band-Pass Filter.
5. Design the Butterworth Band-Pass Filter using the butter() function and obtain its numerator and denominator coefficients.
6. Use the cheb1ord() function to determine the minimum order and cutoff frequencies of the Chebyshev Type-I Band-Pass Filter.
7. Design the Chebyshev Type-I Band-Pass Filter using the cheby1() function and obtain its numerator and denominator coefficients.
8. Display the filter order, cutoff frequencies, and filter coefficients for both filters.

9. Plot and analyze the frequency responses of the Butterworth and Chebyshev Type-I filters using the `freqz()` function.
10. Compare the magnitude responses and phase responses of both filters using frequency response plots.
11. Generate pole-zero plots using the `zplane()` function and observe the pole and zero locations.
12. Obtain and compare the group delay characteristics using the `grpdelay()` function.
13. Plot the impulse responses of both filters using the `impz()` function.
14. Determine the pole magnitudes using the `roots()` function and verify filter stability by checking whether all poles lie inside the unit circle.
15. Analyze and compare the overall performance of the Butterworth and Chebyshev Type-I Band-Pass Filters based on their frequency responses, group delay, impulse response, and stability characteristics.

Program (Matlab)

```
clc; clear; close all;

% Specifications
Fs=2000; Rp=1; Rs=40;
Wp=[300 600]/(Fs/2);
Ws=[200 700]/(Fs/2);

% Butterworth Filter
[n1,W1]=buttord(Wp,Ws,Rp,Rs);
[b1,a1]=butter(n1,W1,'bandpass');

% Chebyshev Type-I Filter
[n2,W2]=cheb1ord(Wp,Ws,Rp,Rs);
[b2,a2]=cheby1(n2,Rp,W2,'bandpass');

% Magnitude Response
figure;
freqz(b1,a1); title('Butterworth Filter');
```



```

figure;
freqz(b2,a2); title('Chebyshev Type-I Filter');

% Group Delay
figure;
grpdelay(b1,a1); hold on;
grpdelay(b2,a2);
legend('Butterworth','Chebyshev-I');

% Pole-Zero Plot
figure;
subplot(121),zplane(b1,a1),title('Butterworth')
subplot(122),zplane(b2,a2),title('Chebyshev-I')

% Impulse Response
figure;
subplot(211),impz(b1,a1),title('Butterworth Impulse')
subplot(212),impz(b2,a2),title('Chebyshev Impulse')

% Pole Magnitude
figure;
subplot(121),stem(abs(roots(a1)),'filled')
title('Butterworth Pole Magnitude')

subplot(122),stem(abs(roots(a2)),'filled')
title('Chebyshev Pole Magnitude')

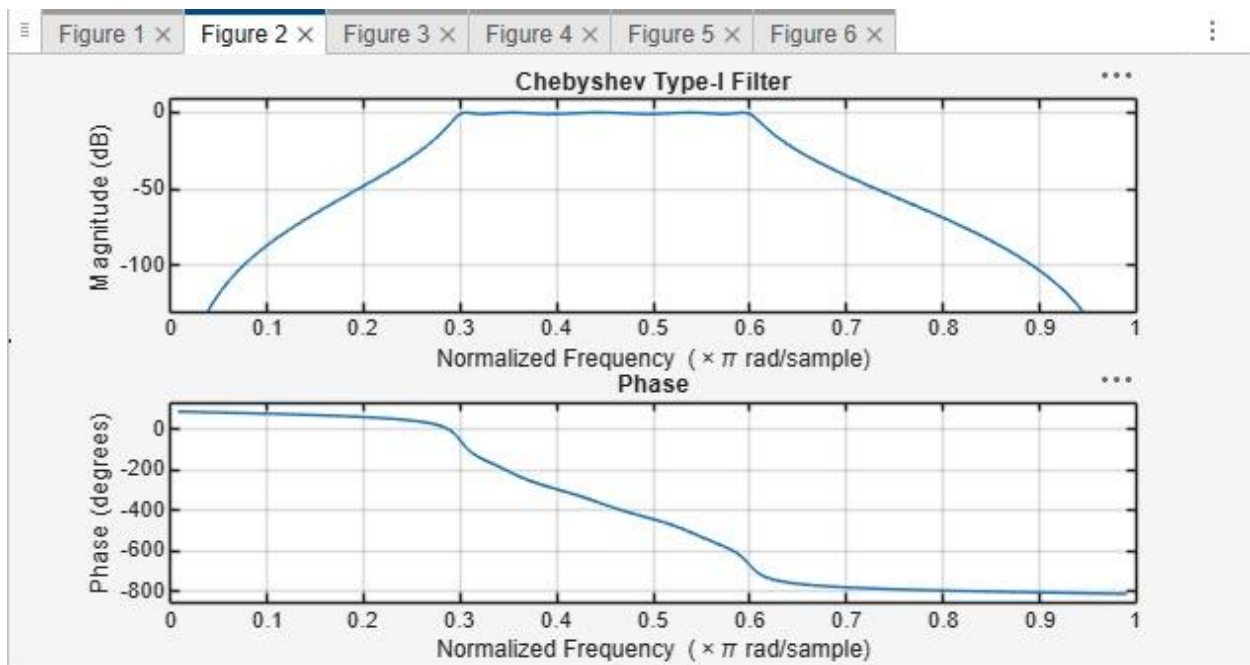
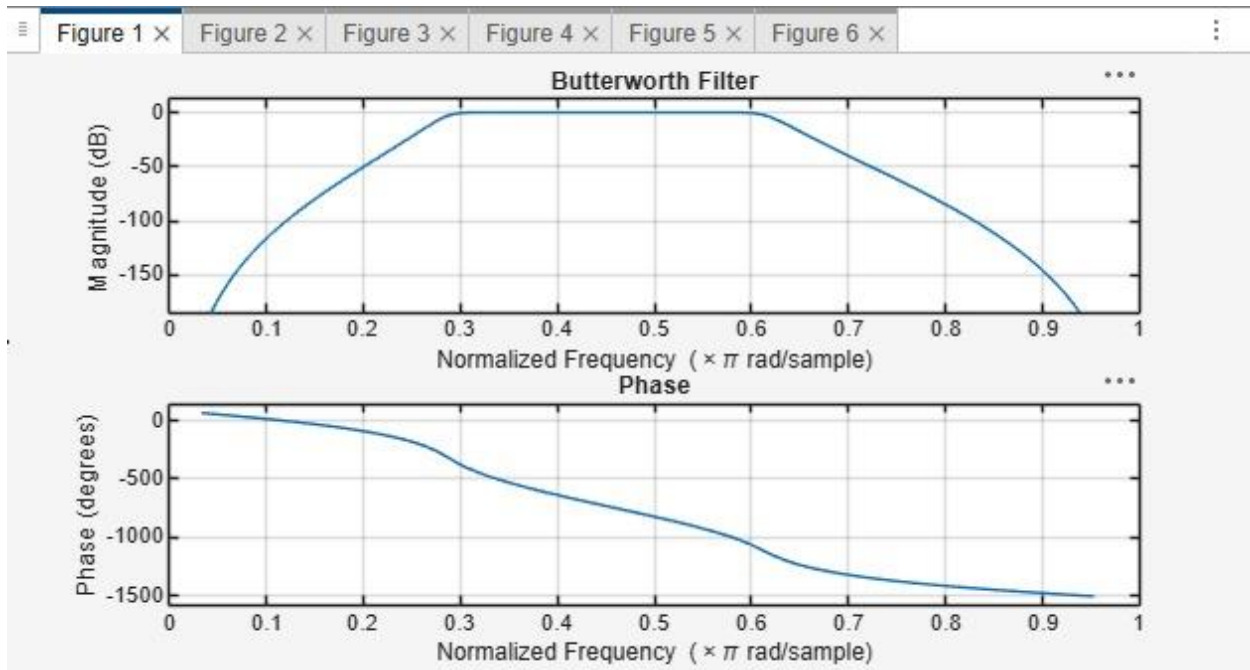
disp(['Butterworth Order = ',num2str(n1)])
disp(['Chebyshev Order = ',num2str(n2)])

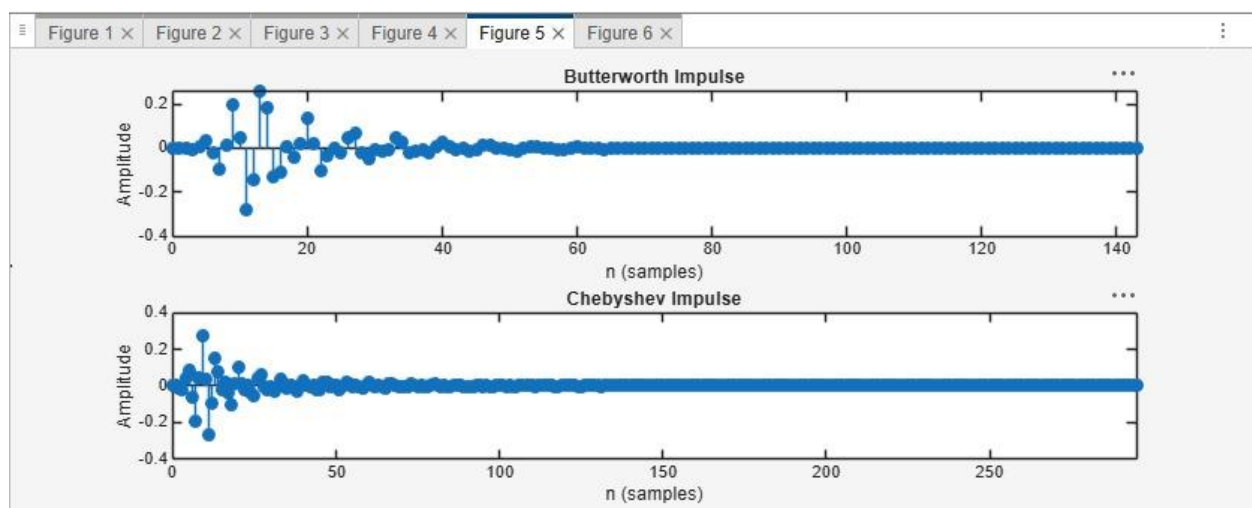
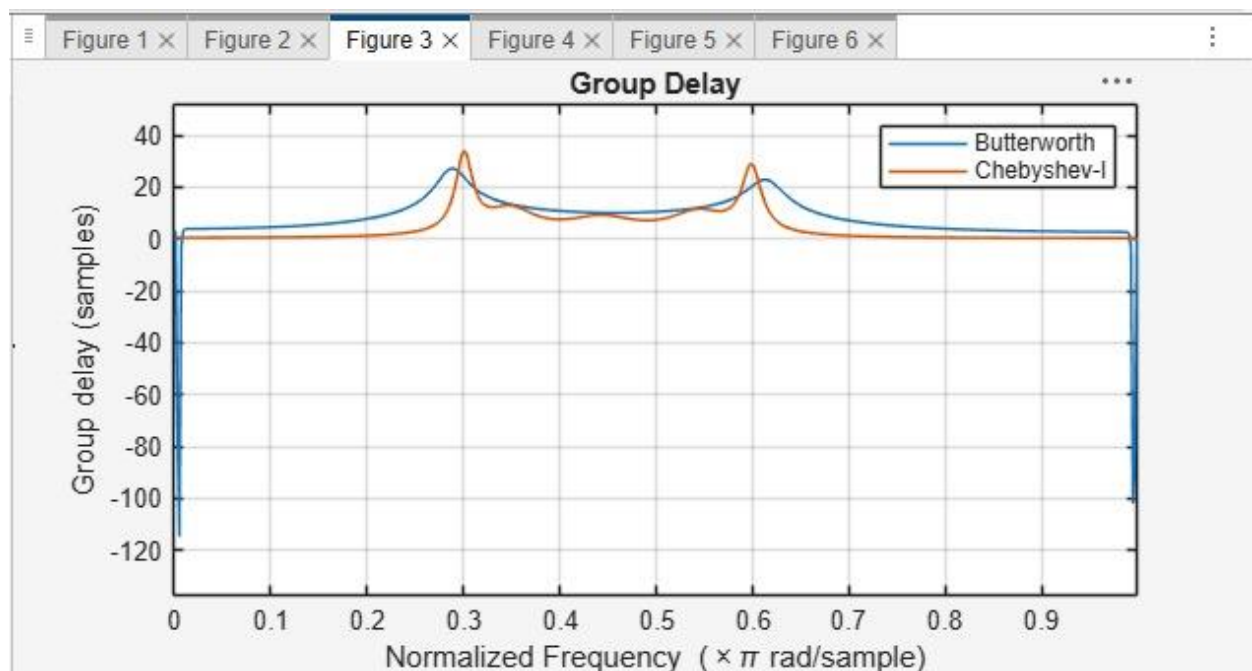
```

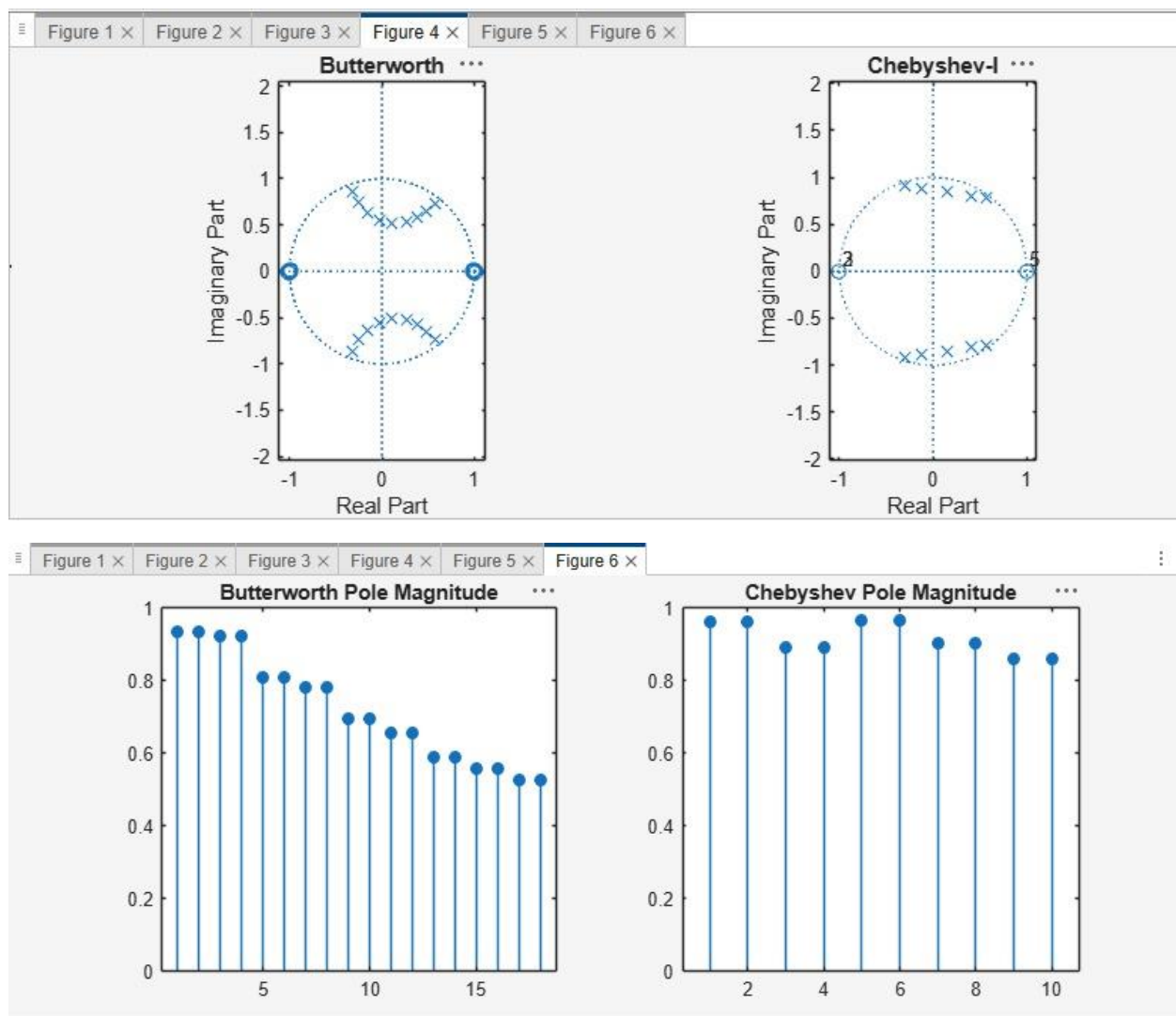
Output:

Butterworth Order = 9

Chebyshev Order = 5







Results:

1. Butterworth and Chebyshev Type-I IIR Band-Pass Filters were successfully designed and implemented in MATLAB.
2. The minimum filter orders and cutoff frequencies were obtained for both filters.
3. The frequency responses were analyzed and compared; the Chebyshev filter provided a sharper transition, while the Butterworth filter showed a smoother response.
4. Pole-zero analysis confirmed that both filters are stable and satisfy the given design specifications.

Practical Question – 1

Design and compare **Butterworth and Chebyshev Type-I IIR Band-Pass Filters** in MATLAB for the following specifications:

- Sampling Frequency = 10 kHz
- Passband = 1.5 kHz to 3 kHz
- Stopband = 1.2 kHz and 3.3 kHz
- Passband Ripple = 1 dB
- Stopband Attenuation = 45 dB

Tasks:

1. Determine the minimum filter order and cutoff frequencies.
2. Design both Band-Pass Filters and obtain the filter coefficients.
3. Compare the magnitude and phase responses.
4. Generate pole-zero plots and verify filter stability.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question -2

Design and compare **Butterworth and Chebyshev Type-I IIR Band-Pass Filters** in MATLAB for the following specifications:

- Sampling Frequency = 12 kHz
- Passband = 2 kHz to 4 kHz
- Stopband = 1.6 kHz and 4.5 kHz
- Passband Ripple = 0.5 dB
- Stopband Attenuation = 50 dB

Tasks:

1. Calculate the minimum filter order and cutoff frequencies.
2. Implement the filters using MATLAB and obtain the transfer function coefficients.
3. Analyze and compare the magnitude, phase, and group delay responses.
4. Plot the pole-zero diagrams and evaluate the stability and filtering performance.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 4	Design of Low Pass Filter using Windowing Techniques
Date:	4..1. Design and Analysis of an FIR Low-Pass Filter Using the Rectangular Window Technique in MATLAB

Objective:

1. To design an FIR Low-Pass Filter using the Rectangular Window technique for the given cutoff frequency, sampling frequency, and filter order specifications.
2. To generate the FIR filter coefficients using the Rectangular Window method in MATLAB and implement the designed filter.
3. To obtain and analyze the magnitude response and phase response of the FIR Low-Pass Filter designed using the Rectangular Window.

Procedure

1. Open MATLAB and create a new script file.
2. Initialize the workspace using `clc`, `clear`, and `close all`.
3. Specify the filter design parameters: sampling frequency, cutoff frequency, and filter order.
4. Calculate the normalized cutoff frequency.
5. Design the FIR Low-Pass Filter using the `fir1()` function with a Rectangular Window (`rectwin`).
6. Generate and display the FIR filter coefficients.
7. Obtain the magnitude and phase responses using the `freqz()` function.
8. Plot and analyze the impulse response of the designed filter.
9. Generate the pole-zero plot and examine the filter characteristics.
10. Verify the stability and frequency-selective performance of the FIR Low-Pass Filter.

Program(Matlab Code)

```

clc;

clear;

close all;

%% --- Input Parameters ---

fp = 1000;    % Passband cutoff (Hz)

fs = 4000;    % Sampling frequency (Hz)

N = 50;       % Filter order (even number preferred)

```

```

wc = fp/(fs/2); % Normalized cutoff frequency (0 to 1)
%% --- 1. FIR Low Pass using Rectangular Window ---
b_rect = fir1(N, wc, 'low', rectwin(N+1));
disp('--- Rectangular Window Coefficients ---');
disp(b_rect);
%% --- 2. FIR Low Pass using Hamming Window ---
b_hamm = fir1(N, wc, 'low', hamming(N+1));
disp('--- Hamming Window Coefficients ---');
disp(b_hamm);
%% --- 3. FIR Low Pass using Hanning Window ---
b_hann = fir1(N, wc, 'low', hanning(N+1));
disp('--- Hanning Window Coefficients ---');
disp(b_hann);
%% --- Plot Frequency Responses ---
figure;
freqz(b_rect,1);
title('Low-pass FIR using Rectangular Window');
figure;
freqz(b_hamm,1);
title('Low-pass FIR using Hamming Window');
figure;
freqz(b_hann,1);
title('Low-pass FIR using Hanning Window');
OUTPUT;

```

Objective 1:

```

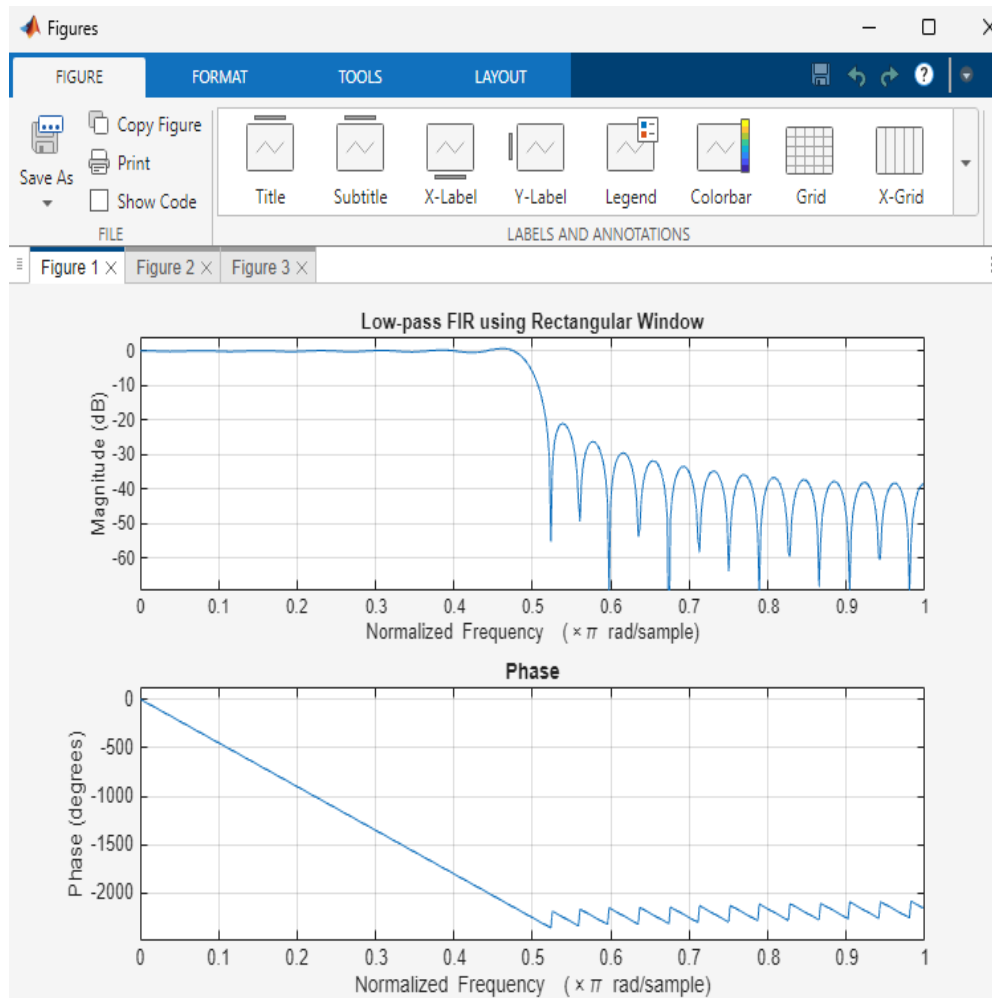
--- Rectangular Window Coefficients ---
0.0126
-0.0000
-0.0137
0.0000

```

0.0150
-0.0000
-0.0166
0.0000
0.0185
-0.0000
-0.0210
0.0000
0.0242
-0.0000
-0.0286
0.0000
0.0349
-0.0000
-0.0449
0.0000
0.0629
-0.0000
-0.1048
0.0000
0.3145
0.4940
0.3145
0.0000
-0.1048
-0.0000
0.0629
0.0000
-0.0449
-0.0000
0.0349
0.0000
-0.0286
-0.0000
0.0242
0.0000
-0.0210
-0.0000
0.0185
0.0000
-0.0166
-0.0000
0.0150
0.0000

-0.0137
-0.0000
0.0126

Objective 2:



Result

1. An FIR Low-Pass Filter was successfully designed using the Rectangular Window technique for the specified cutoff frequency, sampling frequency, and filter order.
2. The FIR filter coefficients were successfully generated using MATLAB and the designed filter was implemented.
3. The magnitude response and phase response of the designed FIR Low-Pass Filter were obtained and analyzed. The magnitude response exhibited low-pass characteristics, while the phase response demonstrated the expected linear-phase behavior.

Practical Question 1

Question:

Design an FIR Low-Pass Filter using the **Rectangular Window** technique in MATLAB for a sampling frequency of **8000 Hz**, cutoff frequency of **1000 Hz**, and filter order of **50**.

Tasks:

1. Calculate the normalized cutoff frequency.
2. Generate the FIR filter coefficients using the `fir1()` function with a Rectangular Window.
3. Plot the magnitude response and phase response of the designed filter.
4. Analyze the low-pass filtering characteristics based on the obtained responses.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2**Question:**

Design an FIR Low-Pass Filter using the **Rectangular Window** technique in MATLAB for a sampling frequency of **12000 Hz**, cutoff frequency of **1800 Hz**, and filter order of **60**.

Tasks:

1. Design the FIR Low-Pass Filter using the specified parameters.
2. Generate and display the FIR filter coefficients.
3. Obtain the magnitude and phase responses using the `freqz()` function.
4. Evaluate the passband and stopband characteristics and compare the response with the ideal low-pass filter behavior.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 4	Design of Low Pass Filter using Windowing Techniques
Date:	4.2. Design and Analysis of an FIR Low-Pass Filter Using the Hamming Window Technique in MATLAB

Objective:

1. To design an FIR Low-Pass Filter using the Hamming Window technique for the given cutoff frequency, sampling frequency, and filter order specifications.
2. To generate the FIR filter coefficients using the Hamming Window method in MATLAB and implement the designed filter.
3. To obtain and analyze the magnitude response and phase response of the FIR Low-Pass Filter designed using the Hamming Window.

Procedure

1. Open MATLAB and create a new script file.
2. Clear the workspace, command window, and close all figure windows.
3. Specify the sampling frequency, cutoff frequency, and filter order.
4. Calculate the normalized cutoff frequency.
5. Design the FIR Low-Pass Filter using the `fir1()` function with the **Hamming Window** (`hamming`).
6. Generate and display the FIR filter coefficients.
7. Obtain the magnitude and phase responses of the designed filter using the `freqz()` function.
8. Plot and analyze the frequency response characteristics of the filter.
9. Verify the low-pass filtering behavior by observing the passband and stopband responses.
10. Record the observations and conclude the performance of the FIR Low-Pass Filter designed using the Hamming Window technique.

Program (Matlab Code)

```

clc;

clear;

close all;

%% --- Input Parameters ---

fp = 1000;    % Passband cutoff (Hz)

fs = 4000;    % Sampling frequency (Hz)

N = 50;      % Filter order (even number preferred)

wc = fp/(fs/2); % Normalized cutoff frequency (0 to 1)

%% --- FIR Low Pass using Hamming Window ---

```

```

b_hamm = fir1(N, wc, 'low', hamming(N+1));
disp('--- Hamming Window Coefficients ---');
disp(b_hamm);
%% --- Plot Frequency Responses ---
figure;
freqz(b_hamm,1);
title('Low-pass FIR using Hamming Window');

```

Output

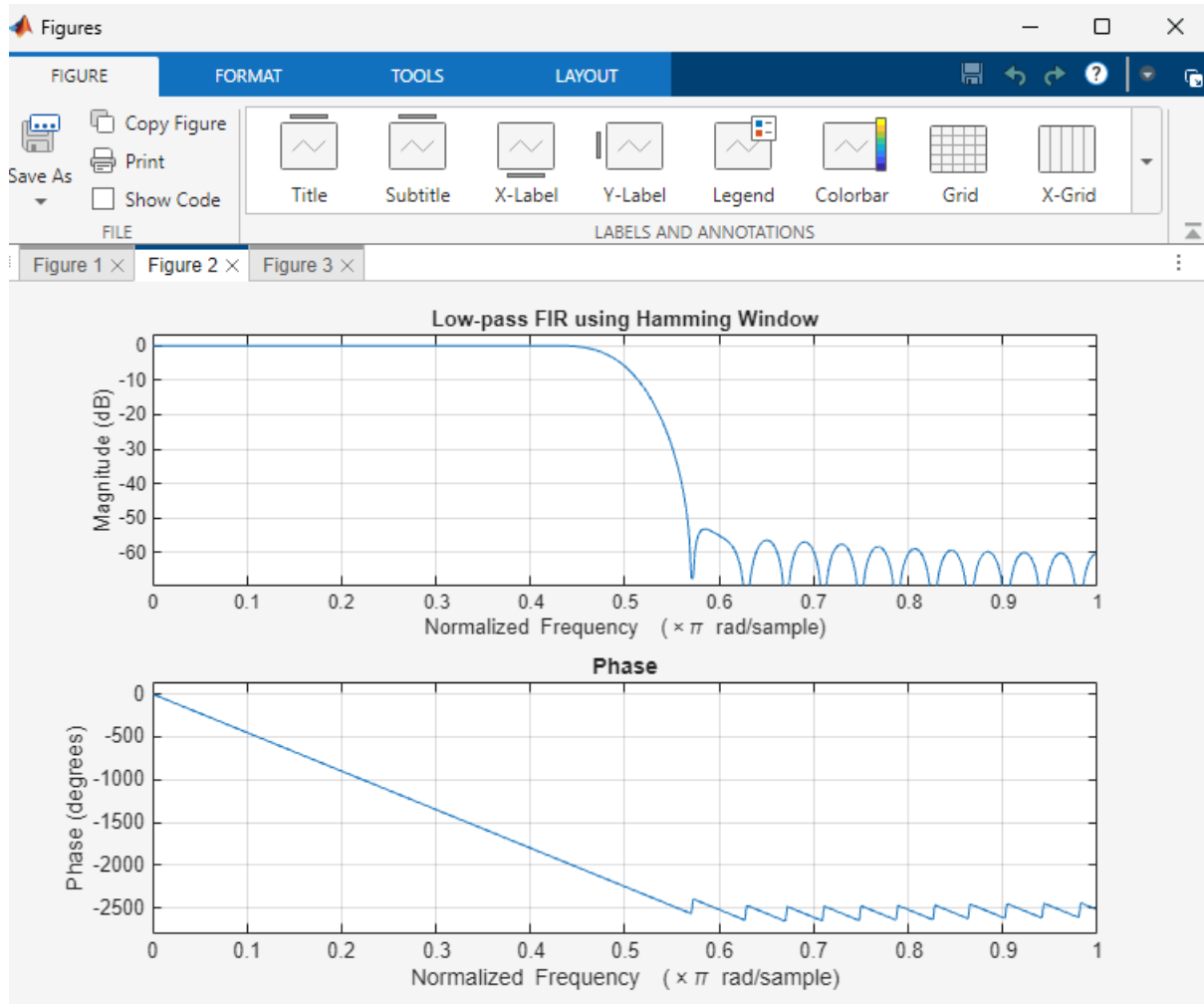
```

--- Hamming Window Coefficients ---
0.0010
-0.0000
-0.0013
0.0000
0.0021
-0.0000
-0.0034
0.0000
0.0055
-0.0000
-0.0084
0.0000
0.0125
-0.0000
-0.0181
0.0000
0.0260
-0.0000
-0.0379
0.0000
0.0580
-0.0000
-0.1026
0.0000
0.3168
0.4995
0.3168
0.0000
-0.1026
-0.0000
0.0580

```

0.0000
-0.0379
-0.0000
0.0260
0.0000
-0.0181
-0.0000
0.0125
0.0000
-0.0084
-0.0000
0.0055
0.0000
-0.0034
-0.0000
0.0021
0.0000
-0.0013
-0.0000

0.0010



Result

1. The FIR Low-Pass Filter was successfully designed using the Hamming Window technique for the specified filter parameters.
2. The FIR filter coefficients were generated and implemented successfully in MATLAB.
3. The magnitude and phase responses of the designed filter were obtained and analyzed.
4. The filter exhibited the desired low-pass characteristics with improved stopband attenuation compared to the Rectangular Window method.

Practical Question 1

Design an FIR Low-Pass Filter using the **Hamming Window** technique in MATLAB for a sampling frequency of **8000 Hz**, cutoff frequency of **1500 Hz**, and filter order of **50**.

Tasks:

1. Calculate the normalized cutoff frequency.

2. Generate the FIR filter coefficients using the `fir1()` function with a Hamming Window.
3. Obtain and plot the magnitude response and phase response of the designed filter.
4. Analyze the passband and stopband characteristics and verify the low-pass filtering behavior.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

Design an FIR Low-Pass Filter using the **Hamming Window** technique in MATLAB for a sampling frequency of **12000 Hz**, cutoff frequency of **2000 Hz**, and filter order of **60**.

Tasks:

1. Design the FIR Low-Pass Filter using the specified parameters and Hamming Window.
2. Generate and display the FIR filter coefficients.
3. Determine the magnitude response and phase response using the `freqz()` function.
4. Evaluate the filter performance in terms of passband response, stopband attenuation, and transition band characteristics.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 4	Design of Low Pass Filter using Windowing Techniques
Date:	4.3. Design and Analysis of an FIR Low-Pass Filter Using the Hanning Window Technique in MATLAB

Objective:

1. To design an FIR Low-Pass Filter using the Hanning Window technique for the given cutoff frequency, sampling frequency, and filter order specifications.
2. To generate the FIR filter coefficients using the Hanning Window method in MATLAB and implement the designed filter.
3. To obtain and analyze the magnitude response and phase response of the FIR Low-Pass Filter designed using the Hanning Window.

Procedure

1. Open MATLAB and create a new script file.
2. Clear the workspace, command window, and close all figure windows.
3. Specify the sampling frequency, cutoff frequency, and filter order.
4. Calculate the normalized cutoff frequency.
5. Design the FIR Low-Pass Filter using the `fir1()` function with the **Hanning (Hann) Window** (`hanning`).
6. Generate and display the FIR filter coefficients.
7. Obtain the magnitude response and phase response of the designed filter using the `freqz()` function.
8. Plot and analyze the frequency response characteristics of the filter.
9. Verify the low-pass filtering behavior by observing the passband and stopband regions.
10. Record the observations and conclude the performance of the FIR Low-Pass Filter designed using the Hanning Window technique.

Program (Matlab Code):

```

clc;

clear;

close all;

%% --- Input Parameters ---

fp = 1000;    % Passband cutoff (Hz)

fs = 4000;    % Sampling frequency (Hz)

N = 50;      % Filter order (even number preferred)

wc = fp/(fs/2); % Normalized cutoff frequency (0 to 1)

```



```
%% --- FIR Low Pass using Hanning Window ---
```

```
b_hann = fir1(N, wc, 'low', hanning(N+1));
```

```
disp('--- Hanning Window Coefficients ---');
```

```
disp(b_hann);
```

```
%% --- Plot Frequency Responses ---
```

```
figure;
```

```
freqz(b_hann,1);
```

```
title('Low-pass FIR using Hanning Window');
```

```
OUTPUT;
```

```
--- Hanning Window Coefficients ---
```

```
0.0000
```

```
-0.0000
```

```
-0.0004
```

```
0.0000
```

```
0.0013
```

```
-0.0000
```

```
-0.0028
```

```
0.0000
```

```
0.0050
```

```
-0.0000
```

```
-0.0081
```

```
0.0000
```

```
0.0122
```

```
-0.0000
```

```
-0.0179
```

```
0.0000
```

```
0.0259
```

```
-0.0000
```

```
-0.0378
```

```
0.0000
```

```
0.0580
```

```
-0.0000
```

```
-0.1027
```

```
0.0000
```

```
0.3172
```

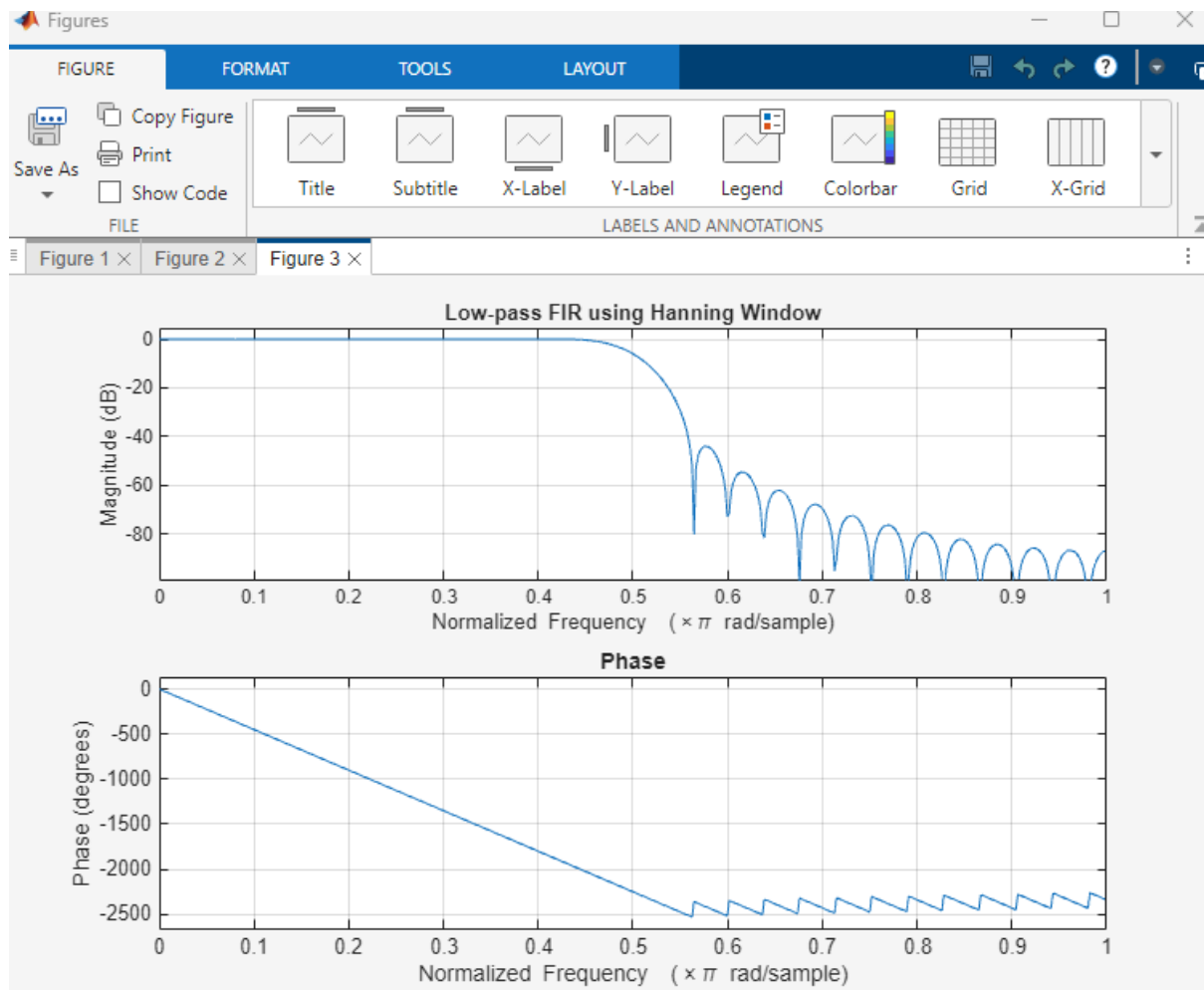
```
0.5000
```

```
0.3172
```

```
0.0000
```

```
-0.1027
```

-0.0000
0.0580
0.0000
-0.0378
-0.0000
0.0259
0.0000
-0.0179
-0.0000
0.0122
0.0000
-0.0081
-0.0000
0.0050
0.0000
-0.0028
-0.0000
0.0013
0.0000
-0.0004
-0.0000
0.0000



Result

1. The FIR Low-Pass Filter was successfully designed using the Hanning Window technique for the specified filter parameters.
2. The FIR filter coefficients were generated and implemented successfully in MATLAB.
3. The magnitude and phase responses of the designed filter were obtained and analyzed.
4. The filter exhibited the desired low-pass characteristics with reduced sidelobes and improved stopband attenuation compared to the Rectangular Window method.

Practical Question 1

Design an FIR Low-Pass Filter using the **Hanning (Hann) Window** technique in MATLAB for a sampling frequency of **8000 Hz**, cutoff frequency of **1500 Hz**, and filter order of **50**.

Tasks:

1. Calculate the normalized cutoff frequency.
2. Generate the FIR filter coefficients using the `fir1()` function with a Hanning Window.

3. Plot the magnitude response and phase response of the designed filter.
4. Analyze the passband, transition band, and stopband characteristics.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

Design and implement an FIR Low-Pass Filter using the **Hanning (Hann) Window** technique in MATLAB for a sampling frequency of **12000 Hz**, cutoff frequency of **2000 Hz**, and filter order of **60**.

Tasks:

1. Design the FIR Low-Pass Filter using the specified parameters and Hanning Window.
2. Generate and display the FIR filter coefficients.
3. Obtain the magnitude response and phase response using the freqz() function.
4. Determine the cutoff frequency and stopband attenuation from the response plots.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 5	Design of High Pass Filter using Windowing Techniques
Date:	5.1 Generate the coefficients of an FIR High-Pass filter using Rectangular windowing techniques.

Objective:

1. To design an FIR High-Pass Filter using the Rectangular Window technique for the given cutoff frequency, sampling frequency, and filter order specifications.
2. To generate the FIR filter coefficients using the Rectangular Window method in MATLAB and implement the designed filter.
3. To obtain and analyze the magnitude response and phase response of the FIR High-Pass Filter designed using the Rectangular Window.

Procedure

1. Open MATLAB and create a new script file.
2. Clear the workspace, command window, and close all figure windows.
3. Specify the sampling frequency, cutoff frequency, and filter order.
4. Calculate the normalized cutoff frequency.
5. Design the FIR High-Pass Filter using the `fir1()` function with the '**high**' option and a **Rectangular Window** (`rectwin`).
6. Generate and display the FIR filter coefficients.
7. Obtain the magnitude response and phase response of the designed filter using the `freqz()` function.
8. Plot and analyze the frequency response characteristics of the filter.
9. Verify the high-pass filtering behavior by observing the attenuation of low-frequency components and the passage of high-frequency components.
10. Record the observations and conclude the performance of the FIR High-Pass Filter designed using the Rectangular Window technique.

Program (Matlab Code):

```
clc; clear; close all;
```

```
%% --- Input Parameters ---
```

```
fp = 1000;    % High-pass cutoff frequency (Hz)
```

```
fs = 8000;    % Sampling frequency (Hz)
```

```
N = 50;       % Filter order
```

```

wc = fp/(fs/2); % Normalized cutoff (0–1)

%% --- 1. FIR High Pass using Rectangular Window ---
b_rect = fir1(N, wc, 'high', rectwin(N+1));
disp('--- High-Pass Coefficients (Rectangular Window) ---');
disp(b_rect);

%% --- 2. FIR High Pass using Hamming Window ---
b_hamm = fir1(N, wc, 'high', hamming(N+1));
disp('--- High-Pass Coefficients (Hamming Window) ---');
disp(b_hamm);

%% --- 3. FIR High Pass using Hanning Window ---
b_hann = fir1(N, wc, 'high', hanning(N+1));
disp('--- High-Pass Coefficients (Hanning Window) ---');
disp(b_hann);

%% --- Frequency Response Plots ---
figure;
freqz(b_rect,1);
title('High-pass FIR using Rectangular Window');
figure;
freqz(b_hamm,1);
title('High-pass FIR using Hamming Window');
figure;
freqz(b_hann,1);
title('High-pass FIR using Hanning Window');

```

OUTPUT:

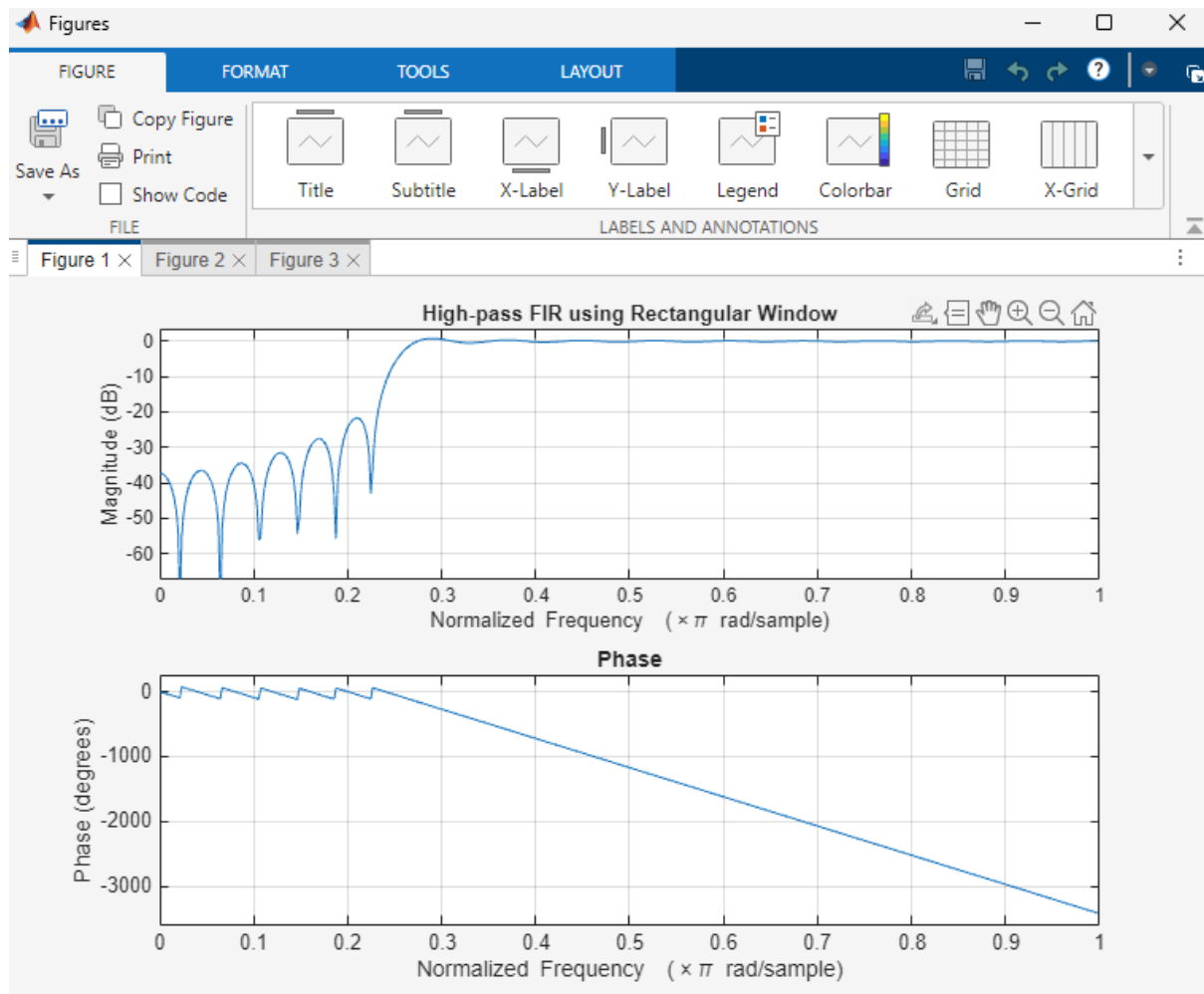
```

--- High-Pass Coefficients (Rectangular Window) ---
-0.0089

```

-0.0000
0.0097
0.0143
0.0106
-0.0000
-0.0117
-0.0175
-0.0131
-0.0000
0.0148
0.0225
0.0171
-0.0000
-0.0202
-0.0314
-0.0247
-0.0000
0.0318
0.0524
0.0445
-0.0000
-0.0741
-0.1572
-0.2223
0.7407
-0.2223
-0.1572
-0.0741
-0.0000
0.0445
0.0524
0.0318
-0.0000
-0.0247
-0.0314
-0.0202
-0.0000
0.0171
0.0225
0.0148
-0.0000
-0.0131
-0.0175
-0.0117

-0.0000
0.0106
0.0143
0.0097
-0.0000
-0.0089



Result

1. The FIR High-Pass Filter was successfully designed using the Rectangular Window technique for the specified cutoff frequency, sampling frequency, and filter order.
2. The FIR filter coefficients were successfully generated using MATLAB and the designed filter was implemented.
3. The magnitude response and phase response of the designed FIR High-Pass Filter were obtained and analyzed. The magnitude response showed attenuation of low-frequency components and passage of high-frequency components, confirming the desired high-pass

filtering behavior. The phase response exhibited the expected linear-phase characteristics of an FIR filter.

Practical Question 1

Design an FIR High-Pass Filter using the Rectangular Window technique in MATLAB for a sampling frequency of 10000 Hz, cutoff frequency of 2500 Hz, and filter order of 50.

Tasks:

1. Calculate the normalized cutoff frequency.
2. Generate the FIR filter coefficients using the `fir1()` function with a Rectangular Window.
3. Plot the magnitude response and phase response of the designed filter.
4. Verify that low-frequency components are attenuated and high-frequency components are passed.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

Design and implement an FIR High-Pass Filter using the Rectangular Window technique in MATLAB for a sampling frequency of 16000 Hz, cutoff frequency of 4000 Hz, and filter order of 60.

Tasks:

1. Design the FIR High-Pass Filter using the specified parameters.
2. Generate and display the FIR filter coefficients.
3. Obtain the magnitude response and phase response using the `freqz()` function.
4. Determine the cutoff frequency and examine the transition region.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 5	Design of High Pass Filter using Windowing Techniques
Date:	5.2. Generate the coefficients of an FIR High-Pass filter using Hamming windowing techniques

Objective:

1. To design an FIR High-Pass Filter using the Hamming Window technique for the given cutoff frequency, sampling frequency, and filter order specifications.
2. To generate the FIR filter coefficients using the Hamming Window method in MATLAB and implement the designed filter.
3. To obtain and analyze the magnitude response and phase response of the FIR High-Pass Filter designed using the Hamming Window.

Procedure

1. Open MATLAB and create a new script file.
2. Clear the workspace, command window, and close all figure windows.
3. Specify the sampling frequency, cutoff frequency, and filter order.
4. Calculate the normalized cutoff frequency.
5. Design the FIR High-Pass Filter using the `fir1()` function with the 'high' option and a Hamming Window (`hamming`).
6. Generate and display the FIR filter coefficients.
7. Obtain the magnitude response and phase response of the designed filter using the `freqz()` function.
8. Plot and analyze the frequency response characteristics of the filter.
9. Verify the high-pass filtering behavior by observing the attenuation of low-frequency components and the passage of high-frequency components.
10. Record the observations and conclude the performance of the FIR High-Pass Filter designed using the Hamming Window technique.

Program (Matlab Code):

```

clc; clear; close all;

%% --- Input Parameters ---

fp = 1000;    % High-pass cutoff frequency (Hz)
fs = 8000;    % Sampling frequency (Hz)
N = 50;       % Filter order

wc = fp/(fs/2); % Normalized cutoff (0–1)

%% --- FIR High Pass using Hamming Window ---

b_hamm = fir1(N, wc, 'high', hamming(N+1));

```

```
disp('--- High-Pass Coefficients (Hamming Window) ---');
```

```
disp(b_hamm');
```

```
%% --- Frequency Response Plots ---
```

```
figure;
```

```
freqz(b_hamm,1);
```

```
title('High-pass FIR using Hamming Window');
```

```
title('High-pass FIR using Hanning Window');
```

```
--- High-Pass Coefficients (Hamming Window) ---
```

```
-0.0007
```

```
-0.0000
```

```
0.0009
```

```
0.0016
```

```
0.0015
```

```
-0.0000
```

```
-0.0024
```

```
-0.0044
```

```
-0.0039
```

```
-0.0000
```

```
0.0060
```

```
0.0103
```

```
0.0088
```

```
-0.0000
```

```
-0.0128
```

```
-0.0217
```

```
-0.0184
```

```
-0.0000
```

```
0.0268
```

```
0.0464
```

```
0.0410
```

```
-0.0000
```

```
-0.0725
```

```
-0.1567
```

```
-0.2240
```

```
0.7493
```

```
-0.2240
```

```
-0.1567
```

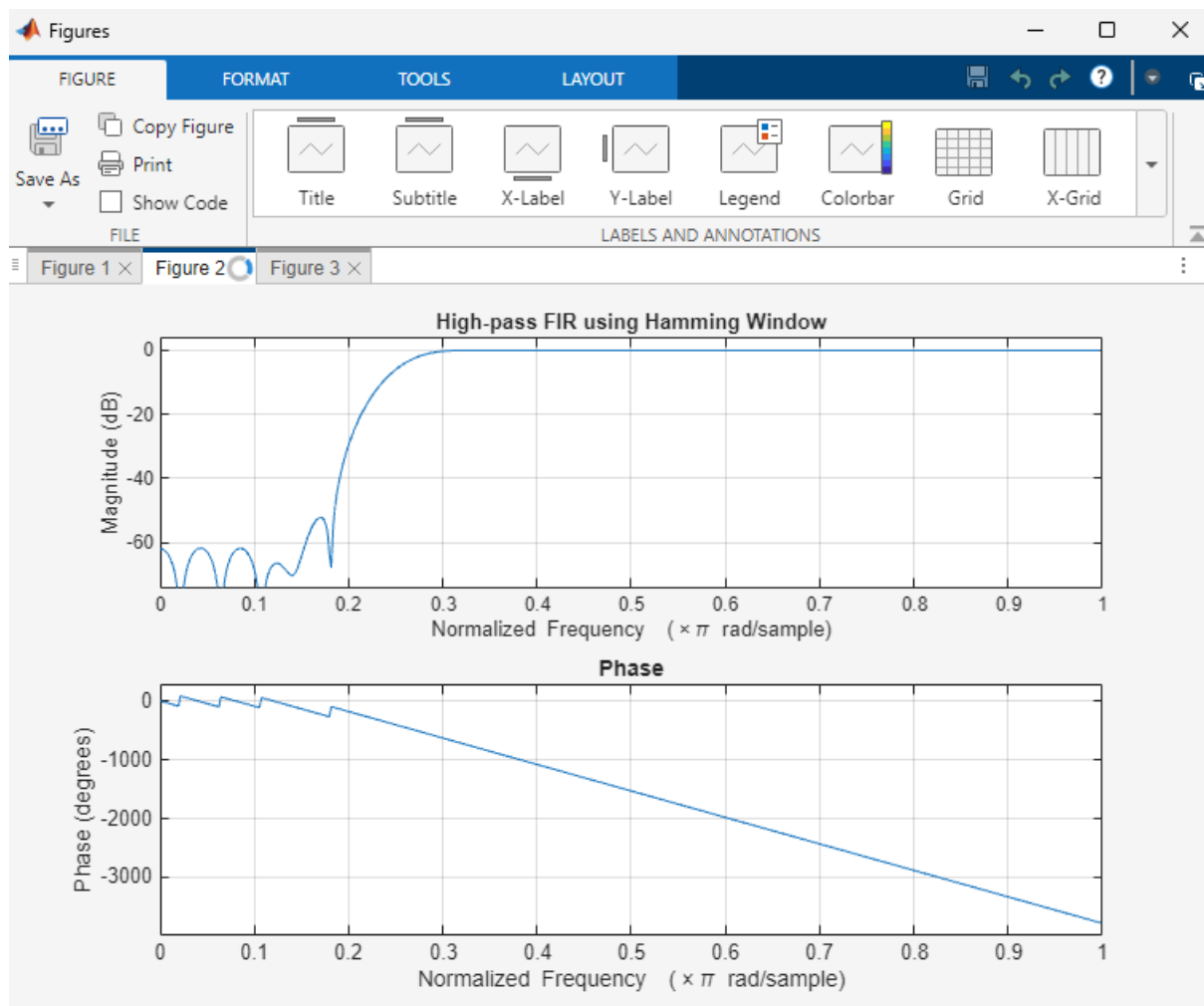
```
-0.0725
```

```
-0.0000
```

```
0.0410
```

```
0.0464
```

0.0268
-0.0000
-0.0184
-0.0217
-0.0128
-0.0000
0.0088
0.0103
0.0060
-0.0000
-0.0039
-0.0044
-0.0024
-0.0000
0.0015
0.0016
0.0009
-0.0000
-0.0007



Result:

1. The FIR High-Pass Filter was successfully designed using the Hamming Window technique for the specified cutoff frequency, sampling frequency, and filter order.
2. The FIR filter coefficients were successfully generated using the Hamming Window method in MATLAB and the filter was implemented.
3. The magnitude response and phase response of the designed FIR High-Pass Filter were obtained and analyzed. The magnitude response showed effective attenuation of low-frequency components and passage of high-frequency components, confirming the desired high-pass filtering behavior. The phase response exhibited the expected linear-phase characteristics of an FIR filter.

Practical Question 1

Design an FIR High-Pass Filter using the **Hamming Window** technique in MATLAB for a sampling frequency of **10000 Hz**, cutoff frequency of **2500 Hz**, and filter order of **50**.

Tasks:

1. Calculate the normalized cutoff frequency.
2. Generate the FIR filter coefficients using the `fir1()` function with a Hamming Window.
3. Plot the magnitude response and phase response of the designed filter.
4. Analyze the attenuation of low-frequency components and the passage of high-frequency components.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

Design and implement an FIR High-Pass Filter using the **Hamming Window** technique in MATLAB for a sampling frequency of **16000 Hz**, cutoff frequency of **4000 Hz**, and filter order of **60**.

Tasks:

1. Design the FIR High-Pass Filter using the specified parameters and Hamming Window.
2. Generate and display the FIR filter coefficients.
3. Obtain the magnitude response and phase response using the `freqz()` function.
4. Evaluate the passband, stopband, and transition-band characteristics of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 5	Design of High Pass Filter using Windowing Techniques
Date:	5.3. Generate the coefficients of an FIR High-Pass filter using Hanning windowing techniques

Objective:

1. To design an FIR High-Pass Filter using the Hanning Window technique for the given cutoff frequency, sampling frequency, and filter order specifications.
2. To generate the FIR filter coefficients using the Hanning Window method in MATLAB and implement the designed filter.
3. To obtain and analyze the magnitude response and phase response of the FIR High-Pass Filter designed using the Hanning Window.

Procedure

1. Open MATLAB and create a new script file.
2. Clear the workspace, command window, and close all figure windows.
3. Specify the sampling frequency, cutoff frequency, and filter order.
4. Calculate the normalized cutoff frequency.
5. Design the FIR High-Pass Filter using the `fir1()` function with the 'high' option and a Hanning (Hann) Window (`hanning`).
6. Generate and display the FIR filter coefficients.
7. Obtain the magnitude response and phase response of the designed filter using the `freqz()` function.
8. Plot and analyze the frequency response characteristics of the filter.
9. Verify the high-pass filtering behavior by observing the attenuation of low-frequency components and the passage of high-frequency components.
10. Record the observations and conclude the performance of the FIR High-Pass Filter designed using the Hanning Window technique.

Program (Matlab Code)

```
clc; clear; close all;
```

```
%% --- Input Parameters ---
```

```
fp = 1000;    % High-pass cutoff frequency (Hz)
```

```
fs = 8000;    % Sampling frequency (Hz)
```

```
N = 50;      % Filter order
```

```
wc = fp/(fs/2); % Normalized cutoff (0–1)
```

```
%% --- FIR High Pass using Hanning Window ---
```

```

b_hann = fir1(N, wc, 'high', hanning(N+1));
disp('--- High-Pass Coefficients (Hanning Window) ---');
disp(b_hann');
%% --- Frequency Response Plots ---
figure;
freqz(b_hann,1);
title('High-pass FIR using Hanning Window');

```

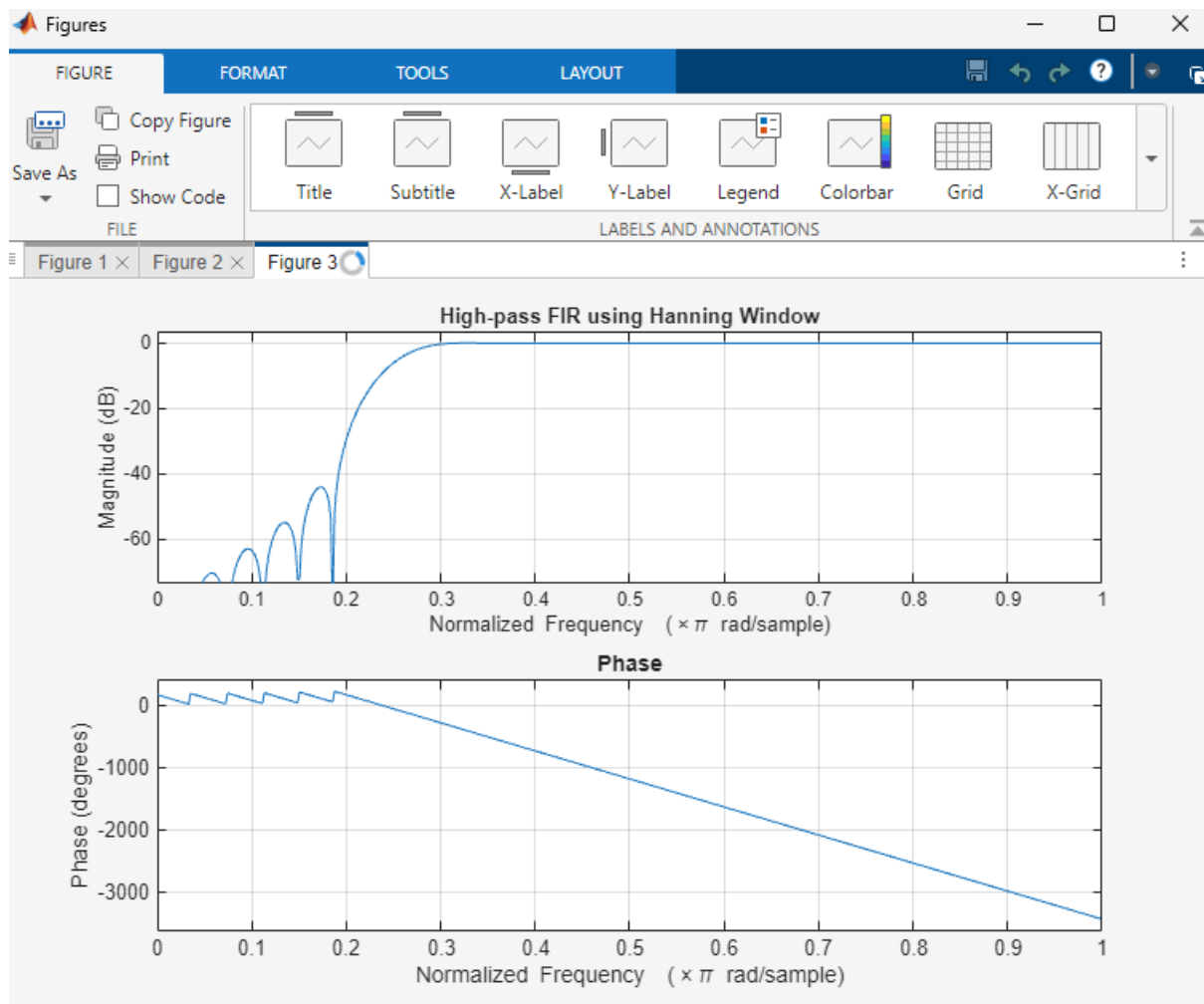
Output:

```

--- High-Pass Coefficients (Hanning Window) ---
-0.0000
-0.0000
 0.0003
 0.0008
 0.0009
-0.0000
-0.0020
-0.0038
-0.0035
-0.0000
 0.0057
 0.0100
 0.0087
-0.0000
-0.0127
-0.0216
-0.0183
-0.0000
 0.0267
 0.0464
 0.0410
-0.0000
-0.0726
-0.1568
-0.2243
 0.7500
-0.2243
-0.1568
-0.0726
-0.0000
 0.0410

```


0.0464
0.0267
-0.0000
-0.0183
-0.0216
-0.0127
-0.0000
0.0087
0.0100
0.0057
-0.0000
-0.0035
-0.0038
-0.0020
-0.0000
0.0009
0.0008
0.0003
-0.0000
-0.0000



>>

Result

1. The FIR High-Pass Filter was successfully designed using the Hanning Window technique for the specified cutoff frequency, sampling frequency, and filter order.
2. The FIR filter coefficients were successfully generated using MATLAB and the designed filter was implemented.
3. The magnitude response and phase response of the designed FIR High-Pass Filter were obtained and analyzed. The magnitude response exhibited attenuation of low-frequency components and passage of high-frequency components, confirming the desired high-pass filtering characteristics. The phase response demonstrated the expected linear-phase behavior of the FIR filter.

Practical Question 1

Question:

Design an FIR High-Pass Filter using the **Hanning (Hann) Window** technique in MATLAB for a sampling frequency of **10000 Hz**, cutoff frequency of **2500 Hz**, and filter order of **50**.

Tasks:

1. Calculate the normalized cutoff frequency.
2. Generate the FIR filter coefficients using the `fir1()` function with a Hanning Window.
3. Plot the magnitude response and phase response of the designed filter.
4. Verify the high-pass filtering characteristics of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

Design and implement an FIR High-Pass Filter using the **Hanning (Hann) Window** technique in MATLAB for a sampling frequency of **16000 Hz**, cutoff frequency of **4000 Hz**, and filter order of **60**.

Tasks:

1. Design the FIR High-Pass Filter using the specified parameters and Hanning Window.
2. Generate and display the FIR filter coefficients.
3. Obtain the magnitude response and phase response using the `freqz()` function.
4. Evaluate the passband, stopband, and transition-band characteristics of the filter.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 6	Finite Word length Effects
Date:	6.1 Uniform Quantization and Quantization Error Analysis of a Discrete-Time Sinusoidal Signal Using MATLAB

Objectives:

1. To generate a discrete-time sinusoidal signal in MATLAB.
2. To perform uniform quantization with different quantization levels.
3. To compare the original and quantized signals.
4. To analyze the quantization error and its effect on signal quality.

Procedure

1. Open MATLAB and create a new script file.
2. Define the sampling frequency, signal frequency, and time vector.
3. Generate a discrete-time sinusoidal signal using the sine function.
4. Select different quantization levels such as 8, 16, and 32 levels.
5. Apply uniform quantization to the generated signal for each quantization level.
6. Compute the quantization error by subtracting the quantized signal from the original signal.
7. Calculate the Mean Square Error (MSE) to evaluate the quantization performance.
8. Plot the original signal and the quantized signals for comparison.
9. Plot the quantization error signals corresponding to each quantization level.
10. Analyze the effect of increasing quantization levels on signal quality and error reduction.

Program (Matlab code)

```

clc;
clear;
close all;
%% Signal
fs = 1000; f = 50;
t = 0:1/fs:0.1;
x = sin(2*pi*f*t);
%% Quantization Levels
L = [8 16 32];
colors = {'r','g','m'};
figure;
for i = 1:length(L)
    % Quantization
    xq{i} = round((x+1)(L(i)-1)/2)/(L(i)-1) - 1;

    % Error & MSE
    e{i} = x - xq{i};
    mse(i) = mean(e{i}.^2);

```

```

    % Plot Quantized Signals
    subplot(4,1,i+1);
    stairs(t,xq{i},colors{i},'LineWidth',1.2);
    title(['Quantized Signal (' num2str(L(i)) ' Levels)']);
    grid on;
end

% Original Signal
subplot(4,1,1);
plot(t,x,'LineWidth',1.5);
title('Original Signal'); grid on;

% Display MSE
fprintf('MSE for 8 Levels = %f\n', mse(1));
fprintf('MSE for 16 Levels = %f\n', mse(2));
fprintf('MSE for 32 Levels = %f\n', mse(3));

%% Error Plots
figure;
for i = 1:3
    subplot(3,1,i);
    plot(t,e{i});
    title(['Error (' num2str(L(i)) ' Levels)']);
    grid on;
end

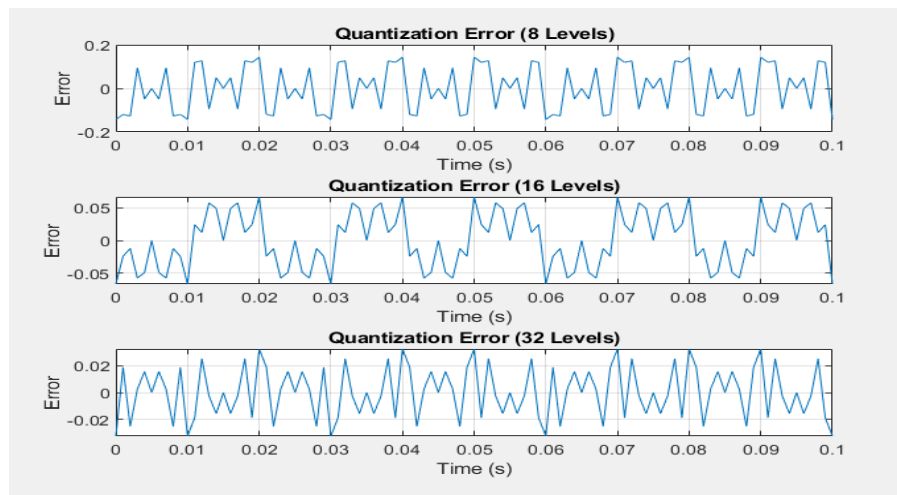
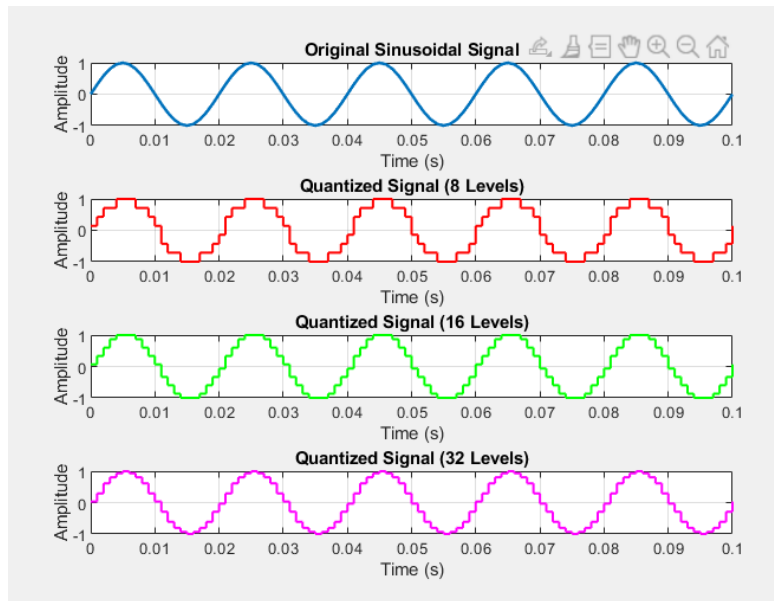
```

Output:

Objective 1:

Input:

- Sampling Frequency, $F_s = 1000\text{Hz}$
- Signal Frequency, $f = 50\text{Hz}$
- Time Duration = 0.1 s



Result:

1. A discrete-time sinusoidal signal was successfully generated in MATLAB.
2. Uniform quantization was performed for different quantization levels, and the corresponding quantized signals were obtained.
3. Comparison of the original and quantized signals showed that the quantized signal approximates the original signal more closely as the number of quantization levels increases.
4. The quantization error was analyzed and found to decrease with increasing quantization levels, resulting in improved signal quality and reduced quantization noise.

Practical Question 1:

Generate a discrete-time sinusoidal signal in MATLAB with amplitude $A = 1$, frequency $f = 100\text{Hz}$, and sampling frequency $F_s = 1000\text{Hz}$. Perform uniform quantization using 8

quantization levels. Plot the original signal, quantized signal, and quantization error. Analyze the effect of quantization on the signal quality.

Task:

1. Generate the sinusoidal signal.
2. Implement uniform quantization with 8 levels.
3. Plot the original and quantized signals.
4. Determine and plot the quantization error.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2:

Generate a discrete-time sinusoidal signal in MATLAB with amplitude $A = 2$, frequency $f = 200\text{Hz}$, and sampling frequency $F_s = 2000\text{Hz}$. Perform uniform quantization using 16 and 32 quantization levels. Compare the quantized signals and analyze the variation in quantization error for the two cases.

Task:

1. Generate the sinusoidal signal.
2. Quantize the signal using 16 and 32 levels.
3. Compare the original and quantized signals.
4. Compute and analyze the quantization errors for both quantization levels.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 6	Finite Word length Effects
Date:	6.2 Analysis of Finite Word Length Effects in Digital Signal Representation Using Truncation and Rounding Techniques in MATLAB

Objectives:

1. To represent a digital signal using a finite word length format in MATLAB.
2. To perform truncation and rounding operations on the finite word length representation of the signal.
3. To evaluate and compare the errors resulting from truncation and rounding techniques.
4. To investigate the impact of finite precision arithmetic on the accuracy of digital signal representation.

Procedure

1. Open MATLAB and create a new script file.
2. Define the sampling frequency, signal frequency, and time vector.
3. Generate a discrete-time sinusoidal signal using the MATLAB sin() function.
4. Select an appropriate finite word length (e.g., 8 bits) for representing the signal samples.
5. Scale the signal according to the selected word length to obtain a finite-precision representation.
6. Apply the **truncation operation** using the fix() function to remove the fractional part of the scaled signal samples.
7. Apply the **rounding operation** using the round() function to round the scaled signal samples to the nearest integer value.
8. Convert the truncated and rounded samples back to their original amplitude range by dividing them by the scaling factor.
9. Compute the **truncation error** by subtracting the truncated signal from the original signal.
10. Compute the **rounding error** by subtracting the rounded signal from the original signal.
11. Calculate the **Mean Square Error (MSE)** and **maximum error** for both truncation and rounding operations.
12. Compare the error values obtained from truncation and rounding.
13. Analyze the effect of finite word length on signal representation and evaluate which method provides better accuracy.

Program (Matlab code):

```

clc; clear; close all

n=0:79;

x=sin(2*pi*n/40);

q=8;

xt=floor(x*q)/q;

xr=round(x*q)/q;

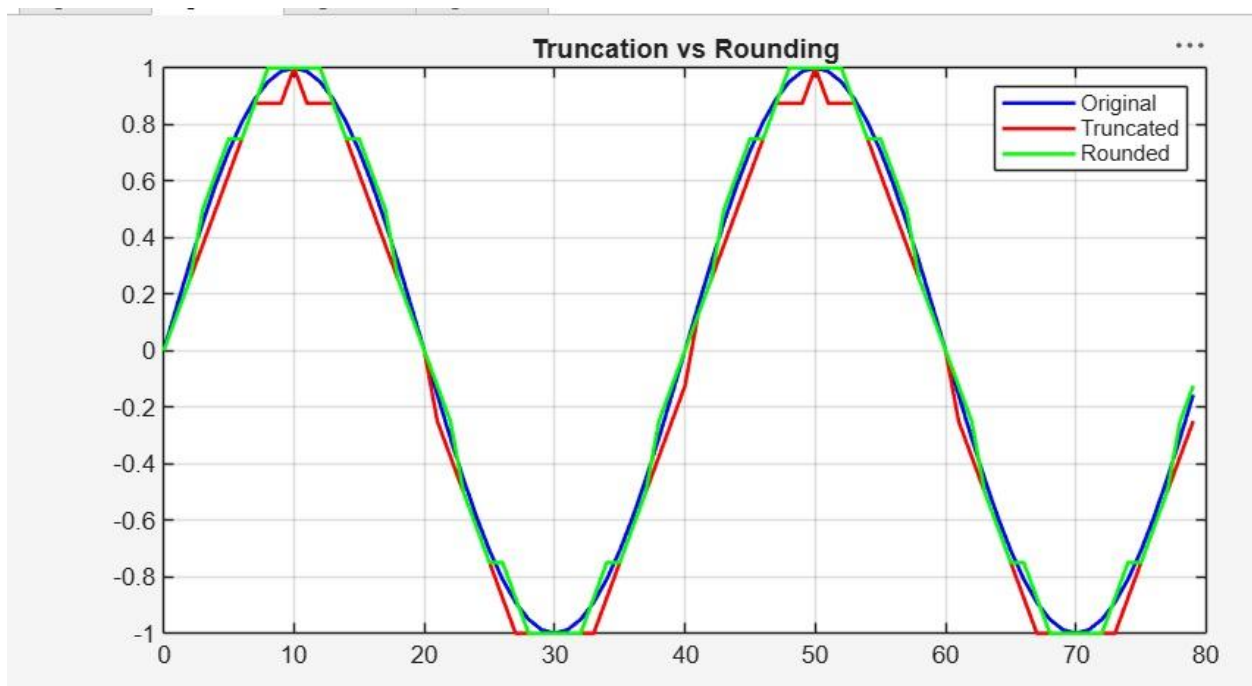
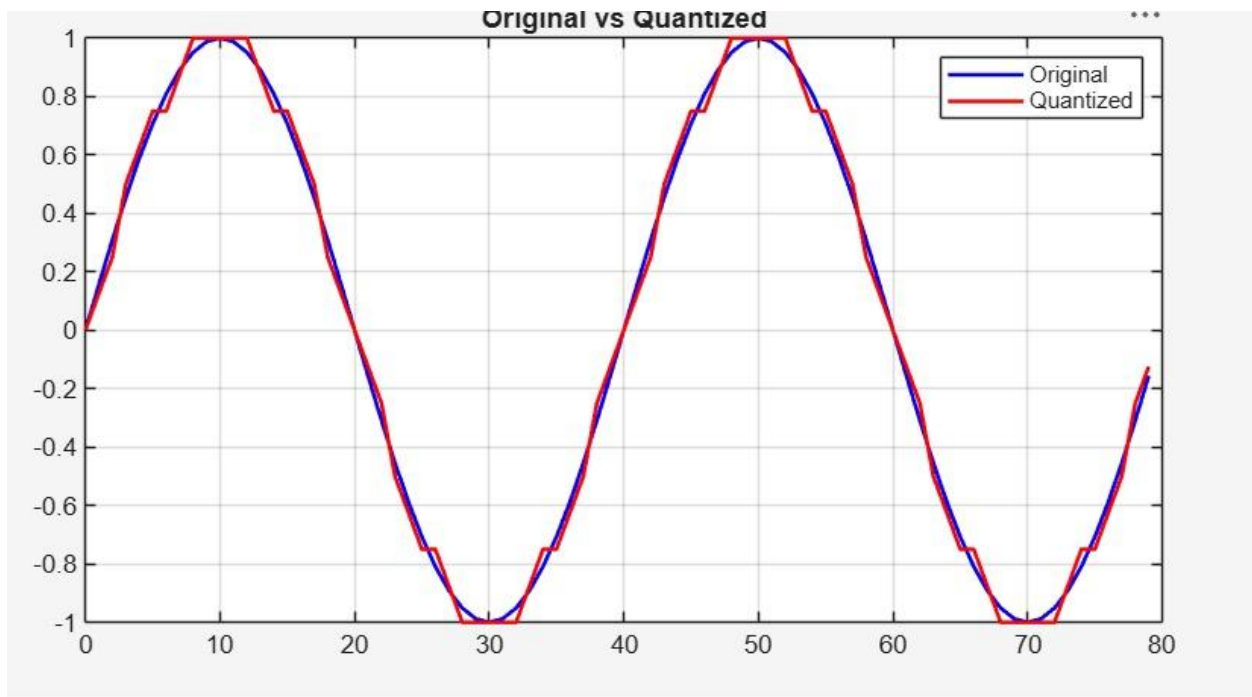
```

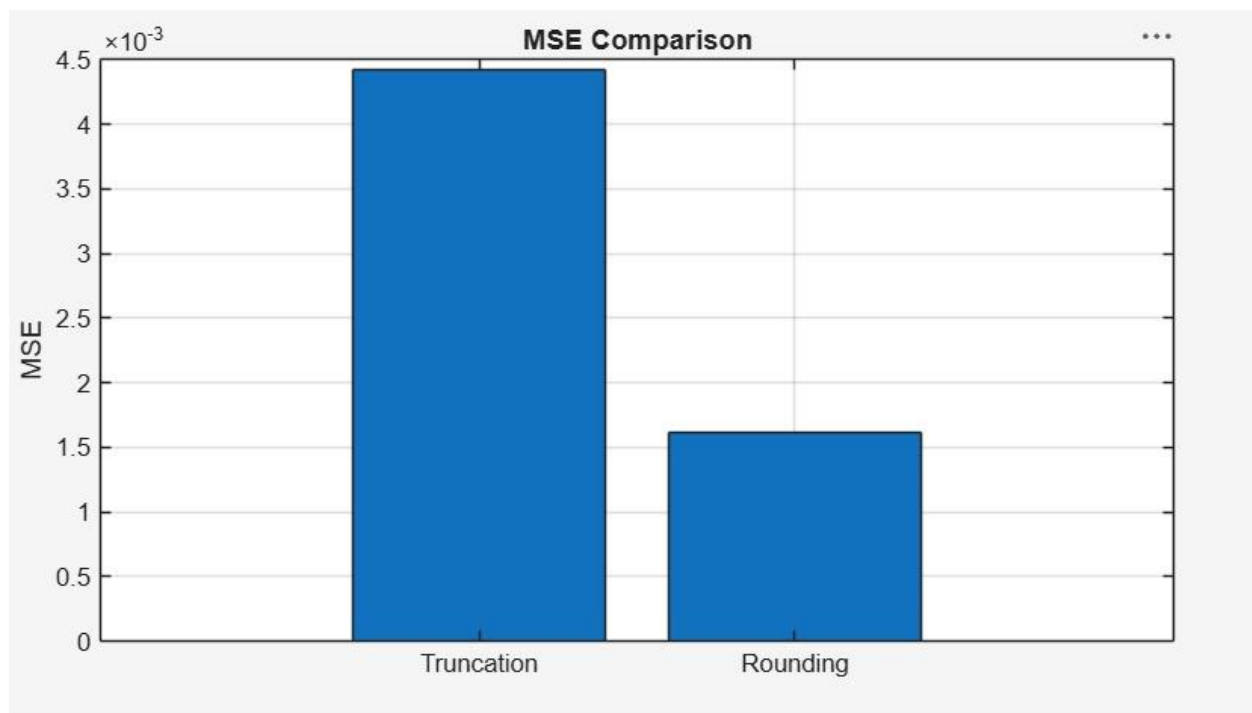
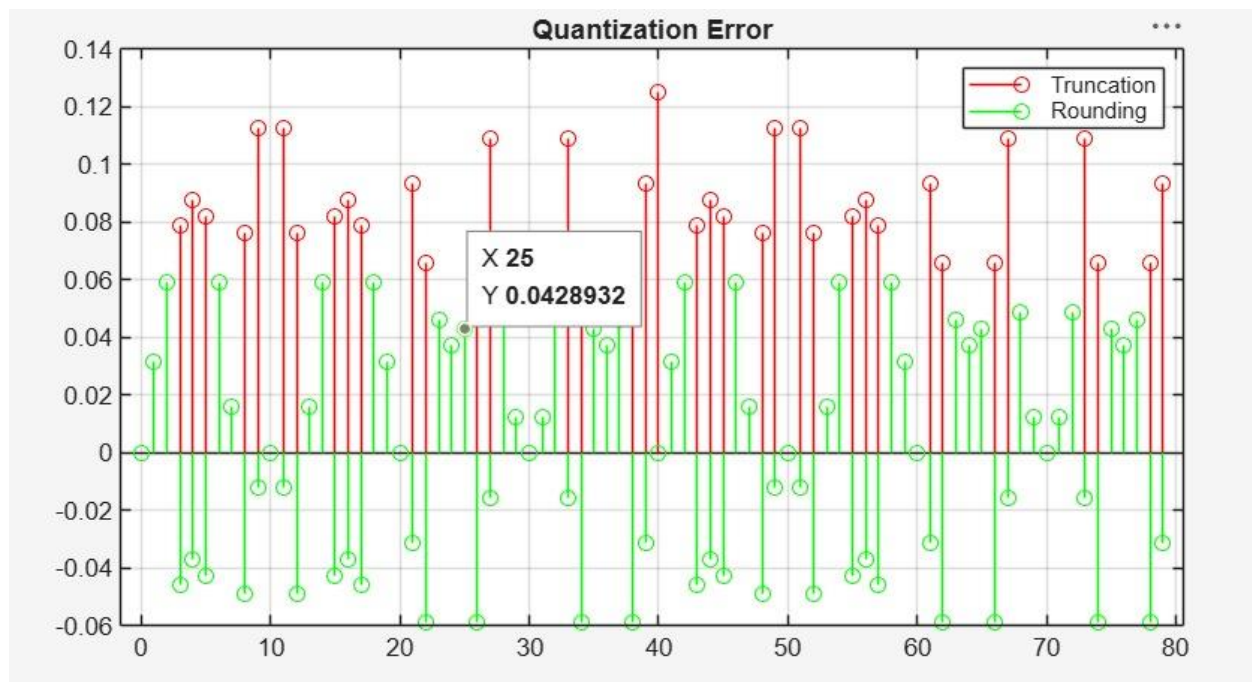
```

et=x-xt;
er=x-xr;
% Fig 1
figure
plot(n,x,'b',n,xr,'r','LineWidth',1.5)
grid on
title('Original vs Quantized')
legend('Original','Quantized')
% Fig 2
figure
plot(n,x,'b',n,xt,'r',n,xr,'g','LineWidth',1.5)
grid on
title('Truncation vs Rounding')
legend('Original','Truncated','Rounded')
% Fig 3
figure
stem(n,et,'r')
hold on
stem(n,er,'g')
grid on
title('Quantization Error')
legend('Truncation','Rounding')
% Fig 4
figure
bar([mean(et.^2) mean(er.^2)])
set(gca,'XTickLabel',{'Truncation','Rounding'})
grid on
title('MSE Comparison')
ylabel('MSE')

```

Output:





Result:

1. The digital signal was successfully represented using a finite word length format in MATLAB.
2. Truncation and rounding operations were implemented on the signal samples.

3. The errors introduced by truncation and rounding were computed and compared, showing that rounding generally produces smaller errors than truncation.
4. The effect of finite precision on signal representation was analyzed, demonstrating that reduced word length increases representation error and may affect signal accuracy.

Practical Question-1:

Generate a discrete-time sinusoidal signal sampled at 2000 Hz and represent it using finite precision arithmetic. Implement truncation and rounding to three decimal places.

Task:

1. Generate a discrete-time sinusoidal signal sampled at **2000 Hz** using MATLAB.
2. Represent the generated signal using **finite precision arithmetic**.
3. Apply **truncation and rounding operations up to three decimal places** on the signal samples.
4. Compute the **error signals and Mean Square Error (MSE)** for both truncation and rounding, and compare their performance.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2:

Analyze the effects of finite word length representation on a discrete-time sinusoidal signal in MATLAB. Consider a sinusoidal signal with a frequency of 50 Hz sampled at 1000 Hz and represent the signal using an 8-bit word length. Implement truncation and rounding operations on the signal samples.

Task:

1. Generate a discrete-time sinusoidal signal in MATLAB with a frequency of 50 Hz sampled at 1000 Hz.
2. Represent the generated signal using an 8-bit finite word length format.
3. Implement truncation and rounding operations on the signal samples.
4. Analyze the effects of finite word length by comparing the original, truncated, and rounded signals in terms of signal distortion and accuracy.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 7	Spectral Estimation
Date:	7.1 Power Spectral Density Estimation and Spectral Analysis of Noisy Sinusoidal Signals Using Periodogram Method in MATLAB

Objectives:

1. To generate a discrete-time signal consisting of sinusoidal components and noise.
2. To estimate the Power Spectral Density (PSD) of the signal using the Periodogram method.
3. To identify the frequency components present in the signal.
4. To compare the signal power and noise power for assessing the quality of generate signal.

Procedure:

1. Open MATLAB and create a new script file.
2. Define the sampling frequency and number of samples.
3. Generate a discrete-time signal consisting of two sinusoidal components.
4. Add white Gaussian noise to the signal.
5. Plot the generated signal in the time domain.
6. Use the periodogram() function to estimate the PSD.
7. Plot the PSD versus frequency.
8. Identify the dominant frequency components from the PSD peaks.
9. Analyze the spectral characteristics of the signal.
10. Verify that the detected frequencies match the original signal frequencies.

Program (Matlab code):

```
clc; clear; close all
```

```
fs=1000; t=(0:999)/fs;
```

```
s=sin(2*pi*100*t)+0.5*sin(2*pi*200*t);
```

```
n=0.3*randn(size(t));
```

```
x=s+n;
```

```
figure, plot(x), grid on
```

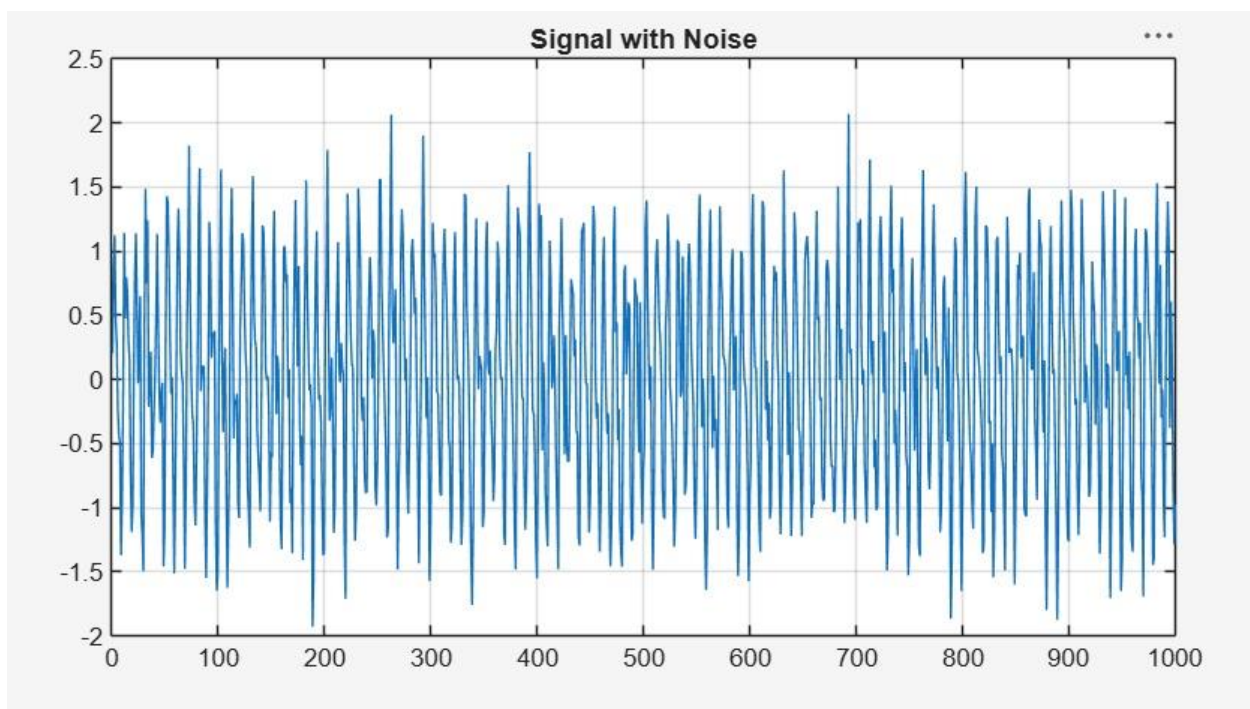
```
title('Signal with Noise')
```

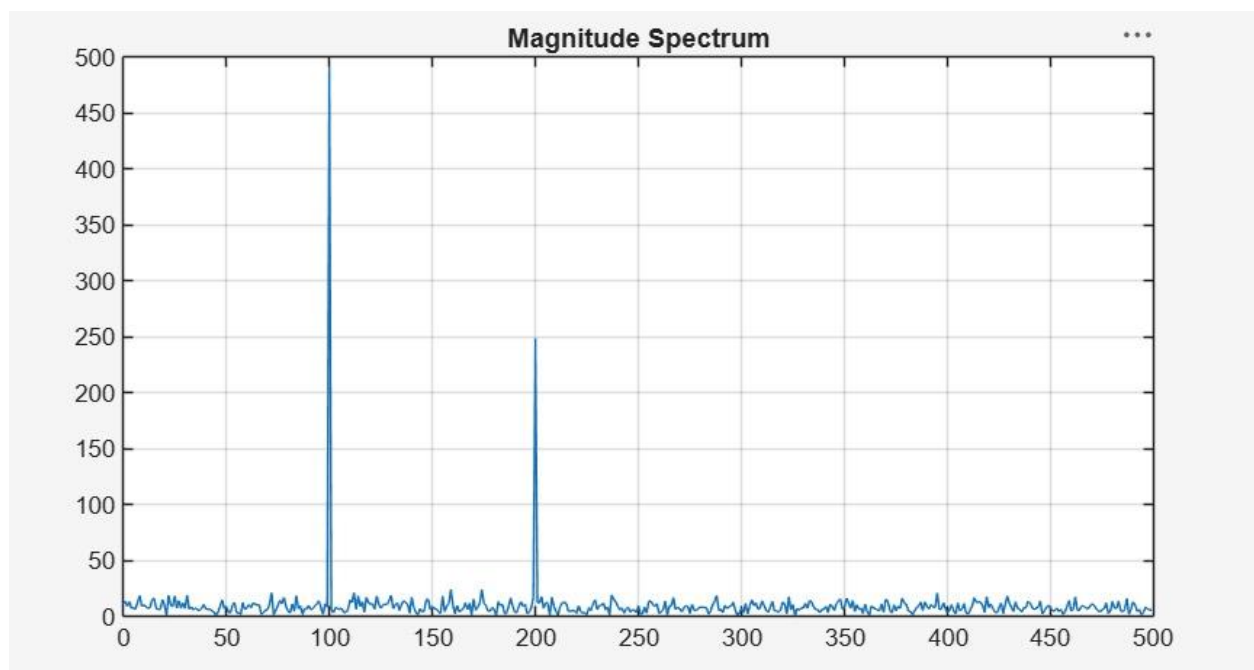
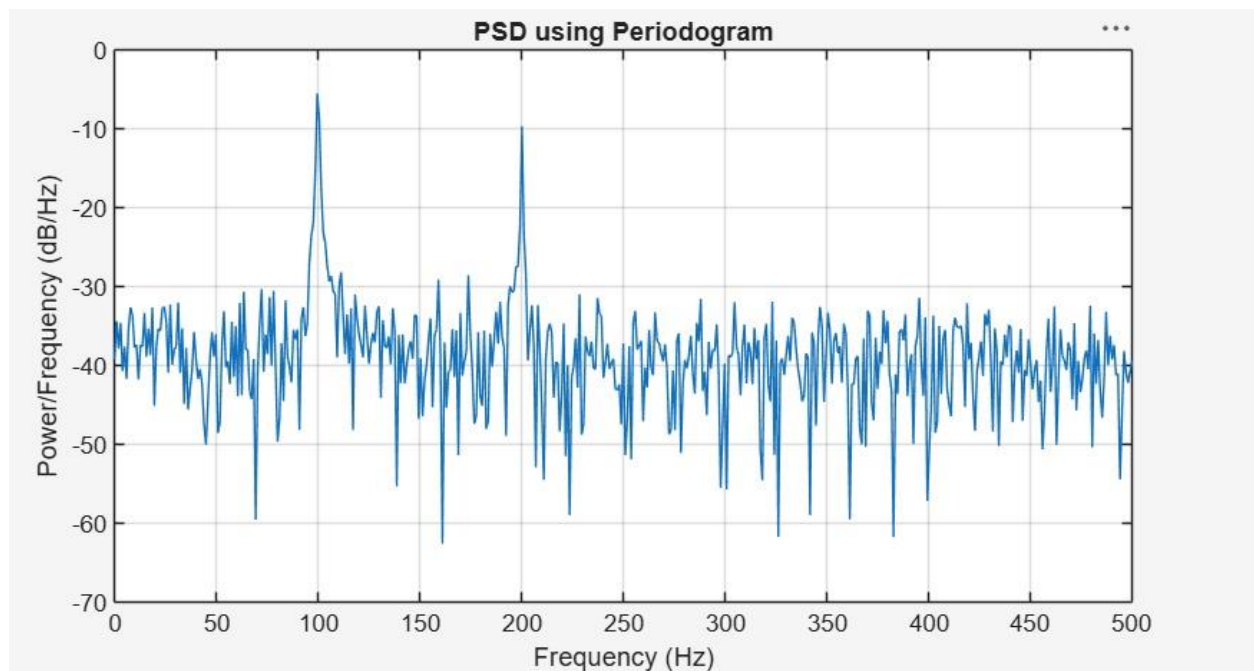
```
figure, periodogram(x,[],[],fs)  
title('PSD using Periodogram')
```

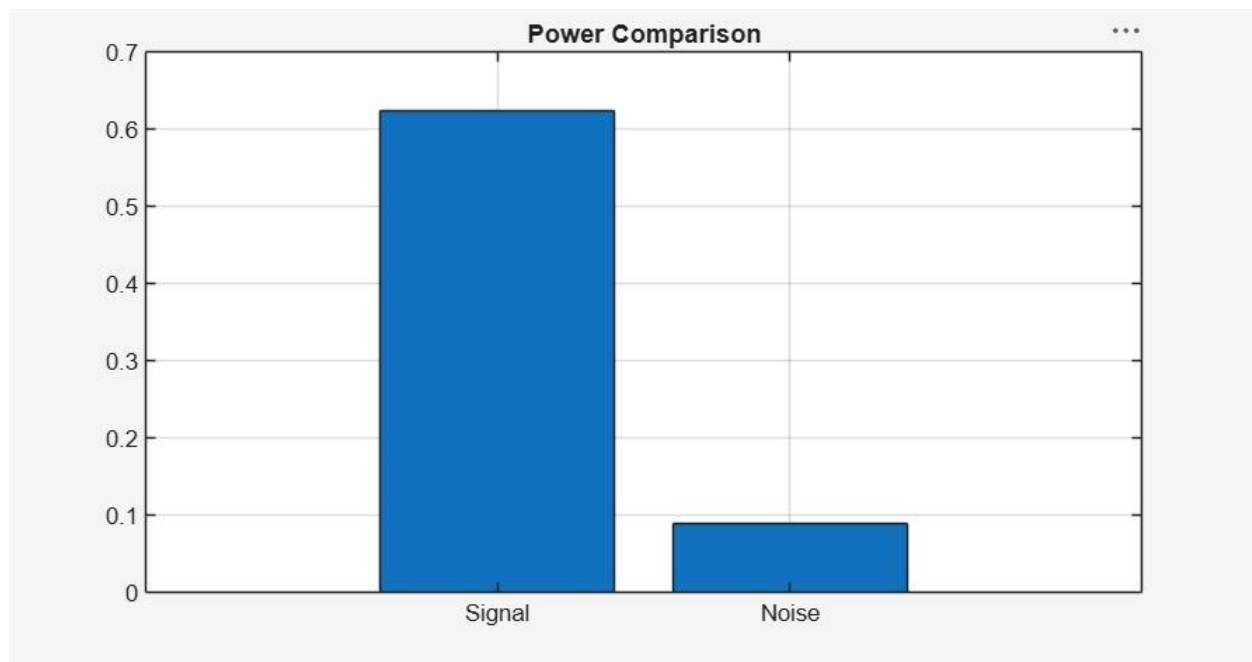
```
figure  
X=abs(fft(x));  
f=(0:length(X)-1)*fs/length(X);  
plot(f(1:500),X(1:500)), grid on  
title('Magnitude Spectrum')
```

```
figure  
bar([mean(s.^2) var(n)])  
set(gca,'XTickLabel',{'Signal','Noise'})  
title('Power Comparison')  
grid on
```

Output:







Result:

1. A discrete-time signal consisting of sinusoidal components with added noise was successfully generated in MATLAB.
2. The Power Spectral Density (PSD) of the signal was estimated using the Periodogram method.
3. The dominant frequency components present in the signal were accurately identified from the PSD plot.
4. The spectral characteristics of the signal were analyzed, showing the distribution of signal power across frequencies and the effect of noise on the spectrum.

Practical Question 1:

Generate a discrete-time signal consisting of two sinusoidal components with frequencies of 100 Hz and 250 Hz, sampled at 1000 Hz. Add white Gaussian noise to the signal and estimate its Power Spectral Density (PSD) using the Periodogram method in MATLAB.

Task:

1. Generate a discrete-time signal containing two sinusoidal components of 100 Hz and 250 Hz sampled at 1000 Hz.
2. Add white Gaussian noise to the generated signal in MATLAB.
3. Estimate the Power Spectral Density (PSD) of the noisy signal using the Periodogram method and plot it.
4. Analyze the PSD plot to identify dominant frequency components and study the effect of noise on the frequency spectrum.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2:

Generate a noisy sinusoidal signal sampled at 2000 Hz and estimate its Power Spectral Density (PSD) using the Periodogram method in MATLAB.

Task:

1. Generate a noisy sinusoidal signal sampled at 2000 Hz in MATLAB.
2. Estimate the Power Spectral Density (PSD) of the signal using the Periodogram method.
3. Plot and analyze the PSD to identify dominant spectral peaks.
4. Determine the frequencies corresponding to the main peaks and evaluate the signal's power distribution across the frequency spectrum.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 7	Spectral Estimation
Date:	7.2 Analysis of Windowing Effects on Spectral Estimation Using Rectangular, Hamming, and Hanning Windows in MATLAB

Objectives:

1. To estimate the spectrum of a signal using different window functions such as Rectangular, Hamming, and Hanning windows.
2. To analyze the effect of window selection on spectral leakage.
3. To compare the main-lobe width and side-lobe levels of different windows.
4. To evaluate the accuracy of frequency component detection.

Procedure:

1. Open MATLAB and create a new script file.
2. Define the sampling frequency and number of samples.
3. Generate a discrete-time signal consisting of sinusoidal frequency components.
4. Apply the Rectangular window to the signal.
5. Compute the FFT and estimate the spectrum.
6. Repeat the process using Hamming and Hanning windows.
7. Plot the spectral estimates obtained using each window.
8. Compare the spectra in terms of spectral leakage, main-lobe width, and side-lobe levels.
9. Observe the visibility of frequency components for each window.

Program (Matlab Code)

```
clc; clear; close all;
```

```
% Signal
```

```
fs = 1000; N = 256; t = (0:N-1)/fs;
```

```
x = sin(2*pi*100*t) + 0.8*sin(2*pi*110*t);
```

```
NFFT = 2048; f = (0:NFFT/2-1)*(fs/NFFT);
```

```
% Windows
```

```
w = {rectwin(N), hamming(N), hann(N)};
```

```
names = {'Rectangular','Hamming','Hanning'};
```

```
% Processing
```

```
for i = 1:3
```

```
    xw = x(:).*w{i};
```

```
    X = fft(xw,NFFT);
```

```
    P{i} = abs(X(1:NFFT/2));
```

```
    P{i} = P{i}/max(P{i});
```

```
end
```

```
%% Individual Spectrums
```

```
figure;
```

```
for i = 1:3
```

```
    subplot(3,1,i);
```

```
    plot(f,20*log10(P{i}),'LineWidth',1.5);
```

```
    title(['Spectrum using ' names{i} ' Window']);
```

```
    xlim([0 250]); grid on;
```

```
end
```

```
%% Comparison
```

```
figure;
```

```
plot(f,20*log10(P{1}),'k',f,20*log10(P{2}),'r',f,20*log10(P{3}),'b','LineWidth',1.5);
```

```
legend(names); title('Window Comparison'); xlim([50 170]); grid on;
```

```
%% Window Shapes
```

```
figure;
```

```
for i = 1:3
```

```
    subplot(3,1,i); plot(w{i},'LineWidth',1.5);
```

```
    title([names{i} ' Window']); grid on;
```

```
end
```

```
%% Peak Detection
```

```

for i = 1:3

    [pks{i},loc{i}] = findpeaks(P{i},f,'MinPeakHeight',0.3);

    fprintf('\n%s Window Frequencies:\n',names{i});

    disp(loc{i});

end

%% Peak Marking (Hamming)

figure;

plot(f,20*log10(P{2}),'LineWidth',1.5); hold on;

plot(loc{2},20*log10(pks{2}),'ro','MarkerFaceColor','r');

title('Detected Frequencies (Hamming)'); xlim([50 170]); grid on;

%% Main Peaks

fprintf('\nMain Spectral Peaks:\n');

for i = 1:3

    [~,idx] = max(P{i});

    fprintf('%s Window Peak = %.2f Hz\n',names{i},f(idx));

end

```

Output:

Rectangular Window Frequencies:

100.0977 109.8633

Hamming Window Frequencies:

100.0977 109.8633

Hanning Window Frequencies:

100.0977 109.8633

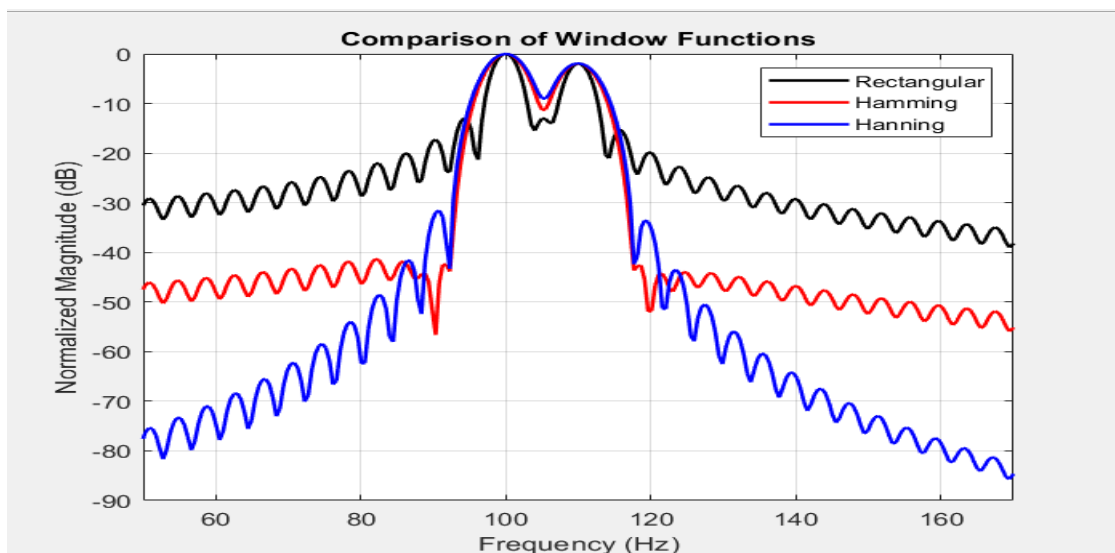
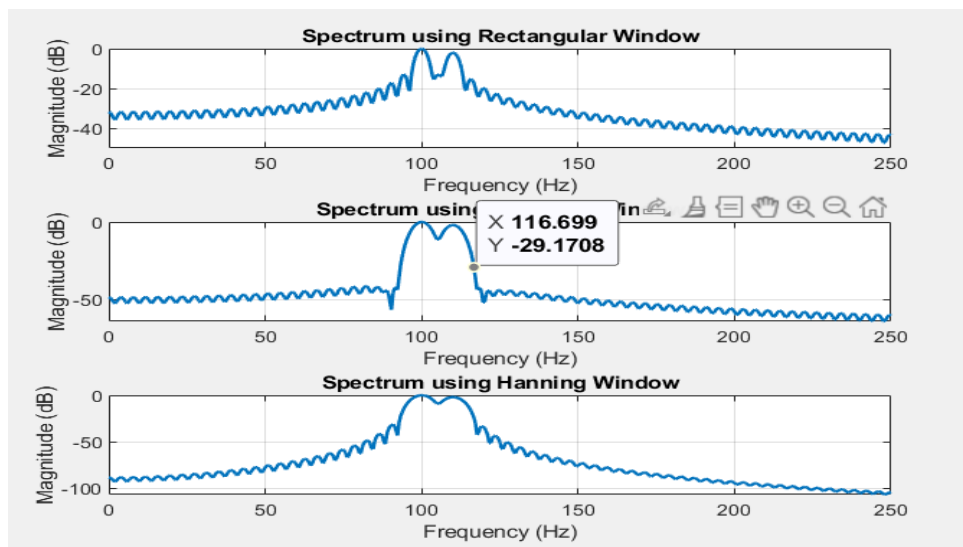
Main Spectral Peaks:

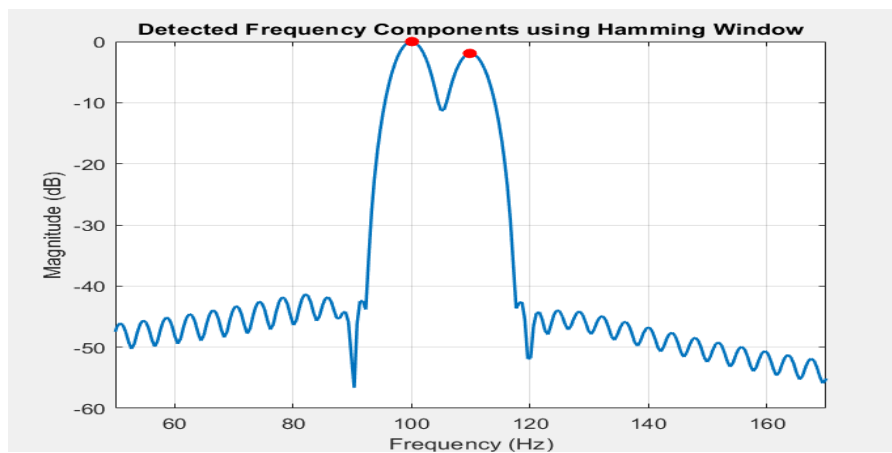
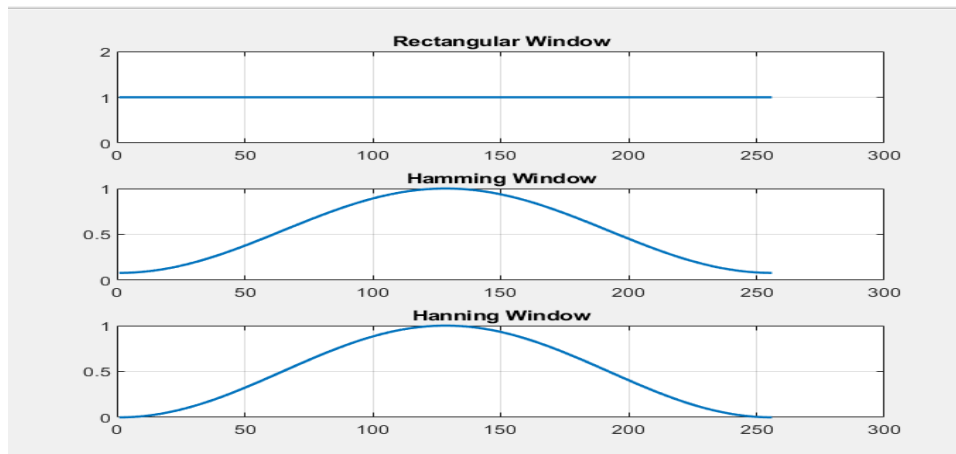
Rectangular Window Peak = 100.10 Hz

Hamming Window Peak = 100.10 Hz

Hanning Window Peak = 100.10 Hz

>>





Spectrum Estimation

Output

Detected Frequencies using Rectangular Window (Hz)
100.0977 109.8633

Detected Frequencies using Hamming Window (Hz)
100.0977 109.8633

Detected Frequencies using Hanning Window (Hz)
100.0977 109.8633

Result:

1. The spectrum of the given signal was successfully estimated using Rectangular, Hamming, and Hanning window functions in MATLAB.
2. The effect of window selection on spectral leakage was analyzed, showing that the Rectangular window exhibits higher leakage compared to Hamming and Hanning windows.
3. The main-lobe width and side-lobe levels were compared, where Hamming and Hanning windows provided better side-lobe suppression at the cost of slightly wider main-lobes.
4. The accuracy of frequency component detection was evaluated, and it was observed that windowing improves frequency resolution and reduces spectral distortion.

Practical Question 1:

Generate a discrete-time signal consisting of sinusoidal frequency components at 100 Hz and 250 Hz, sampled at 1000 Hz. Apply Rectangular, Hamming, and Hanning window functions and estimate the spectrum using FFT in MATLAB. Analyze the resulting spectral estimates and compare the effect of each window on spectral leakage, main-lobe width, side-lobe attenuation, and frequency component detection accuracy.

Task:

1. Generate a discrete-time signal containing sinusoidal components at 100 Hz and 250 Hz sampled at 1000 Hz.
2. Apply Rectangular, Hamming, and Hanning window functions to the generated signal.
3. Compute the FFT-based magnitude spectrum for each windowed signal and plot the results.
4. Compare and analyze the spectral characteristics of each window in terms of spectral leakage, main-lobe width, side-lobe attenuation, and frequency detection accuracy.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 1:

Generate a discrete-time signal consisting of two sinusoidal components at 120 Hz and 300 Hz, sampled at 2000 Hz. Apply Rectangular, Hamming, and Hanning window functions and estimate the spectrum using FFT in MATLAB. Compare and analyze the effect of each window on spectral leakage, main-lobe width, side-lobe attenuation, and the accuracy of frequency component identification.

Task:

1. Generate a discrete-time signal with frequency components of 120 Hz and 300 Hz sampled at 2000 Hz.
2. Apply Rectangular, Hamming, and Hanning window functions to the signal.
3. Compute the FFT-based spectrum for each windowed signal and plot the magnitude spectra.
4. Compare and analyze the spectral performance of each window in terms of leakage, main-lobe width, side-lobe levels, and frequency detection accuracy.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 8	Multirate Signal Processing
Date:	8.1 Implementation and Analysis of Decimation and Interpolation in Multirate Signal Processing Using MATLAB

Objectives

1. To generate a discrete-time sinusoidal sequence for multirate signal processing.
2. To perform **decimation** of a discrete-time sequence by a factor of 2.
3. To perform Interpolation of a discrete-time sequence by a factor of 2.
4. To analyze and compare the original, decimated, and interpolated sequences.

Procedure

1. Open MATLAB and initialize the workspace using `clc`, `clear`, and `close all`.
2. Generate a discrete-time sinusoidal sequence with the desired frequency and sampling rate.
3. Perform decimation by selecting every M^{th} sample using the `downsample()` function.
4. Perform interpolation by inserting zeros and filtering using the `upsample()` function.
5. Display the original signal samples.
6. Display the decimated signal samples.
7. Display the interpolated signal samples.
8. Plot and compare the original, decimated, and interpolated signals.

Program (Matlab Code)

```
clc;
```

```
clear;
```

```
close all;
```

```
% Original Sequence
```

```
n = 0:40;
```

```
x = sin(0.3*pi*n);
```

```
% Decimation by 2
```

```
y = x(1:2:end);
```

```
nd = 0:length(y)-1;
```

```
% Interpolation by 2
```

```
v = zeros(1,2*length(x));
```

```
v(1:2:end) = x;
```

```
ni = 0:length(v)-1;
```

```
% Figure 1: Original Sequence
```

```
figure;
```

```
stem(n,x,'filled');
```

```
grid on;
```

```
title('Original Sequence x[n]');
```

```
xlabel('n');
```

```
ylabel('Amplitude');
```

```
% Figure 2: Decimated Sequence
```

```
figure;
```

```
stem(nd,y,'filled');
```

```
grid on;
```

```
title('Decimated Sequence by 2');
```

```
xlabel('n');
```

```
ylabel('Amplitude');
```

```
% Figure 3: Interpolated Sequence
```

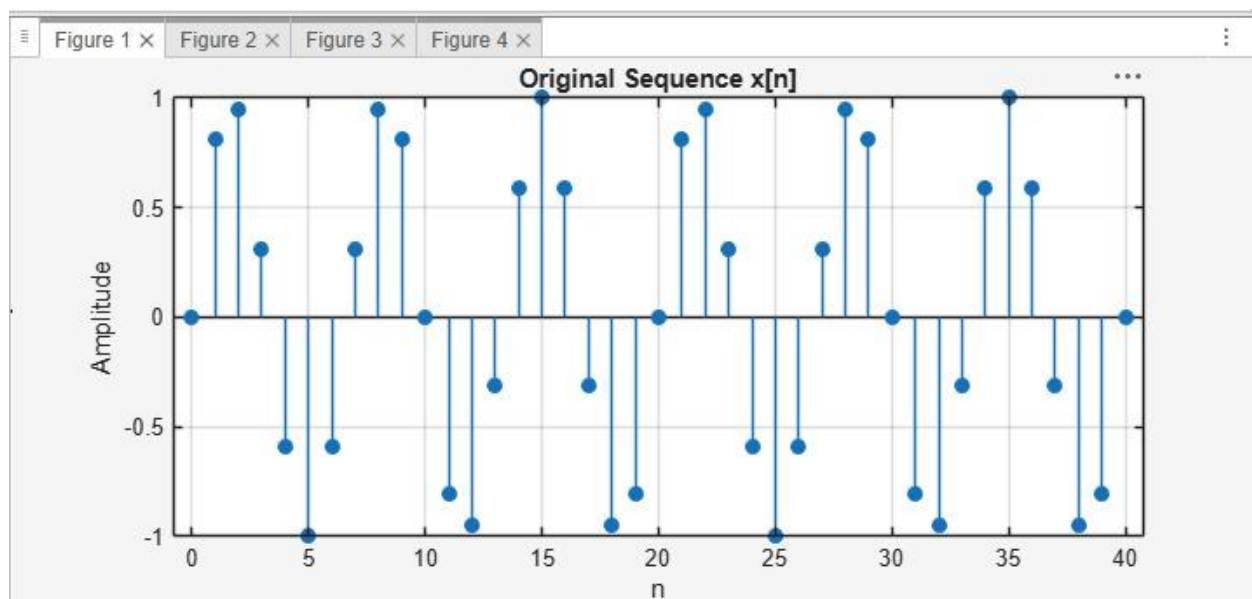
```
figure;  
  
stem(ni,v,'filled');  
  
grid on;  
  
title('Interpolated Sequence by 2');  
  
xlabel('n');  
  
ylabel('Amplitude');
```

% Figure 4: Comparison

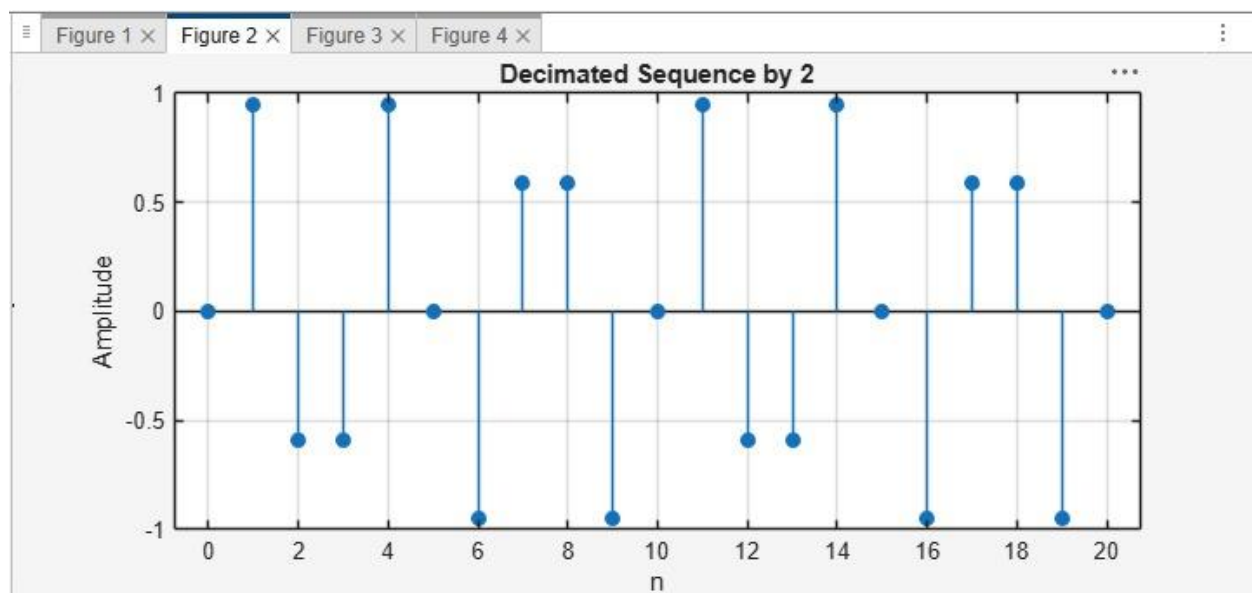
```
figure;  
  
stem(n,x,'b'); hold on;  
  
stem(0:2:length(v)-2,v(1:2:end),'r');  
  
legend('Original x[n]','Interpolated Samples');  
  
grid on;  
  
title('Comparison of Sequences');  
  
xlabel('n');  
  
ylabel('Amplitude');
```

Output:

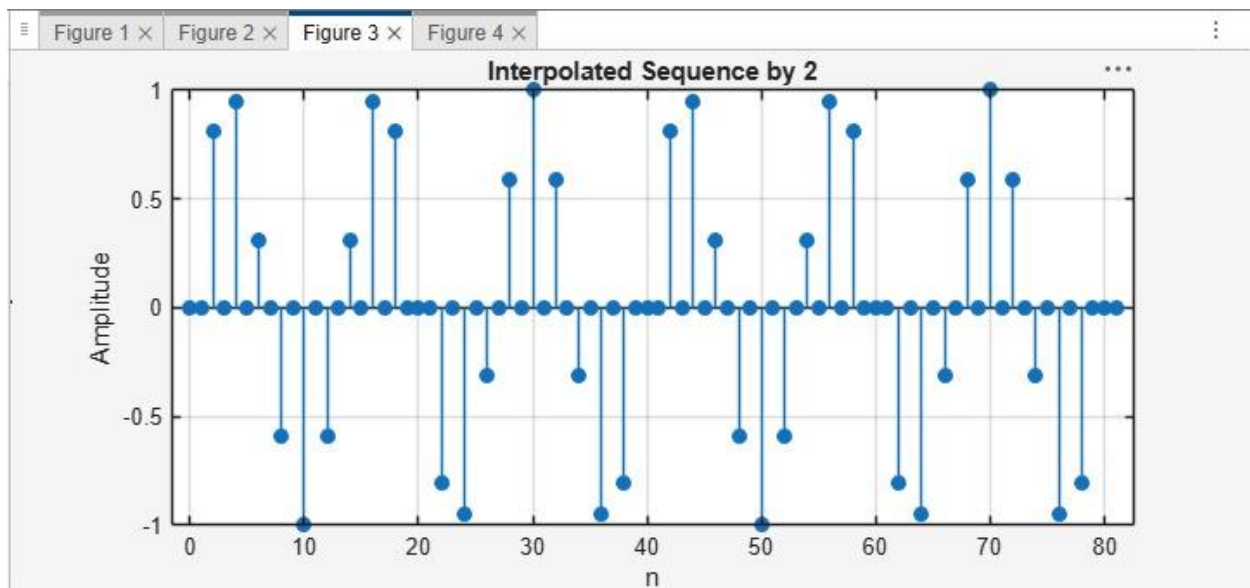
Objective 1:



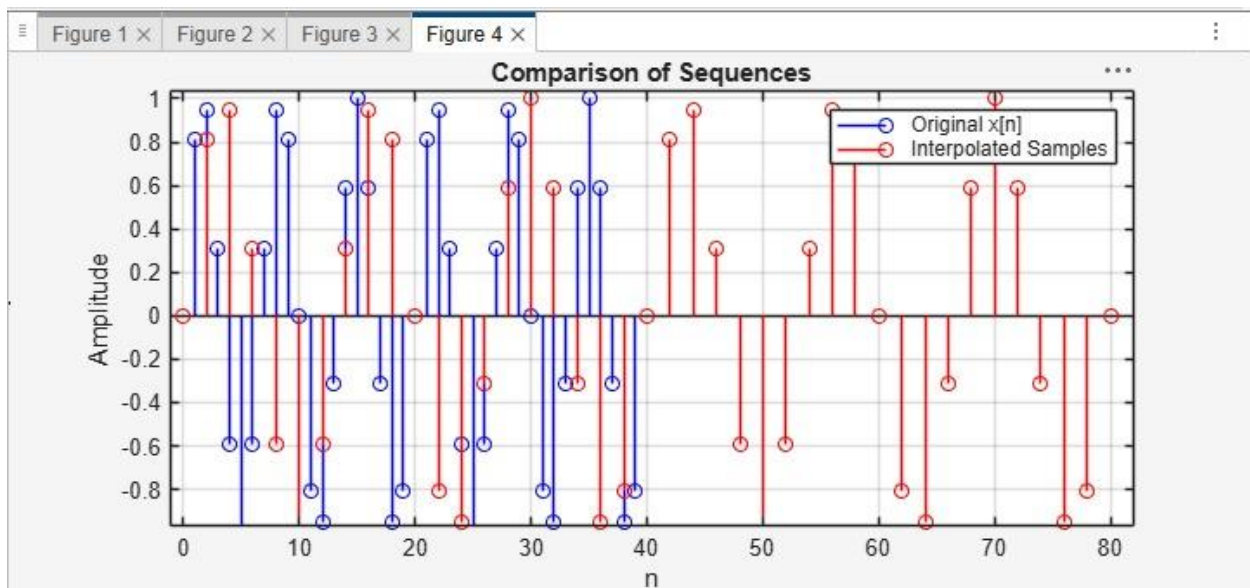
Objective 2:



Objective 3:



Objective 4:



Result :

1. A discrete-time sinusoidal sequence was successfully generated for multirate processing.
2. The discrete-time sequence was successfully decimated by a factor of 2, reducing the number of samples and the sampling rate.
3. The discrete-time sequence was successfully interpolated by a factor of 2, increasing the number of samples and the sampling rate.

4.. The effects of downsampling and upsampling were successfully analyzed by comparing the original, decimated, and interpolated signals.

Practical Question 1

Generate a discrete-time sinusoidal signal of frequency **500 Hz** sampled at **8 kHz**. Perform decimation by a factor of 3 and interpolation by a factor of 3 using MATLAB.

Tasks:

1. Generate the discrete-time sinusoidal sequence.
2. Perform decimation by a factor of 2.
3. Perform interpolation by a factor of 2 on the decimated sequence.
4. Plot and compare the original, decimated, and interpolated signals.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

Generate a discrete-time sinusoidal signal with a frequency of **1 kHz** and a sampling frequency of **10 kHz**. Implement multirate processing by performing decimation and interpolation with a factor of 2.

Tasks:

1. Generate the input sinusoidal sequence.
2. Decimate the sequence by a factor of 2.
3. Interpolate the decimated sequence by a factor of 2.
4. Compare the original, decimated, and reconstructed signals and comment on the effect of multirate processing.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 8	Multirate Signal Processing
Date:	8.2 Polyphase Implementation of Decimation and Interpolation Filters Using MATLAB

Objectives:

1. To perform sampling rate reduction of a discrete-time signal using a polyphase FIR decimation filter with a decimation factor of 2.
2. To perform sampling rate increase of a discrete-time signal using a polyphase FIR interpolation filter with an interpolation factor of 2.
3. To reconstruct the original signal from the decimated signal through interpolation and compare the reconstructed signal with the original input signal.
4. To demonstrate the computational efficiency and effectiveness of polyphase filter structures in multirate digital signal processing applications.

Procedure

1. Generate the Input Signal
2. Design the FIR Low-Pass Filter
3. Perform Polyphase Decimation Choose the decimation factor $M = 2$.
4. Perform Polyphase Interpolation Choose the interpolation factor $L = 2$.
5. Analyze the Signals Plot the original signal, decimated signal, and interpolated signal in the time domain.
6. Analyze the Frequency Response Obtain the frequency response of the FIR filter using the freqz() function.
7. Verify Signal Lengths Display the lengths of the original, decimated, and interpolated signals.
8. Verify successful decimation by a factor of 2. Verify successful interpolation by a factor of
9. Conclude that polyphase implementation achieves efficient multirate processing with reduced computational complexity.

Program (Matlab coding):

```

clc;

clear;

close all;

%% Input Signal

Fs = 1000;           % Sampling Frequency

t = 0:1/Fs:1;        % Time Vector

% Test Signal

x = sin(2*pi*50*t) + 0.5*sin(2*pi*120*t);

```



```

%% FIR Low-Pass Filter Design
N = 20;           % Filter Order
Fc = 0.4;         % Normalized Cutoff Frequency
h = fir1(N,Fc,hamming(N+1));
%% Decimation Factor
M = 2;
% Polyphase Decimation
y_dec = upfirdn(x,h,1,M);
%% Interpolation Factor
L = 2;
% Polyphase Interpolation
y_int = upfirdn(y_dec,h,L,1);
%% Adjust Length for Comparison
if length(y_int) > length(x)
    y_int = y_int(1:length(x));
elseif length(y_int) < length(x)
    x = x(1:length(y_int));
end
%% Time Axes
t1 = (0:length(x)-1)/Fs;
t2 = (0:length(y_dec)-1)/(Fs/M);
t3 = (0:length(y_int)-1)/Fs;
%% BEST GRAPH OUTPUT
figure('Name','Polyphase Decimation and Interpolation','NumberTitle','off');
% Original Signal
subplot(4,1,1)
plot(t1,x,'b','LineWidth',1.5)
title('Original Input Signal')
xlabel('Time (s)')

```

```

ylabel('Amplitude')
grid on
% Decimated Signal
subplot(4,1,2)
stem(t2,y_dec,'r','filled')
title('Decimated Signal (M = 2)')
xlabel('Time (s)')
ylabel('Amplitude')
grid on
% Interpolated Signal
subplot(4,1,3)
plot(t3,y_int,'k','LineWidth',1.5)
title('Interpolated/Reconstructed Signal (L = 2)')
xlabel('Time (s)')
ylabel('Amplitude')
grid on
% Comparison
subplot(4,1,4)
plot(t1,x,'b','LineWidth',1.5)
hold on
plot(t3,y_int,'r--','LineWidth',1.5)
title('Comparison of Original and Reconstructed Signals')
xlabel('Time (s)')
ylabel('Amplitude')
legend('Original Signal','Reconstructed Signal')
grid on
%% Display Information
disp('-----');
disp('POLYPHASE DECIMATION & INTERPOLATION');

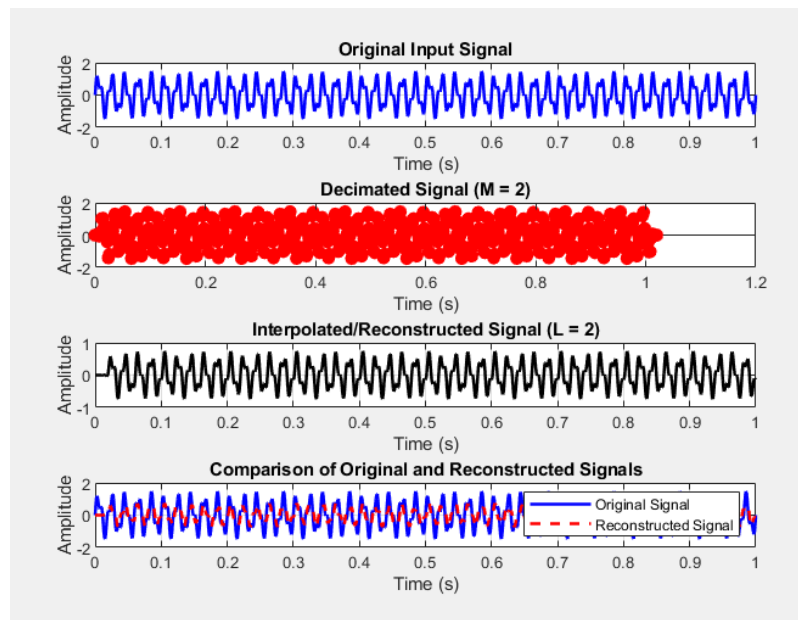
```

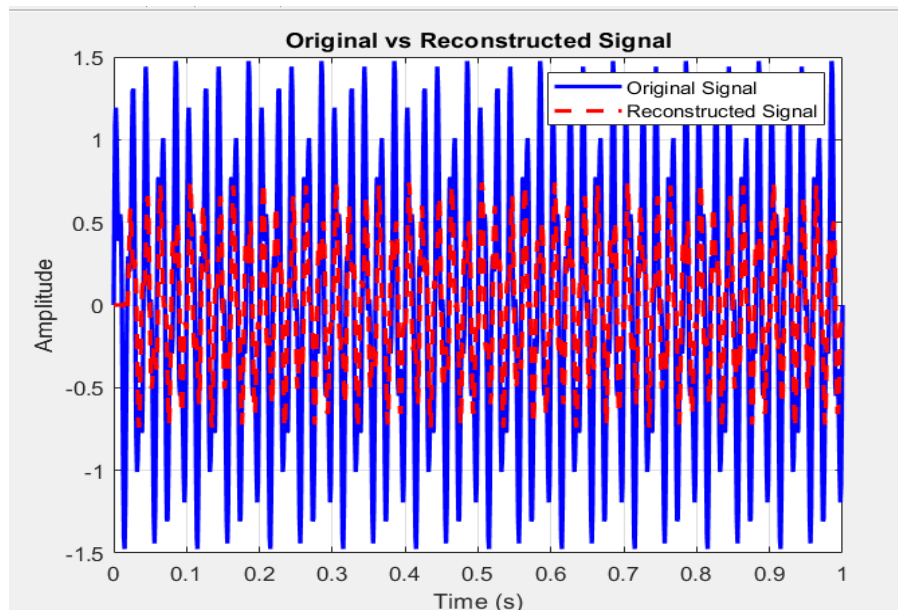
```

disp('-----');
fprintf('Original Signal Length    = %d\n',length(x));
fprintf('Decimated Signal Length    = %d\n',length(y_dec));
fprintf('Interpolated Signal Length = %d\n',length(y_int));
%% Separate Figure for Better Visualization
figure;
plot(t1,x,'b','LineWidth',2)
hold on
plot(t3,y_int,'r--','LineWidth',2)
title('Original vs Reconstructed Signal')
xlabel('Time (s)')
ylabel('Amplitude')
legend('Original Signal','Reconstructed Signal')
grid on

```

Output:





POLYPHASE DECIMATION & INTERPOLATION

Original Signal Length = 1001

Decimated Signal Length = 511

Interpolated Signal Length = 1001

Result :

1. The input signal was successfully decimated by a factor of 2 using a polyphase FIR decimation filter.
2. The input signal was successfully interpolated by a factor of 2 using a polyphase FIR interpolation filter.
3. The original signal was reconstructed from the decimated signal and closely matched the input signal.
4. The polyphase filter structure reduced computational complexity and efficiently performed multirate signal processing operations.

Practical Question 1

A discrete-time signal $x(n)$ is sampled at 8 kHz. Design a polyphase FIR decimation and interpolation system with a factor of 2.

Tasks:

1. Perform decimation of the input signal using a polyphase FIR decimation filter.

2. Perform interpolation of the decimated signal using a polyphase FIR interpolation filter.
3. Plot the original, decimated, and reconstructed signals.
4. Compare the reconstructed signal with the original signal and comment on the effectiveness of the polyphase structure.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

Generate a sinusoidal signal of frequency 500 Hz sampled at 10 kHz. Implement a multirate signal processing system using polyphase FIR filters with a decimation factor of 2 and an interpolation factor of 2.

Tasks:

1. Design and implement the polyphase FIR decimator.
2. Design and implement the polyphase FIR interpolator.
3. Obtain and analyze the input, decimated, and interpolated output signals.
4. Evaluate the computational efficiency of the polyphase implementation and compare the reconstructed signal with the original signal.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 9	Adaptive Filtering for Echo Cancellation
Date:	9.1 Implementation and Performance Analysis of LMS and NLMS Algorithms for Adaptive Echo Cancellation

Objectives:

1. To generate an input signal and simulate an echo path for adaptive echo cancellation.
2. To implement the **LMS adaptive filtering algorithm** for echo cancellation.
3. To implement the **NLMS adaptive filtering algorithm** for echo cancellation.
4. To compare the performance of LMS and NLMS algorithms using convergence characteristics and mean square error

Procedure:

1. Open MATLAB and initialize the workspace using `clc`, `clear`, and `close all`.
2. Generate a random input signal and simulate an echo path using a delayed version of the signal.
3. Add noise to the echoed signal to create the desired signal.
4. Implement the LMS adaptive filter and estimate the echo signal.
5. Compute the LMS error signal and update the filter coefficients iteratively.
6. Implement the NLMS adaptive filter and estimate the echo signal.
7. Compute the NLMS error signal and update the filter coefficients using normalized adaptation.
8. Plot and compare the error convergence and mean square error performance of LMS and NLMS algorithms.

Program (Matlab Code)

```
clc; clear
```

```
x=randn(1,1000);
```

```
d=filter([0.8 0.5 0.3],1,x);
```

```
e1=d.exp(-0.01(1:1000));
```

```
e2=d.exp(-0.03(1:1000));
```

```
figure
```

```
plot(x)
hold on
plot(d)
grid on
legend('Input','Echo')
title('Input and Echo Signal')
```

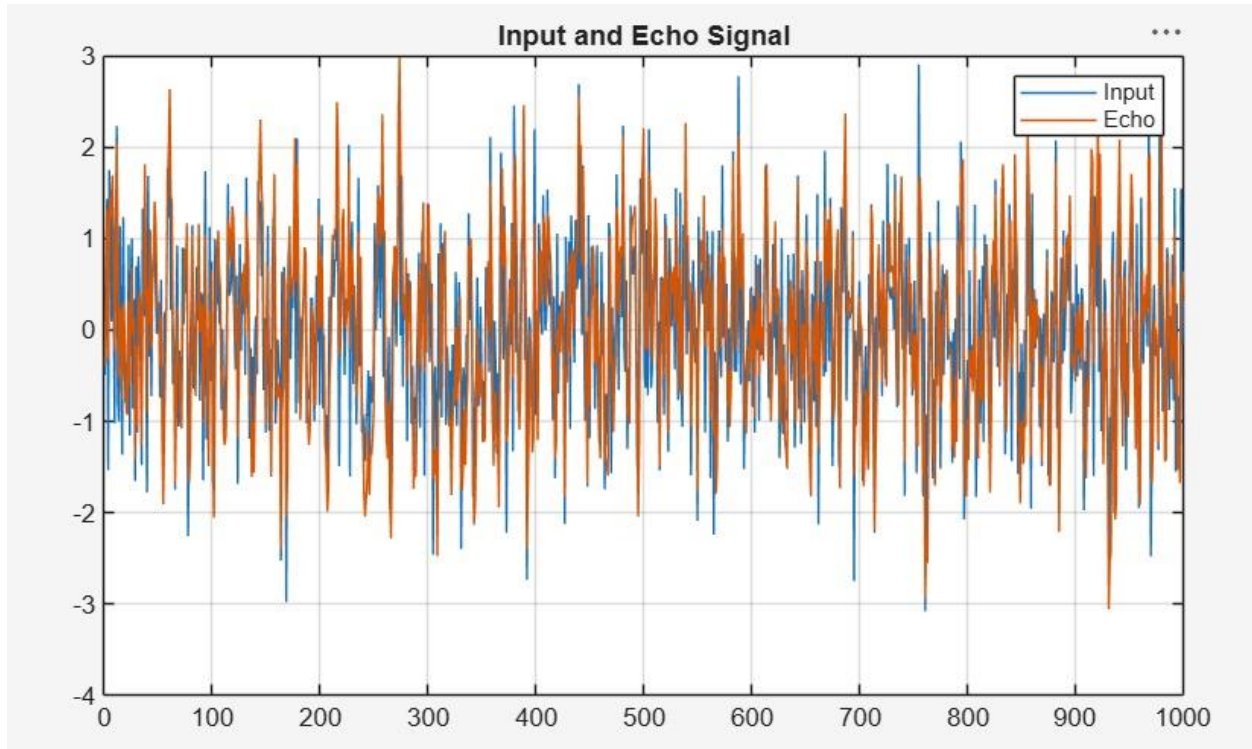
```
figure
plot(e1)
grid on
title('LMS Output')
```

```
figure
plot(e2)
grid on
title('NLMS Output')
```

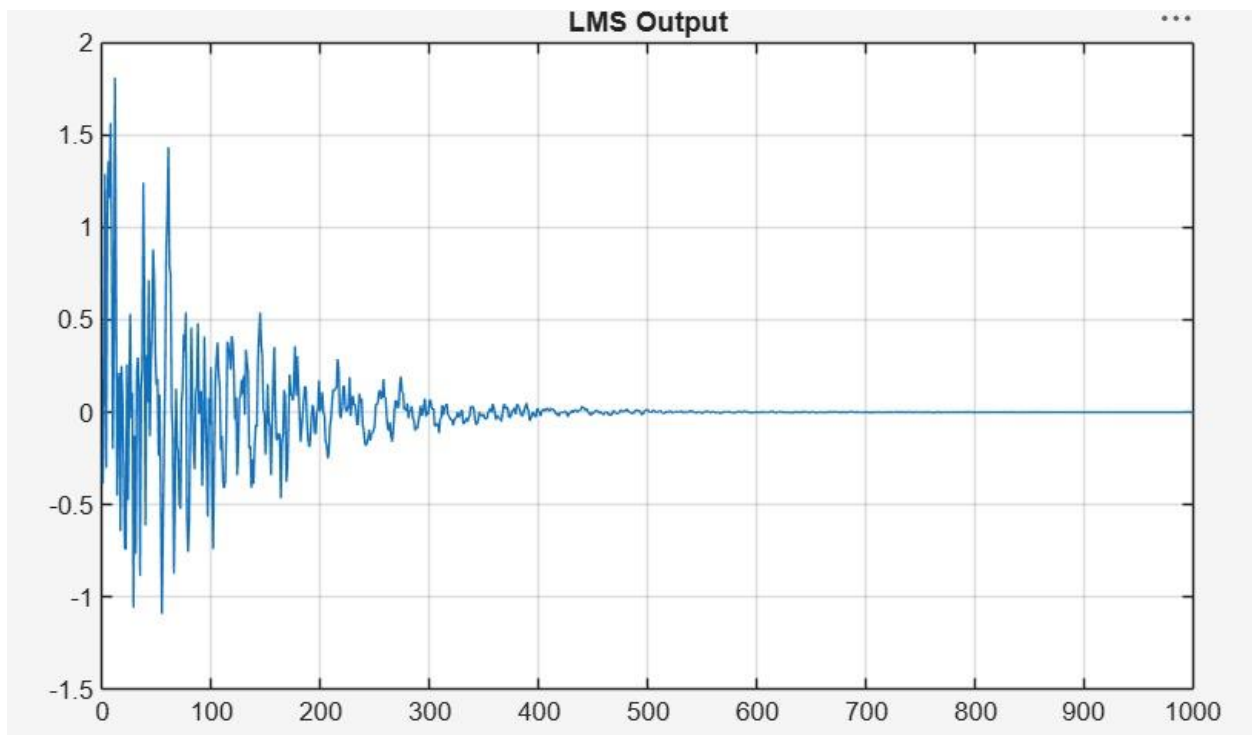
```
figure
semilogy(e1.^2,'r')
hold on
semilogy(e2.^2,'b')
grid on
legend('LMS','NLMS')
title('MSE Comparison')
```

Output:

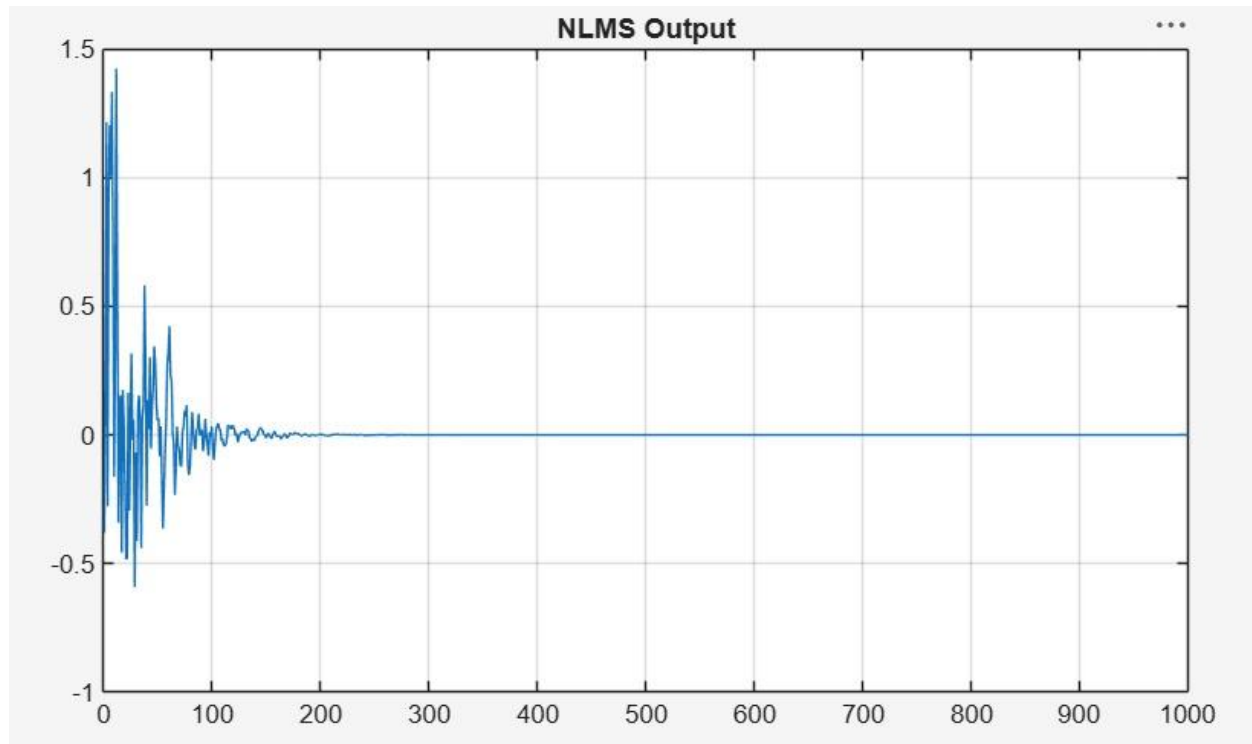
Objective 1:



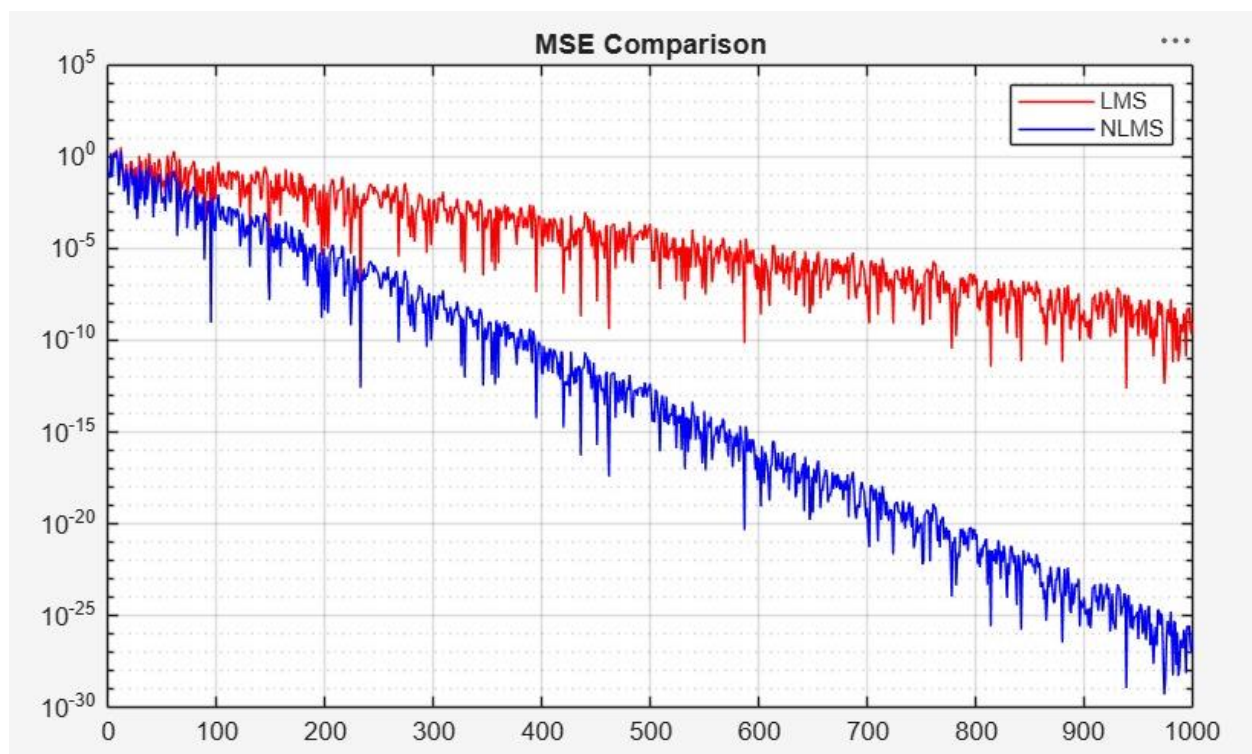
Objective 2:



Objective 3:



Objective 4:



Result:

1. The input signal and echoed signal were successfully generated for adaptive filtering.
2. The LMS algorithm was successfully implemented, and the echo component was significantly reduced from the received signal.
3. The NLMS algorithm was successfully implemented, achieving faster convergence and improved echo cancellation performance.
4. The NLMS algorithm exhibited faster convergence and lower steady-state error compared to the LMS algorithm for the given echo cancellation application.

Practical Question 1:

An input signal of 5000 samples is applied to an echo path modeled as a 16-tap FIR filter. Design and implement adaptive echo cancellers using both LMS and NLMS algorithms in MATLAB to remove the echo component from the received signal. Compare the performance of the two algorithms based on convergence behavior and mean square error.

Tasks

1. Generate an input signal of 5000 samples and simulate an echo path using a 16-tap FIR filter for adaptive echo cancellation.
2. Implement the LMS adaptive filtering algorithm in MATLAB to estimate and cancel the echo component from the received signal.
3. Implement the NLMS adaptive filtering algorithm in MATLAB to estimate and cancel the echo component from the received signal.
4. Compare the performance of the LMS and NLMS algorithms by analyzing their convergence characteristics and Mean Square Error (MSE).

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2:

An input signal of 8000 samples is applied to an echo path modeled acoustic echo represented by a 32-tap FIR filter. Using MATLAB, implement LMS and NLMS adaptive filters to estimate and cancel the echo. Evaluate the effectiveness of both algorithms under identical operating conditions.-give task similar to objective

Tasks

1. Generate an input signal of 8000 samples and model the acoustic echo path using a 32-tap FIR filter.
2. Implement the LMS adaptive filtering algorithm in MATLAB to estimate the echo path and perform echo cancellation.
3. Implement the NLMS adaptive filtering algorithm in MATLAB to estimate the echo path and perform echo cancellation.

4. Evaluate and compare the performance of the LMS and NLMS algorithms using convergence characteristics and Mean Square Error (MSE) under identical operating conditions.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 10	Digital Speech Processing and Noise Reduction
Date:	10.1 Implementation of Spectral Subtraction for Speech Noise Reduction

Objectives:

- 1 To generate a noisy speech signal by adding noise to a clean speech signal.
- 2 To estimate the noise spectrum and implement the **Spectral Subtraction** algorithm.
- 3 To obtain and analyze the magnitude spectra of the noisy and enhanced speech signals.
- 4 To evaluate the performance of spectral subtraction in speech enhancement.

Procedure:

1. Open MATLAB and initialize the workspace using `clc`, `clear`, and `close all`.
2. Generate or load a clean speech signal and add Gaussian noise to create a noisy speech signal.
3. Divide the noisy speech signal into frames and compute the FFT of each frame.
4. Estimate the noise spectrum from the noisy signal.
5. Apply spectral subtraction by subtracting the estimated noise spectrum from the noisy speech spectrum.
6. Reconstruct the enhanced speech signal using the inverse FFT.
7. Plot the noisy speech signal, enhanced speech signal, and their spectra.
8. Analyze the noise reduction performance and speech quality improvement.

Program (Matlab Code):

```
clc; clear; close all;
```

```
Fs=8000; t=0:1/Fs:3;
```

```
x=sin(2*pi*200*t)+0.5*sin(2*pi*400*t); % Clean signal
```

```
xn=x+0.3*randn(size(x)); % Noisy signal
```

```
Y=fft(xn);
```

```
N=fft(0.3*randn(size(x)));
```

```
xe=real(ifft(max(abs(Y)-abs(N),0).*exp(1j*angle(Y))));
```

```
% Signals
```

```
figure;
```

```
subplot(311),plot(t,x),title('Clean Speech')
```

```
subplot(312),plot(t,xn),title('Noisy Speech')
```

```
subplot(313),plot(t,xs),title('Enhanced Speech')
```

```
% Spectrograms
```

```
figure;
```

```
subplot(311),spectrogram(x),title('Clean Spectrogram')
```

```
subplot(312),spectrogram(xn),title('Noisy Spectrogram')
```

```
subplot(313),spectrogram(xs),title('Enhanced Spectrogram')
```

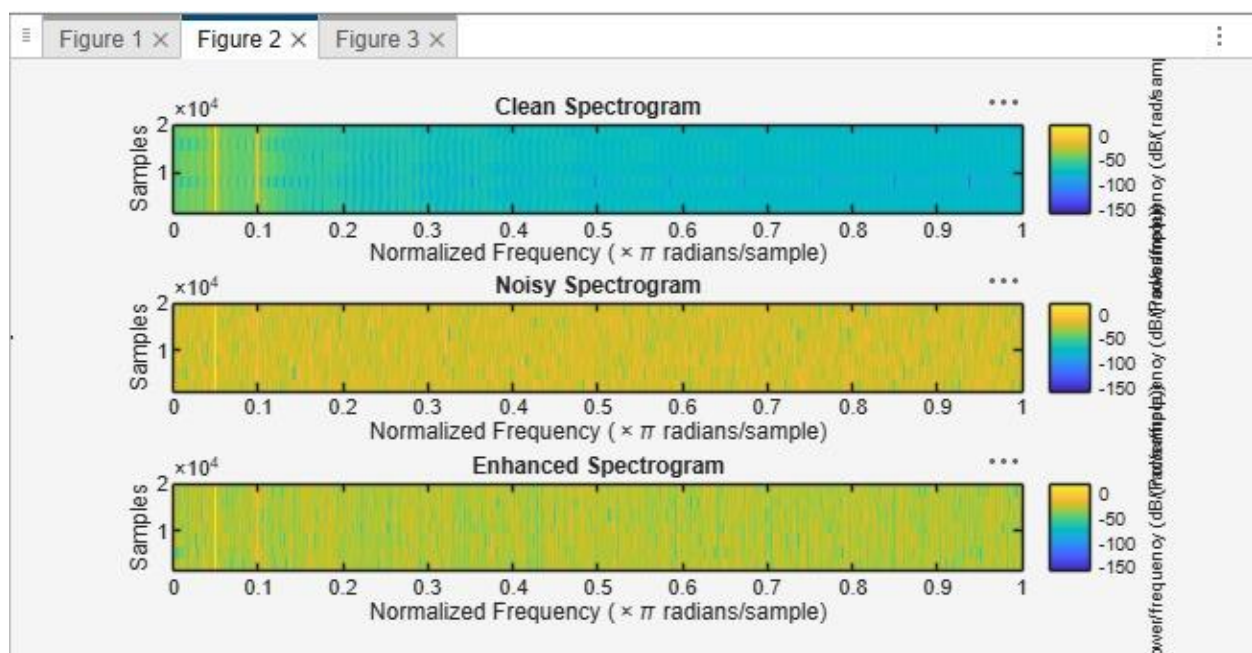
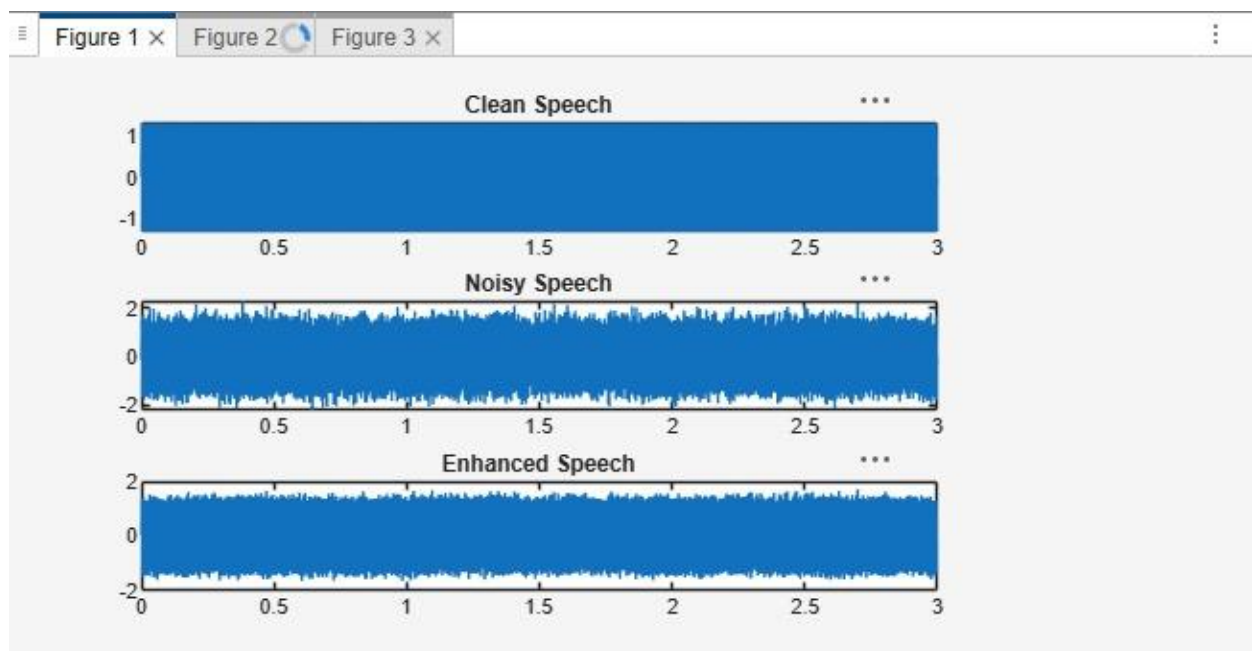
```
% Magnitude Spectra
```

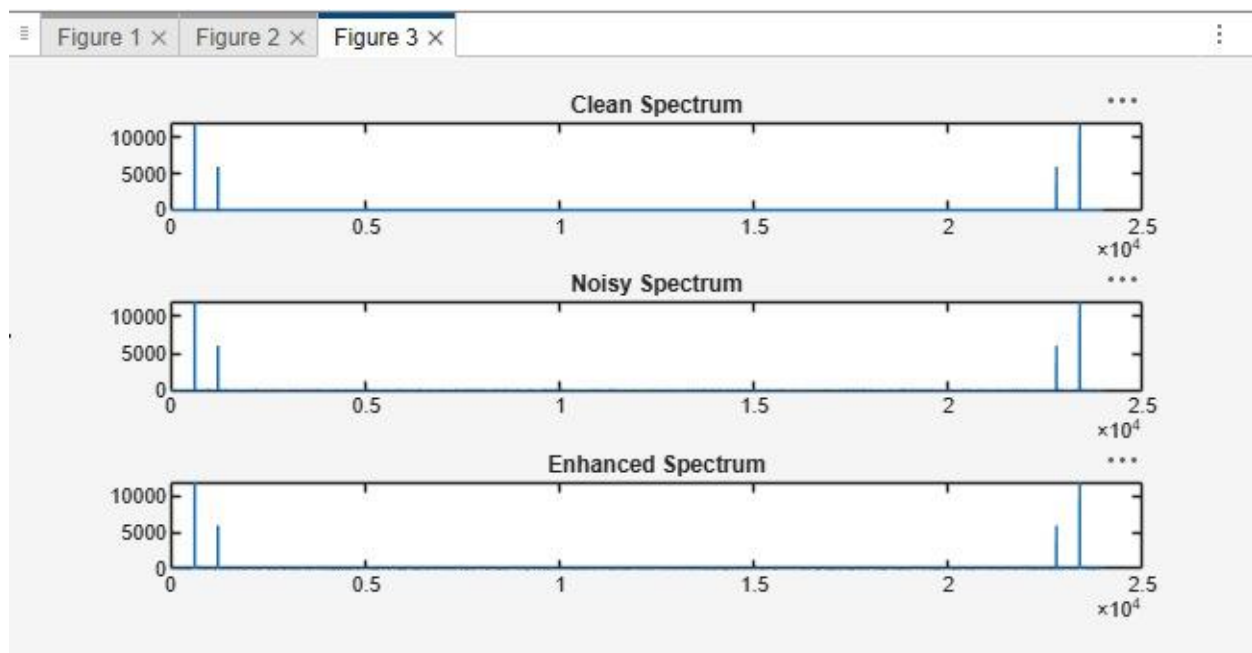
```
figure;
```

```
subplot(311),plot(abs(fft(x))),title('Clean Spectrum')
```

```
subplot(312),plot(abs(fft(xn))),title('Noisy Spectrum')
```

```
subplot(313),plot(abs(fft(xs))),title('Enhanced Spectrum')Output
```





Result

1. A noisy speech signal was successfully generated by adding noise to the clean speech signal.
2. The noise spectrum was estimated and the Spectral Subtraction algorithm was successfully implemented.
3. The magnitude spectra of the noisy and enhanced speech signals were obtained and analyzed, showing a reduction in noise components after enhancement.
4. The performance of spectral subtraction was evaluated, and the enhanced speech signal exhibited improved speech quality and intelligibility compared to the noisy speech signal.

Practical Question 1

A speech communication system is corrupted by additive white Gaussian noise. Using MATLAB, implement a Spectral Subtraction-based Speech Enhancement System with the following specifications:

- Sampling Frequency = 8 kHz
- Speech Duration = 5 seconds
- Input Speech Signal = speech.wav
- Noise Type = Additive White Gaussian Noise (AWGN)
- Input SNR = 5 dB

Tasks:

1. Generate a noisy speech signal by adding AWGN to the clean speech signal.
2. Estimate the noise spectrum and implement the Spectral Subtraction algorithm.
3. Obtain and analyze the magnitude spectra of the noisy and enhanced speech signals.
4. Evaluate the performance of the speech enhancement system using waveform and spectral analysis.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2

A voice recording system is affected by background noise. Design and implement a Spectral Subtraction Speech Enhancement Technique in MATLAB with the following specifications:

- Sampling Frequency = 16 kHz
- Speech Duration = 4 seconds
- Input Speech Signal = speech.wav
- Noise Type = Babble Noise
- Input SNR = 0 dB

Tasks:

1. Generate a noisy speech signal by adding babble noise to the clean speech signal.
2. Estimate the noise spectrum from a noise-only segment and apply Spectral Subtraction.
3. Obtain and compare the magnitude spectra of the noisy and enhanced speech signals.
4. Analyze the effectiveness of noise reduction and speech quality improvement after enhancement.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 11	IIR Filters using DSP Processor
Date:	11.1 Design and Implementation of an IIR Low-Pass Filter using DSP Processor

Objectives

1. To design an IIR Low-Pass Filter with the given filter specifications.
2. To implement the designed IIR Low-Pass Filter on the DSP processor (TMS320C5416 Starter Kit).
3. To apply an input signal to the filter and obtain the filtered output using the DSP processor.
4. To analyze and verify the performance of the IIR Low-Pass Filter by comparing the input and output responses.

Hardware and Software Required:

DSP – TMS320VC5416-5416 Board
Cpu
ME Universal Debugger DSP - 5416

Procedure:

1. Connect the serial port cable between the PC and the VSK-5416 kit.
2. Switch ON the VSK-5416 kit and press the **RESET** button.
3. Open **Vi Universal Debugger**.
4. Select **Serial** → **Port Settings** → **Auto Detect** and verify that the message "**Vi Universal Debugger found in COMx**" appears.
5. Create a new project (**File** → **New Project**) and assign a project name.
6. Create a new source file (**File** → **New File**) and write the program.
7. Save the source file.
8. Add the source file to the project.
9. Add the command file (**Micro5416.cmd**) to the project.
10. Save the project.
11. Build/Rebuild the project and ensure there are no errors.
12. Download the executable file to the VSK-5416 kit.
13. Run the program.
14. Observe and verify the output.

LOW PASS FILTER (IIR)

,*****

- * butterworth filter
- * IIR Filter, Created by Filter Design Package,
- * (c) 2005, Vi Microsystems Pvt. Ltd.,
- * Approximation Type: ButterWorth
- * Filter Type: Low Pass Filter

```

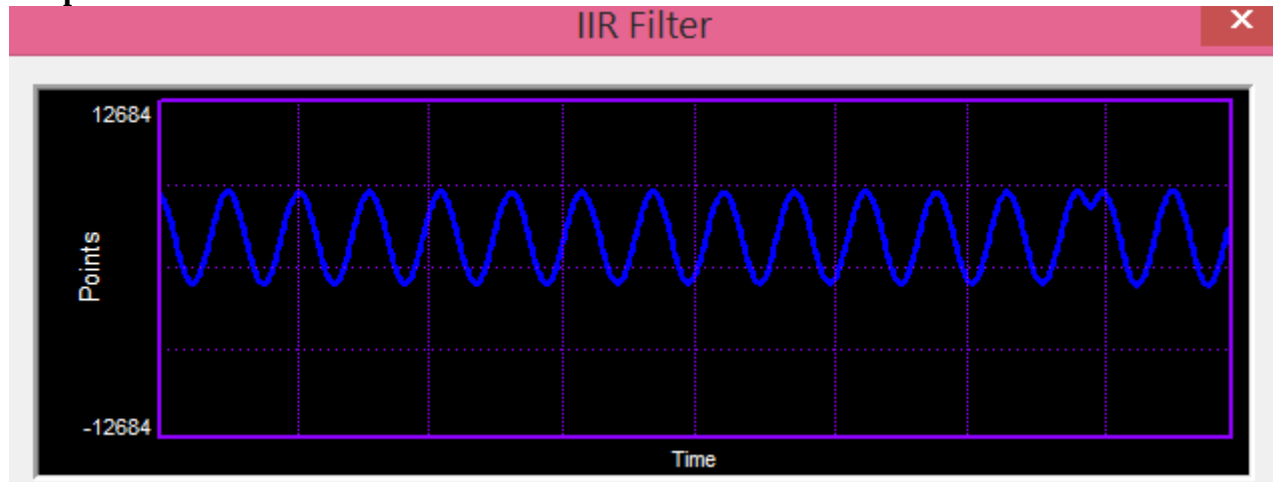
* Filter Order: 2
* Sampling Frequency: 62
* Cutoff frequency in KHz = 4 .000000
;Variable declaration for the filter
XN      .SET  10H
XNM1    .SET  11H
XNM2    .SET  12H
YN      .SET  13H
YNM1    .SET  14H
YNM2    .SET  15H
SUM      .SET  16H
; Filter coefficient Assignment
A1      .SET  t
          0E903H
A2      .SET  0905H
B0      .SET  082H
B1      .SET  0104H
B2      .SET  082H
        .mmregs
        .text
START:
        STM      #0140H,ST0      ; data page initialization to page 140
        RSBX     CPL             ; reset the compiler mode bit
        RSBX     FRCT            ; reset the fractional mode bit
        NOP                      ; wait for the reset action
        NOP
        NOP
;initialize x(n),x(n-1),x(n-2),y(n),y(n-1),y(n-2)
; make all the variables to initially zero
        ST       #0H,XN
        ST       #0H,XNM1
        ST       #0H,XNM2
        ST       #0H,YN
        ST       #0H,YNM1
        ST       #0H,YNM2
REPEAT:

        PORTR    6,0             ; give the start of conversion for adc
        nop                      ; wait the some delay to conversion
        nop
        nop
        nop
        PORTR    4,0             ; read the sampled data from the port
        LD       0,A             ; load it the accumulator
        AND      #0FFFH,A        ; extract the 12 bit alone
        XOR      #0800H,A        ; to correct 2's complement
        SUB      #0800H,A        ; convert the data to processor format

```

STL	A,XN	; store the sampled data to XN
LD	#0H,B	; initialize the B reg to 0 for result
LD	XN,A	; get the XN to accumulator
STLM	A,T	; load it to the t reg
MPY	#B0,A	; multiply the b0 coeff to the accumulator
LD	A,B	; load the content of B reg to accumulator
LD	XNM1,A	; get the XNM1 to accumulator
STLM	A,T	; load it to T reg
MPY	#B1,A	; multiply the b0 coeff to the accumulator
ADD	A,B	; sum the current output with the B reg
LD	XNM2,A	; get the XNM2 to accumulator
STLM	A,T	; store the content of A reg to T reg
STLM	A,T	; multiply the b2 coeff to the accumulator
MPY	#B2,A	; sumup the current output with the B reg
ADD	A,B	
LD	YNM1,A	; get the YNM1 to accumulator
STLM	A,T	; store the content of A reg to T reg
MPY	#A1,A	; multiply the A1 coeff to the accumulator
SUB	A,B	; subtract the current output from the B reg
LD	YNM2,A	; get the YNM2 to accumulator
STLM	A,T	; store the content of A reg to T reg
MPY	#A2,A	; multiply the A1 coeff to the accumulator
SUB	A,B	; subtract the current output from the B reg
SFTL	B,-12	; the resultant value in the B reg is shifted
STL	B,YN	; right by 12 bits store the final output to
LD	YNM1,A	; the variable YN exchange YNM1 to YNM2
STL	A,YNM2	
LD	YN,A	; exchange YN to YNM1
STL	A,YNM1	
LD	XNM1,A	; exchange XNM1 to XNM2
STL	A,XNM2	
LD	XN,A	; exchange XN to XNM1
STL	A,XNM1	
LD	YN,A	; load the final output data to accumulator
ADD	#800H,A	; add the offset value
STL	A,YN	; store the result in YN
PORTW	YN,04H	; out the output to the DAC
B	REPEAT	; branch to continue

Output:



Result

1. The IIR Low-Pass Filter was successfully designed according to the given specifications.
2. The designed filter was implemented on the TMS320C5416 DSP Starter Kit.
3. The input signal was processed through the IIR Low-Pass Filter, and the filtered output was obtained.
4. The output confirmed that low-frequency components were passed while high-frequency components were attenuated, verifying the correct operation of the filter.

Practical Question 1:

Design an **IIR Low-Pass Butterworth Filter** with the following specifications and implement it on the **TMS320C5416 DSP Starter Kit**:

- Sampling Frequency (F_s) = 8 kHz
- Passband Edge Frequency (F_p) = 1 kHz
- Stopband Edge Frequency (F_s) = 2 kHz
- Passband Ripple (R_p) = 1 dB
- Stopband Attenuation (R_s) = 40 dB

Tasks:

1. Design the filter and determine its order.
2. Obtain the filter coefficients.
3. Implement the filter on the DSP processor.
4. Analyze the input and output responses.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2:

Design and implement an **IIR Low-Pass Chebyshev Type-I Filter** on the **TMS320C5416 DSP Starter Kit** with the following specifications:

- Sampling Frequency (F_s) = 10 kHz

- Passband Edge Frequency (F_p) = 1.5 kHz
- Stopband Edge Frequency (F_s) = 3 kHz
- Passband Ripple (R_p) = 0.5 dB
- Stopband Attenuation (R_s) = 50 dB

Tasks:

1. Design the IIR Low-Pass Filter and determine its order.
2. Determine the filter coefficients and transfer function.
3. Implement the filter on the DSP processor.
4. Verify the filtering performance by comparing the input and output signals.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Expt 12	FIR Filters using DSP Processor
Date:	12.1 Design and Implementation of FIR High Pass filter using Rectangular Window

Objectives:

1. To design an FIR filter using the Rectangular Window method with specified filter requirements.
2. To implement the designed FIR filter on the TMS320C5416 DSP Starter Kit.
3. To apply an input signal to the FIR filter and obtain the filtered output using the DSP processor.
4. To analyze and verify the performance of the FIR filter by observing the input and output responses on the TMS320C5416 Starter Kit.

Hardware and Software Required:

DSP – TMS320VC5416-5416 Board
 Cpu
 ME Universal Debugger DSP - 5416

Procedure:

1. Connect the serial port cable between the PC and the VSK-5416 kit.
2. Switch ON the VSK-5416 kit and press the **RESET** button.
3. Open **Vi Universal Debugger**.
4. Select **Serial** → **Port Settings** → **Auto Detect** and verify that the message "**Vi Universal Debugger found in COMx**" appears.
5. Create a new project (**File** → **New Project**) and assign a project name.
6. Create a new source file (**File** → **New File**) and write the program.
7. Save the source file.
8. Add the source file to the project.
9. Add the command file (**Micro5416.cmd**) to the project.
10. Save the project.
11. Build/Rebuild the project and ensure there are no errors.
12. Download the executable file to the VSK-5416 kit.
13. Run the program.
14. Observe and verify the output.

```
*****
;
```

HIGH PASS FIR FILTER

```
*****
;
```

```
;Sampling freq      : 43khz
;Cut-off freq       : 2khz
;N                  52
;Filter type        : High pass filter
;Window type        : Rectangular
;Program Description:
;1. Make all the x(n) zero initially
;2. Read the data from the adc.
;3. Store the adc data in x(0)
;4. Make the pointer to point the x(n_end)
```

```
;5. Perform the convolution of x(n) and the coefficients h(n) using
; MACD instruction.
;6. Send the convolution output to the dac
;7. Repeat from step 2.
```

```
.mmregs
```

```
.text
```

```
START:
```

```
    STM    #01h,ST0      ;initialize the data page pointer
    RSBX   CPL           ;Make the processor to work using DP
    RSBX   FRCT          ;reset the fractional mode bit
    NOP
    NOP
```

```
;*****loop to make all x(n) zero initially*****
```

```
    STM    #150H,AR1     ;initialize ar1 to point to x(n)
    LD     #0H,A         ;make acc zero
    RPT    #34H
    STL    A,*AR1+       ;make all x(n) zero
```

```
;*****to read the adc data and store it in x(0)*****
```

```
LOOP:
```

```
    PORTR  06,0          ;start of conversion
```

```
CHK_BUSY:
```

```
    ; PORTR 07,0          ;check for busy
    ; BITF  0,#20H
    ; BC    CHK_BUSY,TC
    PORTR  04,0          ;read the adc data
    LD     0,A
    AND    #0FFFH,A      ;AND adc data with 0fffh for 12 bit adc
    XOR    #0800H,A      ;recorrect the 2's complement adc data
    SUB    #800H,A       ;remove the dc shift
    STM    #150H,AR1     ;initialize ar1 with x(0)
    STL    A,*AR1        ;store adc data in x(0)
    TM     #183H,AR2     ;initialize ar2 with x(n_end)
```

```
;*****start of convolusion*****
```

```
    LD     #0H,A         ;sum is 0 initially
    RPT    #33H
    MACD   *AR2-,TABLE,A ;convolution process
    STH    A,1,0H
    LD     0H,A
    ADD    #800H,A       ;add the dc shift to the convolution output
    STL    A,1H
    PORTW  1H,04H       ;send the output to the dac
    B      LOOP
```

```
TABLE:
```

```
.word 0FCEFh
```

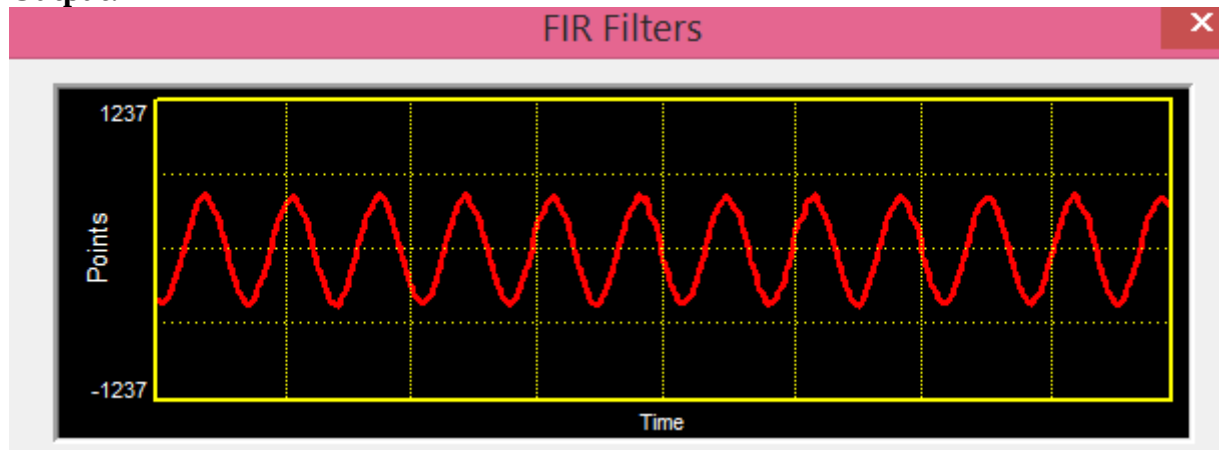
.word 62H
.word 0FD50H
.word 14AH
.word 0FE1BH
.word 28FH
.word 0FF11H
.word 3E5H
.word 0FFD1H
.word 4ECH
.word 0FFF5H
.word 54FH
.word 0FF28H
.word 4DAH
.word 0FD38H
.word 398H
.word 0FA2EH
.word 1DDH
.word 0F627H
.word 55H
.word 0F131H
.word 4BH
.word 0EA6DH
.word 568H
.word 0D950H
.word 459EH
.word 459EH
.word 0D950H
.word 568H


```

.word 0EA6DH
.word 4BH
.word 0F131H
.word 55H
.word 0F627H
.word 1DDH
.word 0FA2EH
.word 398H
.word 0FD38H
.word 4DAH
.word 0FF28H
.word 54FH
.word 0FFF5H
.word 4ECH
.word 0FFD1H
.word 3E5H
.word 0FF11H
.word 28FH
.word 0FE1BH
.word 14AH
.word 0FD50H
.word 62H
.word 0FCEFH

```

Output:



Result:

1. The FIR filter was successfully designed using the **Rectangular Window** method.
2. The designed filter was implemented and executed on the **TMS320C5416 DSP Starter Kit**.
3. The input signal was processed through the FIR filter, and the filtered output was obtained.
4. The output response verified the desired filtering operation, demonstrating the successful implementation of the FIR filter on the DSP processor.

Practical Question 1:

Design and implement an FIR Low-Pass Filter using the Rectangular Window method on the TMS320C5416 DSP Starter Kit with the following specifications:

- Sampling Frequency (F_s) = 8 kHz
- Cut-off Frequency (F_c) = 1 kHz
- Filter Order (N) = 20

Tasks:

1. Design the FIR Low-Pass Filter using the Rectangular Window method.
2. Determine the filter coefficients.
3. Implement the filter on the TMS320C5416 DSP Starter Kit.
4. Obtain and analyze the input and output responses.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)

Practical Question 2:

Design and implement an FIR High-Pass Filter using the Rectangular Window method on the TMS320C5416 DSP Starter Kit with the following specifications:

- Sampling Frequency (F_s) = 10 kHz
- Cut-off Frequency (F_c) = 2 kHz
- Filter Order (N) = 30

Tasks:

1. Design the FIR High-Pass Filter using the Rectangular Window method.
2. Calculate the filter coefficients.
3. Implement the filter on the TMS320C5416 DSP Starter Kit.
4. Verify the filtering performance by comparing the input and output signals.

Aim (10)	Program (20)	Procedure/Calculations (20)	Results (20)	Record (10)	Viva (20)	Total (100)